ФП 2.0 [1][2][3]

осень 2026

Содержание

1 Воспоминания о ФП **5**

1.1 Термы и редукция . . . 5
1.2 Типы . . . 5
1.3 Функции в Haskell . . . 8
1.4 Данные в Haskell . . . 9
1.5 Классы типов в Haskell . . . 11
1.6 Монады в Haskell . . . 12

2 Параметрический полиморфизм **15**

2.1 Параметрический полиморфизм в языке . . . 15
2.1.1 Полиморфизм функций . . . 15
2.1.2 First-class polymorphism . . . 17
2.1.3 Полиморфизм данных . . . 19
2.1.4 Higher-order/kinded polymorphism . . . 21
2.1.5 Обобщённые алгебраические типы данных (GADTs) . . . 22
2.1.6 Структуры на уровне типов, data promotion . . . 23
2.1.7 Факультативно: эмуляция типовых абстракций и аппликаций (`Proxy`) . . . 26
2.2 Реализация параметрического полиморфизма . . . 28
2.2.1 Мономорфизация . . . 28
2.2.2 Стирание типа . . . 29
2.2.3 Гибридный подход . . . 30
2.2.4 Использование виртуальной таблицы свойств типов . . . 30
2.2.5 Сравнение подходов . . . 31
2.3 Полиморфизм по конвенции вызова . . . 32
2.3.1 Разновидности runtime представлений в Haskell . . . 33
2.3.2 Классификация значений по runtime представлению . . . 34
2.3.3 Representation polymorphism . . . 35

3 Специальный (ad-hoc) полиморфизм **36**

3.1 Классы типов в языке . . . 37
3.1.1 Словари . . . 37
3.1.2 Неявные аргументы . . . 39

[1] Автор Андрей Стоян (`andrey.stoyan.csam@gmail.com`).

[2] Спасибо Илье Колегову за первое внимательное прочтение и кучу комментариев. Спасибо моим студентам, которые своим интересом к предмету сделали эту работу возможной.

[3] Версия от 15 октября 2026 г., 09:01.

3.1.3 Вывод инстансов . . . 40
3.1.4 Построение типа по значению . . . 42
3.1.5 Имплиситы и когерентность . . . 43
3.1.6 Правила (rules) и специализация . . . 45
3.1.7 Отступление: дефункционализация . . . 46
3.1.8 Эмуляция полиморфизма высших порядков . . . 46
3.2 Семейства . . . 48
3.2.1 Data families . . . 48
3.2.2 Synonym families . . . 49
3.2.3 Инъективные семейства . . . 50
3.2.4 Семейства первого класса . . . 50
3.3 Кайнд `Constraint` . . . 52
3.4 Использование ad-hoc полиморфизма . . . 54
3.4.1 Сериализация . . . 54
3.4.2 Экзистенциальные типы . . . 55
3.4.3 Разрешение имён . . . 56
3.4.4 Несинтаксические типовые эквивалентности, System FC . . . 57
3.4.5 Коерции и роли . . . 58
3.4.6 Type reflection . . . 60
3.4.7 Data reflection . . . 61
3.4.8 Открытые структуры . . . 62
3.4.9 Исключения и открытая иерархия . . . 63
3.4.10 Легковесные частичные стек-трейсы . . . 65
3.4.11 Кастомизируемые ошибки типизации . . . 65
3.4.12 Вычисление до нормальной формы . . . 66

4 Типы данных **68**
4.1 Вариантность . . . 68
4.1.1 Функторы . . . 68
4.1.2 Знаки позиций в типе . . . 70
4.1.3 Ещё функторы . . . 71
4.1.4 ООП и подтипизация . . . 72
4.2 Изоморфизм . . . 72
4.2.1 Кардинальность: суммы, произведения, экспоненты . . . 73
4.2.2 Алгебраическое представление типа . . . 75
4.3 Рекурсивные типы . . . 77
4.3.1 Просто список . . . 78
4.3.2 Неподвижная точка функтора . . . 80
4.3.3 Схемы рекурсии . . . 81
4.4 Всё через свёртки . . . 84
4.4.1 Deforestation & list fusion . . . 86

4.4.2 Visitor pattern 89

Введение

Многие сложные концепции в дизайне языков и программ могут быть поняты как частные случаи некоторых простых фундаментальных принципов, которые, как правило, считаются общеизвестным фольклором, не требующим дополнительных пояснений. Однако, сложность в том, что эти знания рассеяны по книгам, статьям и "культовым" блог-постам, и требуется довольно много времени и сил для восстановления целостной картины.

Цель данного курса — собрать в одном месте такие фольклорные знания и организовать их в некоторую систему. Курс будет явным образом опираться на классические работы, исследующие принципы построения языков, и помогать в их изучении. Просмотр упоминаемых статей является важной частью самостоятельной работы в рамках курса.

Под функциональным программированием, вынесенным даже в заголовок курса, понимается трепетное отношение к понятию эффекта, которое в ФП, в отличие от других школ мысли, не считается аксиоматической данностью, но предметом для изучения, сознательного конструирования и аккуратного обращения. Этот подход оказывается очень полезным для изучения языков, построения выразительных языковых конструкций, а также является основой для продуктивного стиля программирования. Кроме того, функциональные языки сравнительно просты, в результате чего новые идеи и подходы нередко зарождаются в них и распространяются далее.

В качестве основного языка курса выбран Haskell, так как он, с одной стороны, воплощает в себе многие концепции, часто доведённые до некоторого логического завершения, и достаточно выразителен для кодирования других. С другой стороны, всё ещё является прикладным промышленным языком программирования.

В связи с широтой контекста, данный курс не всегда является глубоким. Так, детали реализации в GHC или теоретико-категорные основания вещей могут даваться в общем виде и без конкретики. В то же время, в плоскости языкового дизайна через оптику функционального программирования курс пытается быть максимально подробным.

Таким образом, данный курс может быть полезен тем, кто интересуется дизайном языков и красивыми обобщениями программистских концепций, хочет улучшить свои навыки проектирования API или планирует вести практическую деятельность на функциональных языках.

Пререквизитом к прохождению курса является знание основ функционального программирования: алгебраических типов данных, паттерн-матчинга, свёрток, параметрического полиморфизма, классов типов, базовых монад. Дополнительно будет полезным умение читать типовые дроби, знакомство с полиморфным λ-исчислением и кодированием Чёрча.

1 Воспоминания о ФП

В этом разделе мы вспомним основные концепции функционального программирования и языка Haskell.

1.1 Термы и редукция

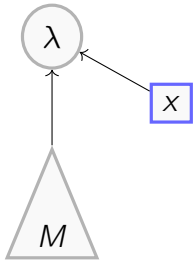
В ФП программы представляют собой выражения. Выполнение программ — редукция таких выражений до более “простых”. Выражения можно представлять как в виде линейной записи символов, так и в виде дерева, для понимания которого не требуется знания вспомогательных правил ассоциативности и проч.

Простейший функциональный язык — λ-исчисление. Выражения в нём называются λ-термами, которые состоят из вершин трёх видов (V — множество валидных идентификаторов, Λ — множество λ-термов):

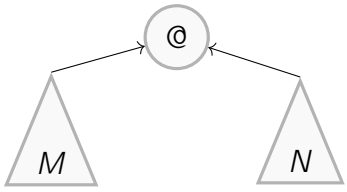
Переменные $x \in \Lambda$, если $x \in V$



Абстракция $(\lambda x.\, M) \in \Lambda$, если $x \in V, M \in \Lambda$



Аппликация $(M\ N) \in \Lambda$, если $M \in \Lambda, N \in \Lambda$



В произвольном выражении можно заменить некоторый его фрагмент на формальный параметр, который должен быть задекларирован выше по дереву с помощью специальной вершины λ. Вместо формального параметра можно в дальнейшем подставлять различные конкретные параметры с помощью вершины-аппликации @, то есть переиспользовать это выражение для различных целей (например, см. рис. 1). Редукция как раз определяется как следующее правило переписывания: ищется применение λ-функции к аргументу и в её тело осуществляется подстановка аргумента во все свободные вхождения переменной, связанной лямбдой (рис. 2).

1.2 Типы

Программное обеспечение — это сложно. Поэтому постоянно и неизбежно в программах возникают ошибки. Их можно искать, в том числе, статически, то есть без запуска программы. Одним из видов статического анализа является анализ типов.

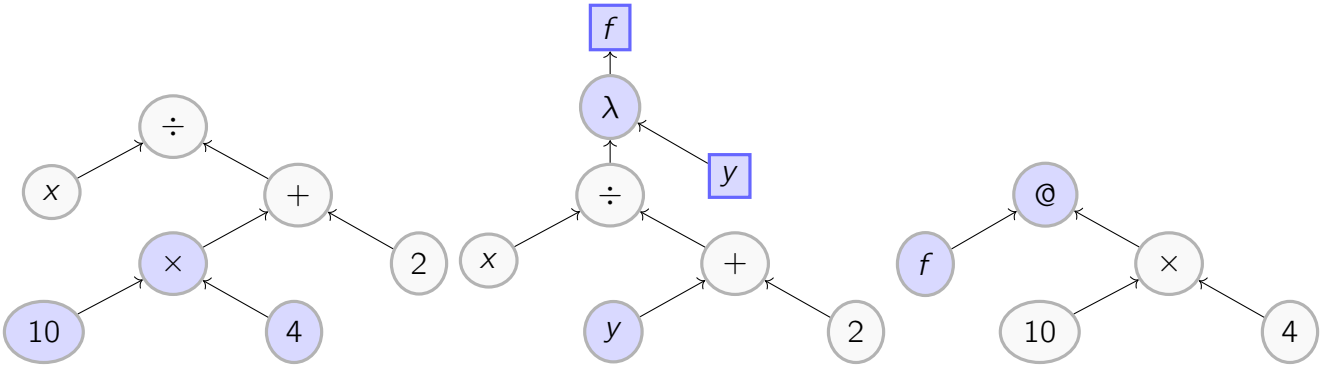


Рис. 1: Выражение с помощью λ вершины преобразуется в функцию одного аргумента.

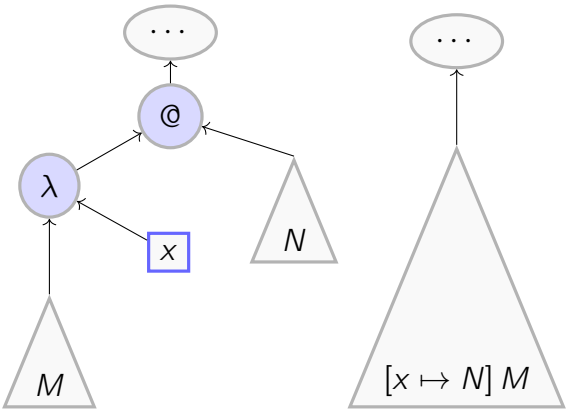


Рис. 2: Редукция переписывает дерево путём подстановки конкретного аргумента вместо формального параметра.

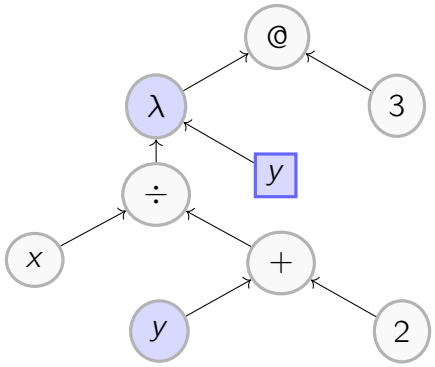


Рис. 3: Дерево соответствующее выражению $(\lambda y.\, x \div (y + 2))\ 3$.

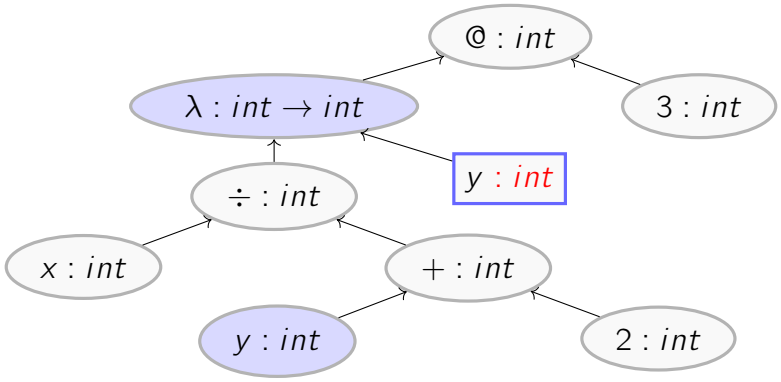


Рис. 4: Дерево выражения $(\lambda y.\, x \div (y + 2))\ 3$ после приписывания типовых меток.

Идея анализа типов состоит в том, что каждой вершине дерева программы мы пытаемся приписать некоторую синтаксическую метку по определённым правилам. Если таким образом каждой вершине можно приписать метку, то мы считаем, что программа проходит проверку типов, и она “хорошая”. Например, на рисунке 3 представлено выражение, а на рисунке 4 каждой вершине приписаны метки в согласие с некоторой системой типов.

Система типов определяет синтаксис типовых меток и правила, по которым их можно приписывать. Синтаксис обычно описывается в классических нотациях а ля BNF, а правила в виде типовых дробей. Например, так выглядят дроби для просто-типизированного λ-исчисления:

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma}\ ctx \qquad \frac{\Gamma \vdash M : \sigma \to \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M\ N : \tau}\ elim \to \qquad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}.\, M : \sigma \to \tau}\ intro \to$$

Типовые метки имеют чисто-синтаксическую природу, однако их можно проинтерпретировать. Самая популярная интерпретация — воспринимать типовую метку как множество. Так, метке $int \to int$ можно поставить в соответствие множество функций между множествами целых чисел $\mathbb{Z} \to \mathbb{Z}$.

1.3 Функции в Haskell

В своей основе Haskell представляет собой расширенное типизированное λ-исчисление, дополненное примитивными типами, возможностью декларировать новые имена, структурами данных и классами типов.

Примеры λ-абстракций в REPL окружении GHCi:

```
ghci> (\x -> x + 1) 4
5
```

Можно узнать тип функции в интерпретаторе (в реальности числа полиморфные, но об этом далее):

```
ghci> :t \x -> x + 1
\x -> x + 1 :: Int -> Int
```

Функциям можно давать имена. Именам можно приписывать типы, это рекомендуется делать явно для деклараций на верхнем уровне файлов исходного кода:

```
f :: Int -> Int
f x = x + 1
```

Если имя типа начинается с маленькой буквы, то это не конкретный тип, а типовая переменная, способная принимать различные значения в зависимости от места вызова. Такая возможность называется **параметрическим полиморфизмом**. Так, функция, которая просто возвращает свой аргумент, никак не ограничивает тип аргумента. Но в то же время тип результата должен совпадать с типом аргумента:

```
id :: a -> a
id x = x

ghci> :t id 'x'
id 'x' :: Char
```

Функции могут принимать другие функции в качестве аргументов (такие функции называются **функциями высших порядков (higher-order functions)**). Имя функции может состоять из специальных символов, тогда она считается оператором и может применяться к своим операндам в инфиксном стиле:

```
($) :: (a -> b) -> a -> b
f $ x = f x
```

Пример рекурсивной функции, использующей охранные выражения для отличения базового случая рекурсии:

```haskell
factorial :: Int -> Int
factorial n
  | n < 1 = 1
  | otherwise = n * factorial (n - 1)
```

Упражнение 1 *Что выведет запрос* `ghci> :t uncurry (flip const)?`

Упражнение 2 *Что выведет запрос* `ghci> :t first . first при`

```haskell
first :: (a -> a') -> (a, b) -> (a', b)
```

Упражнение 3 *Реализуйте факториал с помощью техники аккумулирующего параметра.*

1.4 Данные в Haskell

В Haskell есть встроенная возможность объявлять новые типы данных на основании других типов, а также создавать их экземпляры.

Зададим тип данных, описывающий животных:

```haskell
data Animal
  = Cat String Int
  | Dog String
```

Мы задали тип данных `Animal` и два способа создать значения этого типа: для кошек и собак. `Cat` и `Dog` — это **конструкторы данных**. Они представляют собой функции, реализация которых находится на стороне языка. Они выделяют память под экземпляры данного типа и позиционно размещают компоненты. Кошек мы описываем именем и оставшимся количеством жизней, а собак — только именем.

```haskell
Cat :: String -> Int -> Animal
Dog :: String -> Animal
```

Чтобы воспользоваться информацией, сохранённой в структуре данных, требуется деконструировать её с помощью паттерн-матчинга. Мы сопоставляем значение типа с образцом. Если образец похож на то, как было сконструировано значение, то он выбирается среди других образцов и переменные, задекларированные в нём, начинают ссылаться на соответствующее позиционно содержимое структуры данных:

```haskell
describeAnimal :: Animal -> String
describeAnimal animal = case animal of
  Cat name nLifes -> "This is cat " ++ name ++ " " ++ show nLifes
  Dog name -> "This is dog " ++ name
```

В Haskell есть специальный синтаксис для объявления полей с именованными метками.

```haskell
data Penguin = Penguin { getName :: String, getAge :: Int }
penguin = Penguin { getName = "Andrey", getAge = 500 }
```

Haskell генерирует функции-аксессоры для доступа к полям значения:

```haskell
ghci> :t getName
getName :: Penguin -> String
```

Часто функции в программировании частичные — при некоторых значениях аргументов они могут вернуть результат, а при некоторых — нет. Давайте моделировать это с помощью специального типа данных. Если есть вещественный результат, будем возвращать его. Если нет, будем возвращать специально выделенное константное значение этого типа.

```haskell
data MaybeD = NothingD | JustD Double
safeSqrtD :: Double -> MaybeD
safeSqrtD x = if x < 0 then NothingD else JustD (calcSqrt x)
```

Можно заметить, что так нам придётся объявлять отдельный тип вроде `MaybeD` для каждого типа `D`. Поэтому Haskell позволяет абстрагироваться в типе, аналогично тому как можно абстрагироваться по значениям в терме.

```haskell
data Maybe a = Nothing | Just a
safeSqrt :: Double -> Maybe Double
safeSqrt x = if x < 0 then Nothing else Just (calcSqrt x)
```

Заметьте, что сейчас `Maybe` — это не совсем тип, так как теперь нужно передать типовой параметр, чтобы получить конкретный тип. `Maybe` называют **типовым конструктором**.

Вместе с абстракцией на уровне типов появилась и аппликация типа к типу. А что если дать меньше параметров типовому конструктору, чем ожидается? А что если больше? Контроль за корректностью типовых аппликаций обеспечивает **система кайндов**[4]. Это простейшие "типы для типов", то есть синтаксические метки, контролирующие корректность записанных программистом типов. Так, обычные типы имеют метку (кайнд) `Type`[5]. Типовые конструкторы имеют стрелочные кайнды. Например, `Maybe :: Type -> Type`. Аппликация типового конструктора к типу подходящего кайнда убирает одну стрелку:

```haskell
ghci> :k Int
Int :: Type
ghci> :k Maybe
Maybe :: Type -> Type
ghci> :k Maybe Int
Maybe Int :: Type
```

[4] Иногда в русскоязычной литературе кайнды называют родами типов, но мы не будем так говорить.

[5] Кайнд `Type` экспортируется из модуля `Data.Kind`. В старых текстах и в выводе GHCi по умолчанию он обозначается звёздочкой: `Maybe :: * -> *`. Это одно и то же; чтобы GHCi печатал `Type`, включите `:set -XNoStarIsType`.

Кроме совершенно новых типов данных, в Haskell можно объявлять типовые синонимы. Это имена, которые можно использовать вместо других типов, если, например, запись оригинального типа слишком длинная для повсеместного написания:

```
type T a = VeryLongType Int (a -> AnotherLongType a)
```

Если тип данных содержит только один конструктор и только одно поле, то отсутствует необходимость в аллокации новой памяти, содержащей тег конструктора и набор ссылок на поля. В таком случае, в качестве значения такого типа можно всегда просто использовать значение оборачиваемого типа, оставляя новый тип присутствовать исключительно во время компиляции, снижая нагрузку во время исполнения. Для объявления таких типов-обёрток нужно воспользоваться ключевым словом `newtype` вместо `data`:

```
newtype CourseId = CourseId Int64
newtype ModuleId = ModuleId Int64
```

Упражнение 4 *Определите кайнд конструктора типа*

```
data Free f a = Pure a | Free (f (Free f a))
```

1.5 Классы типов в Haskell

Параметрический полиморфизм позволяет использовать один и тот же код для различных типов входных данных. Классы типов же позволяют одному идентификатору ссылаться на разные реализации для разных типов данных (что аналогично механизму перегрузки (overloading) в других языках). Классы типов, как говорят, являются механизмом **специального (ad-hoc) полиморфизма**. Так, мы можем задекларировать символ ==, выбор реализации которого зависит от выбора типа аргументов `a`:

```
class Eq a where
  (==) :: a -> a -> Bool
```

Для каждого типа можно объявить свою собственную реализацию `Eq`:

```
instance Eq CourseId where
  CourseId x == CourseId y = x == y

instance Eq a => Eq [a] where
  [] == [] = True
  x:xs == y:ys = x == y && xs == ys
  _ == _ = False
```

Теперь в зависимости от конкретного типа `a` в месте вызова, будет выбрана подходящая реализация для этого типа:

```
ghci> CourseId 1 == CourseId 2
False
ghci> [CourseId 1, CourseId 2] == [CourseId 1, CourseId 2]
True
```

Рассмотренные ранее параметрически-полиморфные функции ничего не могли делать со своими аргументами, кроме как возвращать их в качестве результата или передавать в другие полиморфные функции. Чтобы уметь делать что-то ещё, нужна какая-то дополнительная информация про тип, потому что иначе нет никакой гарантии, что над объектом данного типа можно делать все необходимые операции. Так, функция `suc n = n + 1` не будет работать для строчек, потому что для них, очевидно, не определена операция сложения. Поэтому некорректно будет приписать полиморфный тип `suc :: a -> a`.

Классы типов, в отличие от перегрузки, в том числе являются механизмом ограничения полиморфности функций. Мы можем явно задать, что функция требует не произвольный тип на вход, а произвольный тип, для которого определены обязательно нужные нам операции. Так, для типа `suc` достаточно ограничить тип условием наличия плюса для него (операция обозначаемая символом + объявлена в классе типов `Num`):

```
suc :: Num a => a -> a
```

Упражнение 5 *Реализуйте функцию, проверяющую равенство всех элементов данного списка.*

Упражнение 6 *Реализуйте инстанс полугруппы для функций.*

Упражнение 7 *Реализуйте инстанс `Eq` для функций. Подсказка: для произвольных функций это невозможно; подумайте, какие ограничения нужно наложить на тип аргумента, чтобы равенство стало проверяемым.*

1.6 Монады в Haskell

Класс типов `Functor` объявляется для конструкторов типов и позволяет заменить в некотором контейнере все элементы одного типа на все элементы другого, оставляя структуру контейнера неизменной.

```
class Functor (f :: Type -> Type) where
  fmap :: (a -> b) -> f a -> f b

instance Functor [] where
  fmap :: (a -> b) -> [a] -> [b]
  fmap _ [] = []
  fmap f (x:xs) = f x : fmap f xs
```

В Haskell любая функция просто вычисляет результат некоторого типа. Однако в программирования часто требуются функции, которые не только вычисляют результат, но и делают что-то ещё. Например, изменяют какое-то состояние или пишут в консоль. Иными словами, производят побочные эффекты. В любом случае в Haskell мы можем вернуть из функции только результат, поэтому такие побочные эффекты мы кодируем в качестве дополнительной структуры, оборачивающей чистый результат. Т.е. если функция без побочных эффектов возвращала какой-то тип `a`, то после добавления побочных эффектов в её реализацию, она будет возвращать некоторый тип *вычислений* `f a`.

- Если функция кидает ошибку, то `f = Maybe`.
- Если функция читает глобальное состояние типа `e`, то `f = e -> _`.
- Если функция читает глобальное состояние `s` и обновляет его, то `f = s -> (s, _)`.

Стандартная библиотека Haskell предоставляет несколько классов типов для работы со значениями вида `f a`. Они позволяют абстрагироваться от структуры `f` и работать со значениями `a` внутри, как будто нет никакой дополнительной структуры.

Первый такой класс типов позволяет писать выражения над вычислениями `f a`.

```haskell
class Functor f => Applicative (f :: Type -> Type) where
  pure :: a -> f a
  liftA2 :: (a -> b -> c) -> f a -> f b -> f c

instance Applicative Maybe where
  pure :: a -> Maybe a
  pure = Just

  liftA2 :: (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c
  liftA2 _ Nothing _ = Nothing
  liftA2 _ _ Nothing = Nothing
  liftA2 f (Just x) (Just y) = Just (f x y)
```

Второй класс типов позволяет делать последовательную композицию вычислений в императивном стиле[6][7]:

```haskell
class Applicative m => Monad (m :: Type -> Type) where
  (>>=) :: m a -> (a -> m b) -> m b

newtype State s a = State { runState :: s -> (s, a) }
```

[6] В коде ниже лямбда передаётся последним аргументом без скобок: `State \s -> ...` вместо `State (\s -> ...)`. Так позволяет писать расширение BlockArguments, мы будем пользоваться им повсеместно.

[7] В библиотеке `mtl` компоненты пары идут в обратном порядке: `s -> (a, s)`. Мы в конспекте ставим состояние первым.

```haskell
instance Monad (State s) where
  (>>=) :: State s a -> (a -> State s b) -> State s b
  m >>= k = State \s ->
    let (s', x) = runState m s in
    runState (k x) s'
```

Теперь если мы определим базовые операции работы с состоянием, мы сможем писать код в императивном стиле с побочными эффектами.

```haskell
get :: State s s
get = State \s -> (s, s)

put :: s -> State s ()
put newS = State \oldS -> (newS, ())

example :: State Int Int
example =
  get >>= \x ->
  put 42 >>= \() ->
  get >>= \y ->
  pure (x + y)

ghci> runState example 1
(42, 43)
```

Для таких монадических цепочек существует специальный синтаксический сахар:

```haskell
example :: State Int Int
example = do
  x <- get
  put 42
  y <- get
  pure (x + y)
```

Упражнение 8 *Реализуйте `liftA3` через `liftA2`.*

Упражнение 9 *Реализуйте `>>=` через `join` и наоборот.*

Упражнение 10 *Прочитайте два числа с консоли, поделите одно на другое нацело и распечатайте результат; если остаток не нулевой, распечатайте его тоже.*

2 Параметрический полиморфизм

Никакое нетривиальное семантическое свойство программ не может быть алгоритмически проверено[8]. Чтобы оставаться разрешимыми (в смысле проверки типов и/или вывода), многие системы типов жертвуют полнотой и, помимо некорректных программ, отвергают много корректных. В то же время системы типов также стараются предоставлять различные возможности, позволяющие протипизировать как можно больше корректных программ. Одна из них — параметрический полиморфизм.

Под **параметрическим полиморфизмом** мы будем подразумевать возможность кода единообразно работать с произвольными типами данных Strachey [2000], Cardelli and Wegner [1985], что позволяет во многих случаях избегать дублирования кода.

В этой главе мы рассмотрим, как описывают полиморфизм в самом простом виде — в типизированном λ-исчислении. Изучим различные формы параметрического полиморфизма и сопутствующие техники безопасного программирования. Проанализируем возможные способы эффективной реализации параметрического полиморфизма. И в завершение рассмотрим полиморфизм по рантайм-представлению, “полиморфизм по полиморфизму”.

2.1 Параметрический полиморфизм в языке

Рассмотрим, что нужно, чтобы получить полиморфизм в языке программирования и какие возможности это даёт.

2.1.1 Полиморфизм функций

Начнём со знакомого примера из предыдущей главы:

```
1 id :: a -> a
2 id x = x
```

Без полиморфизма нам пришлось бы написать отдельные `idInt`, `idChar` и так далее: корректные, но совершенно одинаковые программы. Типовая переменная `a` избавляет от дублирования, но оставляет вопрос: *кто и когда выбирает конкретный тип вместо* `a`*?*

Тип выбирает вызывающий код, причём отдельно в каждом месте вызова. Иначе говоря, у `id` есть ещё один, скрытый аргумент — сам тип. В Haskell этот аргумент можно сделать явным с обеих сторон: и принять его в определении функции, и передать в месте вызова[9]:

```
1 id :: forall a . a -> a
2 id @a (x :: a) = x

3 ghci> :t id @Int
4 id @Int :: Int -> Int
```

[8] `https://en.wikipedia.org/wiki/Rice%27s_theorem`

[9] Расширения TypeAbstractions и TypeApplications; явный квантор в сигнатуре — ExplicitForAll.

```
ghci> id @Char 'x'
'x'
```

Чтобы понять, откуда берётся скрытый аргумент, посмотрим, как полиморфизм описывают в самом простом виде — в типизированном λ-исчислении. Вспомним правила просто типизированного λ-исчисления из предыдущей главы. λ-абстракция позволяет обобщать выражения по значениям, каждая абстракция добавляет стрелку в тип выражения, аппликация же снимает стрелку:

$$\frac{x : \tau, \Gamma \vdash M : \sigma}{\Gamma \vdash \lambda x : \tau.\, M : \tau \to \sigma}\ Lam \qquad \frac{\Gamma \vdash M : \tau \to \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash M\ N : \sigma}\ App$$

Полиморфное λ-исчисление (*System F*) добавляет точно такую же пару конструкций, только для типов [Pierce, 2002, глава 23]. Λ-абстракция $\Lambda\alpha.\, M$ обобщает выражение по типу: это функция, аргументом которой является тип. Она добавляет в тип выражения квантор $\forall$ подобно тому, как λ добавляет стрелку[10]. Применение терма к типу $M\ \sigma$ (**универсальная аппликация (universal application)**) снимает квантор: вызывающий код выбирает, с каким конкретным типом σ он будет использовать терм, и σ подставляется в тип вместо α:

$$\frac{\alpha, \Gamma \vdash M : \tau}{\Gamma \vdash \Lambda\alpha.\, M : \forall\alpha.\, \tau}\ TLam \qquad \frac{\Gamma \vdash M : \forall\alpha.\, \tau}{\Gamma \vdash M\ \sigma : [\alpha \mapsto \sigma]\,\tau}\ TApp$$

Таким образом, например, функция *id* фактически принимает *два аргумента: тип и значение*:

$$id : \forall\alpha.\, \alpha \to \alpha$$
$$id = \Lambda\alpha.\, \lambda x : \alpha.\, x$$
$$id\ nat : nat \to nat$$
$$id\ nat\ 42 : nat$$

Сравните с записью на Haskell выше: определение `id @a (x :: a) = x` — это в точности $\Lambda\alpha.\, \lambda x : \alpha.\, x$, а вызов `id @Int` — это $id\ nat$.

В обычном коде на Haskell типовые абстракции и аппликации, как и кванторы в начале типа, расставляются неявно механизмом вывода типов. Достаточно конвенции именования: конкретные типы начинаются с большой буквы, а типовые переменные — с маленькой. Писать их вручную приходится, когда информации из терма недостаточно, чтобы вывести тип, когда нужно задать порядок типовых параметров, или чтобы сослаться на абстрагированный тип в теле функции[11].

Упражнение 11 *Какой тип у выражения* `const @Int`*? А у* `const @Int @Char`*? От чего зависит порядок, в котором передаются типы?*

[10] В теории зависимых типов и стрелка, и квантор $\forall$ являются частными случаями одной конструкции — П-типа.

[11] Ссылаться из тела функции на типовые переменные из её сигнатуры позволяет расширение ScopedTypeVariables.

Цена такого полиморфизма на уровне типов невелика: пока кванторы стоят только снаружи типа, алгоритм Хиндли–Милнера выводит все типовые абстракции и аппликации сам. Платить приходится, когда кванторы появляются в других местах (2.1.2), а также во время исполнения (2.2).

2.1.2 First-class polymorphism

Рассмотрим функцию `f g = (g 42, g 'a')`. Программа корректна, если передать в неё, например, `id`. Однако алгоритм Хиндли–Милнера её отвергает: аргумент функции он считает мономорфным, а `g` применяется и к числу, и к символу. Чтобы протипизировать такую программу, нужно разрешить квантору стоять не только снаружи типа, но и слева от стрелки.

Иначе говоря, нужна возможность писать функции, которые принимают другие полиморфные функции в качестве аргументов. Типы таких функций называются **типами высшего ранга (higher-rank types)**[12]. Так, типовой параметр функции `g` определяет функция `f`, а не вызывающий функцию `f`:

```
1 f :: (forall a . a -> a) -> (Int, Char)
2 f g = (g @Int 42, g @Char 'a') -- универсальная аппликация для наглядности
3 ghci> f (\x -> x)
```

Цена типов высшего ранга в том, то есть глобальный вывод типов Haskell в этом случае перестаёт работать. Но если типы высшего ранга приписать вручную, остальной вывод будет работать как раньше. Например, тип числа Чёрча полиморфен, поэтому функции над числами Чёрча имеют высший ранг[13][14]:

```
1 suc :: (forall a . (a -> a) -> a -> a) -> (b -> b) -> b -> b
2 suc n s z = s (n s z)
```

Упражнение 12 *Какую ошибку выдаст GHC, если убрать у функции `f` сигнатуру и типовые аппликации? Какой тип для `g` он пытался вывести?*

Упражнение 13 *Какой ранг имеет тип `Int -> (forall a . a -> a)`?*

От многих проблем сопутствующих типам высших рангов можно избавиться, если создавать для них обёртки. Например, для чисел Чёрча можно создать обёртку `newtype Church`. Теперь код, работающий с обёрткой, может быть протипизирован типами первого ранга, только конструктор имеет тип высшего ранга:

```
1 newtype Church = Church (forall a . (a -> a) -> a -> a)
2 (+) :: Church -> Church -> Church -- rank 1
```

[12] В Haskell их включает расширение RankNTypes.

[13] `https://okmij.org/ftp/tagless-final/course/Boehm-Berarducci.html`

[14] Любому интересующемуся языками программирования предлагается провести на сайте Олега Киселёва не один месяц жизни: `https://okmij.org/ftp/README.html`.

Аналогичный код можно написать и на Java или Kotlin:

```
interface Church { fun <a> fold(s: (a) -> a, z: a): a }
fun plus(n: Church, m: Church): Church = object : Church {
    override fun <a> fold(s: (a) -> a, z: a): a = n.fold(s, m.fold(s, z))
}
```

По умолчанию типовые параметры можно специализировать только на конкретные типы. Специализацию типовых параметров на полиморфные типы (включающие `forall`'ы внутри себя) называют **импредикативным применением**[15].

```
runST :: (forall s. ST s c) -> c
($) :: forall a b . (a -> b) -> a -> b
foo = runST $ ... -- a специализируется на (forall s. ST s c)
```

Этот пример настолько частый, что GHC делает для оператора `$` исключение и типизирует его без расширения. Для любой другой функции с таким же типом, например, собственного оператора аппликации `app`, потребуется само расширение; с ним GHC применяет алгоритм Quick Look, о котором ниже.

Higher-rank типы можно использовать как type-based escape analysis, иначе говоря, не позволять пользователю передавать некоторое значение вовне определённого скоупа. Так, например, Haskell предоставляет эффективную монаду `ST`, позволяющую в рамках ограниченного скоупа работать с мутабельными ячейками памяти Launchbury and Peyton Jones [1995][Maguire, a, 7.2, ST trick]:

```
newtype ST s a = ST (IO a) -- упрощённо, в GHC устроено сложнее
runST :: (forall s. ST s a) -> a

sumTo :: Int -> Int
sumTo n = runST do
  ref <- newSTRef 0
  forM [0..n] \i -> modifySTRef ref (+ i)
  readSTRef ref
```

Заметим, что если попытаться вернуть из `runST` ссылку на мутабельную ячейку, то тип результата должен будет содержать фантомный параметр `s`. Но в сигнатуре `runST` тип результата `a` стоит вне области действия квантора по `s`: этот квантор связан внутри аргумента, и снаружи `s` просто не существует. Поэтому такая программа не проходит проверку типов:

```
newSTRef :: v -> ST s (STRef s v)
ghci> runST (newSTRef True) -- ошибка типизации
```

[15] В Haskell его включает расширение ImpredicativeTypes.

На практике, чтобы отличать такие локально связанные типовые переменные, используют концепцию уровней[16] Peyton Jones [2019].

Типы высших рангов вместе с импредикативным применением образуют **полиморфизм первого класса (first-class polymorphism)**, когда полиморфные типы могут использоваться почти так же свободно, как и любые другие. Классический алгоритм глобального вывода Хиндли-Милнера не справляется (и в общем случае задача неразрешима), так что существует большое количество решений, делающих различные компромиссы. Можно сделать вывод типов локальным, опирающемся только на соседние ноды AST и вспомогательные типовые аннотации Pierce and Turner [2000], Christiansen [2013], Dunfield and Krishnaswami [2019]. Либо же можно попытаться помочь глобальному выводу дополнительной предобработкой (Quick Look[17] Serrano et al. [2020], реализованный в GHC начиная с версии 9.2 и включаемый расширением ImpredicativeTypes) или дополнительными регулирующими конструкциями (FreezeML Emrich et al. [2020]).

2.1.3 Полиморфизм данных

Полиморфными бывают не только функции, но и типы данных. В предыдущей главе мы видели, что `Maybe` — это не тип, а типовой конструктор: чтобы получить тип, ему нужно передать аргумент, например, `Maybe Int`. Если ранее мы управляли типом с уровня термов универсальной аппликацией, то теперь мы управляем типом на уровне типов. Иначе говоря, на уровне типов появляется своё маленькое λ-исчисление: λ-абстракция в типах, аппликация в типах и, соответственно, β-редукция.

Для примера определим тип пары по Чёрчу, абстрагированный от конкретных типов компонент, чтобы пользователь мог выбрать нужные ему. Сначала на Haskell, где вычислительную семантику типов можно проследить в синонимах типов:

```
1 type Pair a b = forall c . (a -> b -> c) -> c

2 pair :: a -> b -> Pair a b
3 pair x y = \f -> f x y

4 fst :: Pair a b -> a
5 fst p = p (\x _ -> x)
```

Пара здесь закодирована по Чёрчу: это функция, которая отдаёт обе компоненты любому, кто скажет, что с ними делать. Синоним `Pair` — функция на типах: `Pair Int Int` вычисляется в `forall c . (Int -> Int -> c) -> c`. Обычные конструкторы типов, напротив, номинативны: `(Int, Int)` или `Maybe Int` никуда далее не вычисляются.

А раз на уровне типов есть функции и аппликации, то и типы можно составить неправильно: `Maybe Maybe` или `Int Bool`. Значит, типам нужна своя система типов. Это уже знакомая

[16] `https://okmij.org/ftp/ML/generalization.html`
[17] (youtube) A Quick Look at Impredicativity (Simon Peyton Jones)

нам система кайндов: она (пока) представляет собой простейшую "систему типов для типов" и обеспечивает well-formedness типов и строгую нормализуемость[18]. Получается двухэтажная конструкция (рис. 5): вершинам дерева терма приписаны типы, но типы — тоже деревья, и уже их вершинам приписаны кайнды. Кайнды, в свою очередь, тоже деревья (например, `Type -> Type`); что приписывается их вершинам, мы увидим в конце этого раздела (рис. 7).

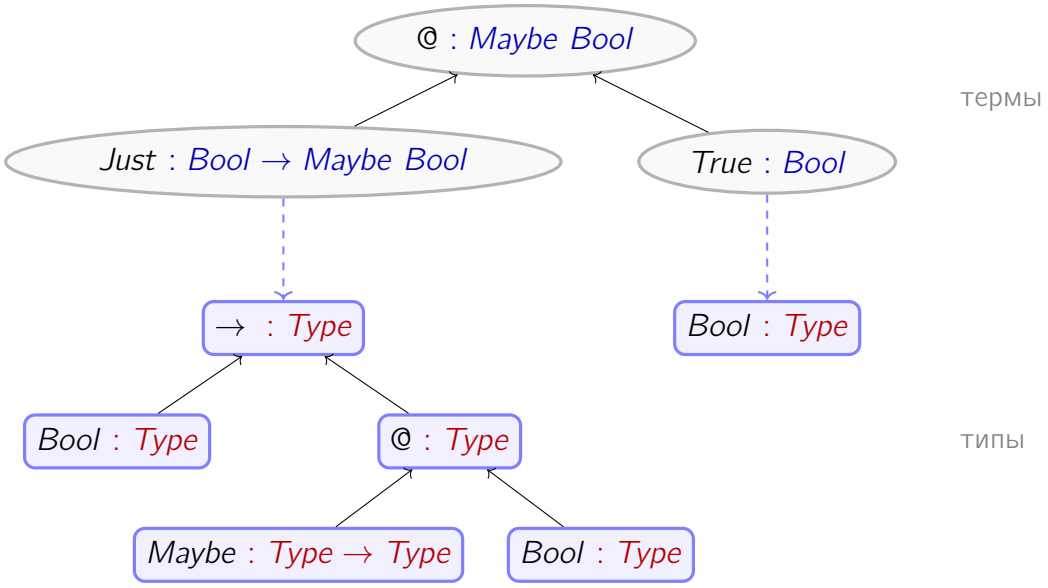


Рис. 5: Два этажа типизации выражения `Just True`. Вершинам дерева терма приписаны типы (синим). Каждый тип сам является деревом (пунктирные стрелки), и его вершинам приписаны кайнды (красным).

Правила приписывания кайндов устроены в точности как правила *Lam* и *App* для термов, только этажом выше: абстракция по типовой переменной добавляет стрелку в кайнд, аппликация типа к типу её снимает [Pierce, 2002, глава 29][19]:

$$\frac{\alpha : \kappa_1, \Gamma \vdash \tau : \kappa_2}{\Gamma \vdash \lambda \alpha : \kappa_1 . \tau : \kappa_1 \to \kappa_2}\ KLam \qquad \frac{\Gamma \vdash \tau : \kappa_1 \to \kappa_2 \quad \Gamma \vdash \sigma : \kappa_1}{\Gamma \vdash \tau\ \sigma : \kappa_2}\ KApp$$

Аппликация абстракции к аргументу редуцируется подстановкой, как и на уровне термов:

$$(\lambda \alpha : \kappa . \tau)\ \sigma \rightsquigarrow [\alpha \mapsto \sigma]\, \tau$$

Кроме того, нужны правила для конструкций, которые строят обычные типы: стрелка и квантор из типов кайнда $*$ делают тип кайнда $*$. Квантор при этом связывает типовую переменную любого кайнда:

$$\frac{\Gamma \vdash \tau : * \quad \Gamma \vdash \sigma : *}{\Gamma \vdash \tau \to \sigma : *}\ KArrow \qquad \frac{\alpha : \kappa, \Gamma \vdash \tau : *}{\Gamma \vdash \forall \alpha : \kappa . \tau : *}\ KAll$$

[18] **Строгая нормализуемость** — любой порядок редукций приводит к нормальной форме.

[19] В формулах кайнд Type обозначаем традиционной звёздочкой, как это принято в литературе Pierce [2002].

В Haskell кайнды, как и типы, выводятся автоматически, а посмотреть их можно командой `:k`, как мы делали в предыдущей главе.

В исчислении то же самое записывается с явными абстракциями и аппликациями на обоих уровнях (кайнды связанных типовых переменных указаны верхним индексом):

$$
\begin{aligned}
&Pair : * \to * \to * \\
&Pair = \lambda \tau^{*}\ \sigma^{*}.\, \forall \gamma.\, (\tau \to \sigma \to \gamma) \to \gamma \\
&pair : \forall \alpha\ \beta.\, \alpha \to \beta \to Pair\ \alpha\ \beta \\
&pair = \Lambda \alpha^{*}\ \beta^{*}.\, \lambda x^{\alpha}\ y^{\beta}.\, (\Lambda \gamma^{*}.\, \lambda f^{\alpha \to \beta \to \gamma}.\, f\ x\ y) \\
&fst : \forall \alpha\ \beta.\, Pair\ \alpha\ \beta \to \alpha \\
&fst = \Lambda \alpha^{*}\ \beta^{*}.\, \lambda p^{Pair\ \alpha\ \beta}.\, p\ \alpha\ (\mathbf{K}\ \alpha\ \beta)
\end{aligned}
$$

Прочитаем определение fst. Принимаем два типа и пару p. Пара — полиморфная функция, поэтому сначала выбираем для неё тип результата α, а затем передаём ей функцию **K**, возвращающую первый из двух аргументов.

Упражнение 14 *Постройте по правилам кайндизации дерево вывода $Pair : * \to * \to *$. Какие правила и в каком порядке в нём применяются? Поредуцируйте тип Pair nat bool до нормальной формы.*

Упражнение 15 *Реализуйте на Haskell функции `snd` и `swap :: Pair a b -> Pair b a`. Запишите `snd` в исчислении с явными типовыми абстракциями и аппликациями.*

Haskell не позволяет создавать функции на типах по месту с помощью явной типовой лямбды[20] ввиду проблематичности этой конструкции для вывода типов. Однако именованные функции на типах есть, и мы рассмотрим их далее 3.2. В Scala существует нетривиальный трюк[21][22], который позволяет этого добиться. Scala3, однако, включила эту возможность непосредственно в язык[23].

2.1.4 Higher-order/kinded polymorphism

Функция `fmap` работает одинаково для списков, `Maybe` и деревьев. Чтобы записать её тип, нужно абстрагироваться уже не по типу элементов, а по самому контейнеру, то есть по типовому конструктору кайнда `Type -> Type`.

Haskell позволяет также абстрагироваться по типам произвольных кайндов, а не только `Type`, как в `data` декларациях (**higher-order/kinded types (HKT)**[24]), так и в полиморфных функциях. Далее мы встретим немало примеров. Так, тип `Fix`, с которым мы подробно познакомимся в 4.3.2, параметризован не типом, а типовым конструктором, и потому имеет кайнд `(Type -> Type) -> Type`. Катаморфизм же абстрагирован по типу стрелочного кайнда:

[20] `https://stackoverflow.com/questions/4069840/lambda-for-type-expressions-in-haskell`
[21] (stackoverflow) Scala type lambdas.
[22] `https://stackoverflow.com/questions/9443004/what-does-the-operator-mean-in-scala`
[23] `https://docs.scala-lang.org/scala3/reference/new-types/type-lambdas.html`
[24] `https://serokell.io/blog/kinds-and-hkts-in-haskell`

```haskell
newtype Fix f = Fix (f (Fix f))
cata :: forall (f :: Type -> Type) a . Functor f => (f a -> a) -> Fix f -> a
```

Упражнение 16 *Какой кайнд у* `Fix Maybe`*? Приведите несколько значений этого типа. Какой знакомый тип они напоминают?*

Цена: чтобы вывод типов оставался простым, Haskell требует, чтобы на месте `f` стоял именно типовой конструктор, а не произвольная функция на типах (мы вернёмся к этому в 3.2). Во многих же промышленных языках полиморфизма высших порядков нет вовсе, и далее мы рассмотрим технику, позволяющую закодировать его в таком языке (см. 3.1.8).

2.1.5 Обобщённые алгебраические типы данных (GADTs)

Обобщённые алгебраические типы данных (generalized algebraic data types, GADTs) позволяют приписывать данным на уровне типов больше информации. В качестве модельного примера возьмём синтаксис крошечного языка программирования. Зададимся целью не допустить возможности конструирования в Haskell некорректных с точки зрения типов синтаксических деревьев.

```haskell
data Expr = Const Int | IsZero Expr | If Expr Expr Expr
```

Сейчас ничто не мешает составить бессмысленную программу вроде `IsZero (IsZero (Const 1))`: некорректная программа на нашем языке является совершенно корректным значением Haskell.

Как мы знаем, конструкторы данных в Haskell — это обычные функции с той лишь разницей, что их реализация генерируется компилятором (аллокация памяти, размещение полей...). У функций есть тип. Например, `IsZero :: Expr -> Expr`.

В Haskell есть синтаксис определения `data` через задание типов конструкторов[25]. Он совершенно аналогичен рассмотренному ранее, только гораздо более удобен для сложно организованных структур данных. Рассмотренный ранее тип термов `Expr` будет выглядеть следующим образом:

```haskell
data Expr where
  Const :: Int -> Expr
  IsZero :: Expr -> Expr
  If :: Expr -> Expr -> Expr -> Expr
```

Для полиморфных структур данных, на примере списка, используется следующий синтаксис. Имя `elem` нужно исключительно для документации и больше никак его использовать нельзя, оно только маркирует наличие типового параметра и позволяет ему вручную задать кайнд[26].

[25] `https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/gadt_syntax.html#gadt-style`

[26] Кайнд можно не писать. Либо можно не писать имена параметров и просто приписать кайнд типовому конструктору: `data List :: Type -> Type where ...`.

```haskell
data List (elem :: Type) where
  Nil :: List a
  Cons :: a -> List a -> List a
```

Добавим к `Expr` фантомный типовой параметр `ty`, обозначающий тип Haskell, в который должно быть проинтерпретировано данное выражение, и с помощью GADT зададим конкретные значения `ty` результирующим типам конструкторов. Так, мы говорим, что программа сконструированная с помощью `Const` вычисляется в число, `IsZero` вычисляется в булево значение, а условное выражение — в тип веток:

```haskell
data Expr ty where
  Const :: Int -> Expr Int
  IsZero :: Expr Int -> Expr Bool
  If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty

eval :: Expr ty -> ty
```

Теперь мы можем написать безопасный типизированный интерпретатор. Обратите внимание, что при сопоставлении с образцами конструкторов, у нас уточняется информация о типовом параметре:[27]

```haskell
eval :: Expr ty -> ty
eval = \case
  Const x  -> x           -- ty ~ Int
  IsZero t -> eval t == 0 -- ty ~ Bool
  If c t e -> if eval c then eval t else eval e
```

Цена: вывод типов при сопоставлении с образцом по GADT без подсказки не работает, поэтому функциям вроде `eval` сигнатура обязательна. Далее мы рассмотрим, как GADT в Haskell выражаются через более базовые механизмы языка и почему вывод типов для них сложнее (3.4.4).

Упражнение 17 *Добавьте в язык пары: конструкторы* `MkPair :: Expr a -> Expr b -> Expr (a, b)` *и* `Fst :: Expr (a, b) -> Expr a`. *Допишите* `eval`. *Какие равенства типов возникают в новых ветках?*

2.1.6 Структуры на уровне типов, data promotion

Чтобы обрести больший контроль корректности программ, научимся кодировать произвольные структуры данных на уровне типов. В качестве модельной задачи зададим структуру данных, моделирующую вектор, но с контролем длины.

Для начала определим натуральные числа на уровне типов в стиле Пеано:

[27] Тут используется удобное расширение LambdaCase, позволяющее не вводить лишние имена.

```haskell
data Zero
data Suc n
```

Упражнение 18 *Сколько обитателей типа* `Suc (Suc Zero)`*?*

Теперь мы можем задать тип вектора, содержащий информацию о длине:

```haskell
data Vec (size :: Type) (elem :: Type) where
  VNil :: Vec Zero a
  VCons :: a -> Vec n a -> Vec (Suc n) a

example :: Vec (Suc (Suc Zero)) Int
example = VCons 1 (VCons 2 VNil)
```

Для такого типа, например, можно написать безопасную функцию `zip`, работающую только на векторах одинаковой длины:

```haskell
vzip :: Vec n a -> Vec n b -> Vec n (a, b)
vzip VNil VNil = VNil                                    -- n ~ Zero
vzip (VCons x xs) (VCons y ys) = VCons (x, y) (vzip xs ys) -- n ~ Suc n'
```

Заметьте, что в остальных ветках `vzip` должны возникнуть эквивалентности, начинающиеся с различных конструкторов, например, `Zero` $\sim$ `Suc n`. Поскольку невозможно построить такие аргументы функции, Haskell позволяет соответствующие ветки не рассматривать.

Упражнение 19 *Напишите функцию добавления в конец элемента вектора. Двигайтесь последовательно, заполняя типовые дыры и отслеживая возникающие эквивалентности.*

Сейчас язык кайндов, который должен контролировать типы, слишком беден. Действительно, кайнд `Suc` — `Suc :: Type -> Type`, соответственно ничто не мешает написать, например, `Suc (Maybe Int)`. В то же время он слишком ограничивающий, поскольку не поддерживает полиморфизм, что дало начало большому количеству дублирований а ля `Typeable (ty :: Type)`, `Typeable1 (ty :: Type -> Type)`...

Современный Haskell позволяет объявлять новые типы и кайнды подобно тому, как `data` позволяет объявлять новые типы[28].

```haskell
type data Nat = Zero | Suc Nat
```

Теперь вектору можно приписать более точный кайнд:

```haskell
data Vec (size :: Nat) (elem :: Type) where
  VNil :: Vec Zero a
  VCons :: a -> Vec n a -> Vec (Suc n) a
```

[28] Расширение TypeData.

Упражнение 20 *Что выведет* `ghci> :k Vec`*?*

Другой вариант добиться того же самого — автоматически продвигать (promotion) все `data` декларации на уровень выше[29] Yorgey et al. [2012]. А именно: любой конструктор типа также становится кайндом, а конструктор данных — конструктором типа. Так, в примере с числами, мы можем задекларировать натуральные числа как обычно и использовать на уровне типов:

```
data Nat = Zero | Suc Nat
ghci> :k Suc -- тут понятно, что Suc используется как тип
Suc :: Nat -> Nat
```

Поскольку типы и термы в Haskell живут в разных пространствах имён, можно называть конструкторы типов и данных одинаково. Однако если продвинуть такой тип данных, возникнет неоднозначность: мы имеем в виду тип или продвинутый конструктор. Haskell позволяет указать явно, что речь идёт о продвинутом конструкторе с помощью одинарной кавычки.

```
data T = T Nat
ghci> :k T
T :: Type       -- про конструктор типа
ghci> :k 'T
'T :: Nat -> T -- про продвинутый конструктор данных
```

Не любые `data` декларации подходят для продвижения, в то же время `type data` декларации позволяют явно запросить структуру уровня типов и получить внятные ошибки, если декларация написана неправильно.

В случае продвижения полиморфного типа, мы получаем полиморфные кайнды[30]:

```
data [a] = [] | (:) a [a]
ghci> :k '(:)
'(:) :: forall k . k -> [k] -> [k]
```

Примеры продвижения различных конструкций можно увидеть в таблице 6.

В качестве примера, зададим гетерогенный список, индексированный типами элементов:

```
data HList (tys :: [Type]) where
  HNil :: HList '[]
  HCons :: ty -> HList tys -> HList (ty ': tys)

example :: HList '[Int, Bool, Double]
example = HCons 42 $ HCons True $ HCons 12.5 HNil
```

[29] Расширение DataKinds.

[30] Расширение PolyKinds.

Term	Type	Kind
`Zero`	`Nat`	`Type`
`[Zero, Suc Zero]`	`[Nat]`	`Type`
`[]`	`forall a. [a]`	`Type`
`(:)`	`forall a. a -> [a] -> [a]`	`Type`
	`'Suc 'Zero`	`Nat`
	`'['Zero, 'Suc 'Zero]`	`[Nat]`
	`'[Int, Double]`	`[Type]`
	`'[]`	`forall k. [k]`
	`'(:)`	`forall k. k -> [k] -> [k]`

Рис. 6: Пример продвижений в Haskell.

Структуры данных тоже могут быть полиморфными по кайндам. Рассмотрим тип `Tagged`, позволяющий дополнить тип значения дополнительным типовым тегом. Кайнд тега может быть произвольным, поэтому, например, можем использовать встроенные в систему типов строковые и числовые константы[31] (другой пример использования полиморфных кайндов — тип `Proxy`, см. 2.1.7):

```
1 newtype Tagged (tag :: k) (a :: Type) = Tagged a
2 ghci> :t Tagged
3 Tagged :: forall k (tag :: k) a. a -> Tagged tag a

4 example :: Tagged ("dbId" :: Symbol) Int
5 example = Tagged 42
```

Современный Haskell в итоге пришёл к тому, что система типов не делает различий между типами и кайндами (рис. 7). В частности, `Type :: Type`. Это нужно для расширения возможностей Haskell в сторону программирования с зависимыми типами путём добавления несинтаксических эквивалентностей для кайндов[32]. Соответствующее расширение *System FC*, внутреннего языка GHC (см. далее 3.4.4), равенствами кайндов описано в работе Weirich et al. [2013][33][34].

2.1.7 Факультативно: эмуляция типовых абстракций и аппликаций (`Proxy`)

Это факультативный и специфичный для Haskell материал. Его можно пропустить при первом чтении и вернуться, когда тип `Proxy` встретится в коде следующих глав.

[31] TypeLits: литералы уровня типов кайндов `Symbol` и `Nat`.
[32] Расширение TypeInType, ныне часть PolyKinds.
[33] (youtube) Мини-курс на русском языке про развитие Haskell в сторону зависимой типизации.
[34] (youtube) Мини-курс на русском языке — система вывода типов Haskell.

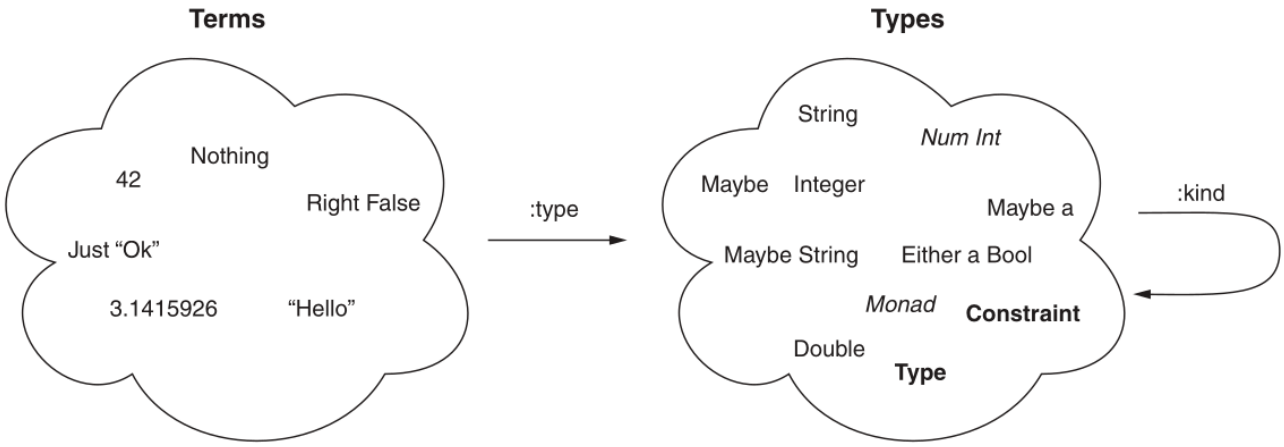


Рис. 7: Типы и кайнды — одно (Bragilevsky).

Проблема здесь историческая: расширения, позволяющие вручную задавать типовые аппликации и абстракции, появились в Haskell сравнительно недавно[35]. До этого явно передать тип в функцию было нельзя, и пользовались следующей техникой, которая до сих пор встречается во многих библиотеках.

В стандартной библиотеке определён тип `Proxy` с одним параметром. Это **фантомный типовой параметр** — значения соответствующего типа не хранятся в структуре данных, он только позволяет размещать дополнительную информацию на уровне типов[36]. Соответственно, неинформативную константу `Proxy` можно проаннотировать нужным типом и передать в функцию, чтобы специализировать типовой параметр на нужный тип. Или можно принять `Proxy` и воспользоваться ScopedTypeVariables для типовых сигнатур в паттернах[37].

```
1 data Proxy a = Proxy

2 id :: Proxy a -> a -> a
3 ghci> :t id (Proxy :: Proxy Int)
4 id (Proxy :: Proxy Int) :: Int -> Int

5 id (Proxy :: Proxy a) x = (x :: a)
```

Иногда прокси-тип оставляют полиморфным, чтобы не зависеть от `Data.Proxy`. Вместо конкретного значения иногда передают специализированное значение $\bot$, а получатель, не зная тип, не сможет его форсировать (однако, любые вхождения $\bot$ в терм слишком настораживают, поэтому это скорее не очень хорошая практика).

[35] TypeApplications, TypeAbstractions.

[36] `https://wiki.haskell.org/Phantom_type`

[37] Типовой параметр на самом деле имеет полиморфный кайнд: `data Proxy (a :: k) = Proxy`, чтобы эта техника работала с типами произвольных кайндов (см. 2.1.6).

```haskell
id :: proxy a -> a -> a
id (_ :: proxy a) x = (x :: a)

ghci> :t id (undefined :: Proxy Int)
id (undefined :: Proxy Int) :: Int -> Int
```

2.2 Реализация параметрического полиморфизма

До сих пор полиморфизм был для нас свойством системы типов: мы обсуждали, какие программы он позволяет протипизировать. Теперь посмотрим, сколько он стоит во время исполнения.

Конвенция вызова[38] представляет собой набор соглашений между тем как функция компилируется и как должна вызываться. Например, функция принимает два аргумента, каждый размером в машинное слово, и возвращает один результат размером в машинное слово. Тогда сгенерированный низкоуровневый код этой функции может, например, ожидать, что оба аргумента передаются через специальную пару регистров, а складывать результат он будет в третий. В таком случае вызывающий код обязан предоставить аргументы в правильных регистрах и ожидать результата в некотором третьем, заранее оговоренном регистре.

В общем случае, конвенция вызова функции зависит от типов аргументов и результата. Нужно знать как минимум их размер, чтобы понять, размещать их в регистрах или на стеке. Нужно знать, это указатель (**reference type**) или значение само по себе (**value type**), чтобы понимать, как с ним работать. В структурах данных нужно знать смещения полей.

Таким образом, реализация параметрического полиморфизма в языке — это не тривиальная задача. Разные языки используют различные подходы, все со своими достоинствами и недостатками.

2.2.1 Мономорфизация

Мономорфизация — самый прямолинейный подход: полиморфные функции и структуры компилируются для каждого набора типовых аргументов. Так, если различных наборов типовых аргументов, с которыми эта функция вызывается, например, 100 (что запросто может быть), то её код будет компилироваться сто раз и занимать в бинарнике в сто раз больше места. Так делают, например, C++ и Rust.

На самом деле всё ещё хуже. Если проект многомодульный и состоит из множества единиц компиляции (кусков, которые компилируются отдельно), то одна и та же специализация функции на типовые аргументы будет компилироваться заново во всех единицах компиляции, где такая специализация нужна. А затем, линкер будет заниматься удалением дубликатов, что тоже не самый быстрый и эффективный процесс.

\+ Порождаемый код максимально эффективен для каждого типа;

[38] https://en.wikipedia.org/wiki/Calling_convention

+ Легко на этапе компиляции отрабатывают `is`-проверки значений на принадлежность определённому типу (в остальных подходах с этим всё сложно);
– Время компиляции крайне велико;
– Существенно увеличивается размер результирующего бинарного файла, что может быть критично для некоторых приложений;
– Может неэффективно работать из-за засорения кеша кода в процессоре;
– В интерфейсах не может быть полиморфных методов, так как мы не знаем в месте вызова, к какому именно наследнику относится вызываемый метод, и какой код нужно специализировать (аналогично, не работает higher-rank полиморфизм);
– К полиморфным функциям нельзя динамически линковаться (у них нет кода до специализации);
– В общем случае нельзя поддержать variance, потому что код компилируется для конкретного типа и в общем случае не может работать для произвольного подтипа или супертипа (если reference и value типы могут находиться в одной иерархии подтипизации).

Некоторые языки (например, C++ и Zig) не делают инстанциацию скрытой деталью реализации языка, а предоставляют её как инструмент пользователям. Это даёт следующие возможности:

- Если разрешить использовать значения в типах, инстанциация может использоваться как механизм вычислений на этапе компиляции.
- Если отложить проверку ошибок на стадию инстанциирования, то мы получим своего рода статическую утиную типизацию. Это позволит не описывать сложные сигнатуры полиморфных функций. Однако тогда функции для тестирования придётся вручную инстанцировать для всевозможных типов, иначе нельзя будет статически понять, компилируется ли она хотя бы для этих типов.

2.2.2 Стирание типа

Можно всё сделать наоборот, унифицировав значения, которые приходят на вход полиморфным функциям и хранятся в полиморфных структурах данных, вместо того, чтобы компилировать код под каждый тип.

Пусть каждое значение будет аллоцировано в куче и передаваться по указателю. Тогда мы сможем переиспользовать один и тот же код для разных типовых аргументов — он просто будет ожидать указатели.

+ Каждая функция компилируется ровно один раз — быстро;
+ Можно динамически загружать новые полиморфные функции и типы и использовать их друг с другом;
+ Гибкость — вариантность, полиморфные методы в интерфейсах, higher-rank types и т.д. просто работают;
– Аллокация в куче и разыменование указателя может очень сильно замедлить код;

– Поскольку информация о типах стирается, нельзя ничего сделать с типовым аргументом, не имея его обитателей (например, запросить рефлексией информацию или сделать `is` проверку).

Такого подхода придерживаются JVM, Haskell и, как правило, другие функциональные языки ввиду его гибкости и скорости компиляции.

Особую проблему вызывает работа с примитивами и другими value-типами, потому что каждое значение приходится сначала боксить (переносить в кучу), а потом уже использовать в полиморфном контексте. Поэтому языки борются с этим как могут. Некоторые языки урезают диапазоны значений примитивов, чтобы зарезервировать бит, определяющий, это указатель или значение. Код консультируется с этим битом для работы (похоже на 2.2.4). Так делают, например, OCaml и Koka. Агрессивный инлайнинг вместе с другими оптимизациями тоже помогают Blanchet [1999]. Java, например, также пытается аккуратно двигаться в сторону возможности мономорфизации[39][40].

2.2.3 Гибридный подход

C# реализует гибридный подход[41]. Они различают значения, хранимые в куче — reference types, и значения, хранимые на стеке — value types. Для первых они генерируют одну специализацию, работающую с указателями. Для каждого набора value-типов они генерируют лениво, в рантайме, специализации.

То есть следы дженериков в таком подходе есть и в промежуточном представлении CIL, и в рантайме.

+ value-типы хранятся и передаются as-is без боксинга;
+ Доступна рефлексия по дженерикам;
+ Небольшое время компиляции;
– Инстанциация в рантайме замедляет исполнение;
– Variance работает только для reference types (что странно — есть “правильная” подтипизация, а есть “неправильная”).

2.2.4 Использование виртуальной таблицы свойств типов

Swift[42] вместе с каждым типовым параметром передаёт value witness table (рис. 8). Это таблица со всей необходимой информацией о типе: размер и выравнивание, что нужно сделать при копировании и перемещении объекта (например, инкрементировать счётчик ссылок). Таким образом, скомпилированный код постоянно обращается к этой таблице и делает виртуальные вызовы функций из неё (рис. 9).

[39] Type Specialization of Java Generics - What If Casts Have Teeth?
[40] `https://cr.openjdk.org/~jrose/values/parametric-vm.html`
[41] Generics in the runtime (C# programming guide).
[42] (youtube) 2017 LLVM Developers' Meeting: “Implementing Swift Generics”

```
struct value_witness_table {
  size_t size, align;
  void (*copy_init)(opaque *dst, const opaque *src, type *T);
  void (*copy_assign)(opaque *dst, const opaque *src, type *T);
  void (*move_init)(opaque *dst, opaque *src, type *T);
  void (*move_assign)(opaque *dst, opaque *src, type *T);
  void (*destroy)(opaque *val, type *T);
};
```

Рис. 8: Swift value witness table.

Example:

```
func f<T>(_ t: T) -> T {
  let copy = t
  return copy
}
```

Implementation:

```
void f(opaque *result, opaque *t, type *T) {
  opaque *copy = alloca(T->vwt->size);
  T->vwt->copy_init(copy, t, T);
  T->vwt->move_init(result, copy, T);
  T->vwt->destroy(t, T);
}
```

Рис. 9: Код полиморфной функции, порождаемый компилятором Swift.

+ Небольшое время компиляции;
+ Предсказуемая эффективность (не приводит к неожиданным паузам в рантайме);
+ Эффективная работа с value-значениями;
+ Высокая гибкость;
+ Информация о типах не стирается;
– Серьёзный константный оверхед на динамические вызовы через таблицу, эффективность очень сильно зависит от компиляторных оптимизаций.

Это своего рода реализация параметрического полиморфизма через специальный. Вместе с типом в функцию неявно передаётся таблица операций, которые умеет этот тип: узнать размер, скопировать, переместить. В следующей главе мы увидим, что ровно так же, передачей словарей функций, реализуются классы типов (3.1.1), только набор операций там задаёт программист, а не компилятор.

2.2.5 Сравнение подходов

Сведём четыре подхода в таблицу 10. Ни один из них не выигрывает по всем строкам: каждый язык выбирает, чем платить.

Упражнение 21 *В Kotlin (JVM, стирание типов) проверка* `x is T` *для типового параметра* `T` *разрешена только внутри* `inline`*-функций, где параметр помечен как* `reified`*. Объясните это*

	Мономорфизация (2.2.1)	Стирание типа (2.2.2)	Гибридный подход (2.2.3)	Таблица свойств типа (2.2.4)
Языки	C++, Rust	JVM, Haskell	C#	Swift
Скорость кода	+ лучшая	− боксинг, индирекции	+ для value-типов	− вызовы через таблицу
Время компиляции, размер бинарного файла	−	+	+	+
Динамическая загрузка полиморфного кода	−	+	+	+
Полиморфные методы в интерфейсах, higher-rank	−	+	+	+
Тип известен во время исполнения	− только статически	− стёрт	+	+

Рис. 10: Сравнение реализаций параметрического полиморфизма.

ограничение. Какой из рассмотренных подходов к реализации полиморфизма здесь локально применяется?

Упражнение 22 *В Rust (мономорфизация) трейт с обобщённым методом нельзя использовать как trait object (*`dyn Trait`*). Каким из перечисленных недостатков мономорфизации это объясняется?*

Упражнение 23 *Приложение подгружает плагин, скомпилированный отдельно, и хочет вызвать из него обобщённую функцию с типом, о котором автор плагина не знал. Для каждого из четырёх подходов определите, возможно ли это, и почему.*

2.3 Полиморфизм по конвенции вызова

Как мы уже обсуждали выше 2.2.2, параметрический полиморфизм в Haskell реализуется следующим образом: все значения хранятся в куче и передаются в полиморфные функции по указателю. Однако, если для вычислительного кода важна производительность, такой подход не годится ввиду большой нагрузки на подсистему управления памятью и множества индирекций. Поэтому Haskell позволяет также писать код с использованием unboxed значений.

А если конвенция вызова не принципиальна, можно по ней абстрагироваться и писать один код для boxed и unboxed значений Eisenberg and Peyton Jones [2017].

2.3.1 Разновидности runtime представлений в Haskell

	Boxed	Unboxed
Lifted	*Int* *Bool*	
Unlifted	$ByteArray_{\#}$	$Int_{\#}$ $Char_{\#}$

Рис. 11: Виды значений в Haskell с примерами Eisenberg and Peyton Jones [2017].

На рисунке 11 можно увидеть классификацию значений в Haskell с примерами типов. **Unboxed типы** — их значения удерживаются и передаются по значению. **Boxed**, соответственно, наоборот, передаются по указателю и хранятся в куче. Обычный `Int` является просто декларацией следующего вида, где `I#` — это обычный конструктор с необычным именем, содержащий unboxed значение.

```
data Int = I# Int#
```

Lifted типы — содержат $\perp$ в качестве значения. Иначе говоря, могут содержать отложенные вычисления (это для них специальным образом обеспечивают компилятор и рантайм). **Unlifted типы** — наоборот, не могут быть отложенными. Операции, производящие значения unlifted типов всегда энергичные. Свойство lifted/unlifted называют **levity**. Чтобы распространить дальнейшее изложение на энергичные языки, можно levity заменить на boxity и всё останется справедливым.

`#` в именах типов и функций — это конвенция, показывающая, что где-то рядом происходит работа с unlifted значениями[43].

Также в Haskell есть unboxed кортежи, которых не существует на этапе исполнения. Например, следующая функция как бы возвращает пару значений, но в действительности компилятор может их разместить, например, в паре регистров. Соответственно, паттерн-матчинг по таким кортежам, просто позволяет сослаться на каждое из этих значений.

```
divMod# :: Int -> Int -> (# Int, Int #)
case divMod# n k of (# quot, rem #) -> ...
```

Соответственно, во время исполнения нет никакого различия между по-разному вложенными unboxed кортежами, все они представлены одним и тем же набором значений:

```
(# A, (# B, C #) #) ≡ (# (# A, B #), C #) ≡ (# A, B, C #)
```

[43]Нужно подключить расширение MagicHash, чтобы пользоваться `#` в идентификаторах.

2.3.2 Классификация значений по runtime представлению

Значения различных типов могут быть на этапе исполнения устроены по-разному. То есть нам нужна некоторая система классификации типов. Но такая система в Haskell уж есть — кайнды. Опишем в виде структур данных предметную область, а потом продвинем на нужный уровень (2.1.6).

Стандартная библиотека Haskell предоставляет следующие типы данных[44]:

```
TYPE :: RuntimeRep -> Type

data Levity = Lifted | Unlifted

data RuntimeRep = BoxedRep Levity
                | IntRep | DoubleRep
                | TupleRep [RuntimeRep]
                | SumRep [RuntimeRep]
                | ...

type LiftedRep = BoxedRep Lifted

type Type = TYPE LiftedRep
```

`TYPE` — это магический тип, определённый в компиляторе. Он параметризован runtime-представлением значений. Теперь привычный `Type` — это частный случай с boxed lifted значениями.

- `Int :: TYPE (BoxedRep Lifted)` или `:: Type`
- `IntRep` и `DoubleRep` соответствуют представлению численных констант (в зависимости от архитектуры процессора, целые числа и числа с плавающей запятой может быть необходимо располагать в различных специальных регистрах)
  `Int# :: TYPE IntRep`
- `Maybe Int :: Type`
- `Maybe :: Type -> Type`
- `TupleRep` и `SumRep` — unboxed алгебраические типы, представления параметризованы представлениями хранимых значений
  `(# Int, Bool #) :: TYPE (TupleRep '[LiftedRep, LiftedRep])`
- Хотя во время исполнения вложенность кортежей неразличима, на уровне кайндов она для простоты сохраняется, и типы по-разному вложенных кортежей не унифицируются

```
(# Int#, (# Int, Double# #) #)
  :: TYPE (TupleRep '[IntRep, TupleRep '[LiftedRep, DoubleRep]])
```

[44] https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/representation_polymorphism.html

Упражнение 24 *Какой кайнд у типа* `(# Int#, Bool #)`*? Проверьте себя в GHCi, включив расширения MagicHash и UnboxedTuples.*

2.3.3 Representation polymorphism

Выставив runtime-представление в структуре кайндов, мы теперь можем параметризоваться по ним. Например, кайнд функциональной стрелки выглядит следующим образом[45]:

```
ghci> :k (->)
(->) :: forall {q :: RuntimeRep} {r :: RuntimeRep}. TYPE q -> TYPE r -> Type
```

Упражнение 25 *Подумайте, почему функция имеет boxed тип. Может ли быть иначе? Может ли это быть полезным?*

К сожалению, Haskell выставляет довольно строгое ограничение: связыватели не могут иметь тип, полиморфный по runtime представлению. Можно легко предположить, почему, — нельзя сгенерировать код функции для работы с параметром произвольного рантайм-представления. Это можно решить только мономорфизацией 2.2.1, но Haskell избегает этого подхода[46]. Сообщество также пытается найти другие решения[47] (что-то вроде 2.2.4).

Например, изначально оператор аппликации был обобщён только по возвращаемому типу. Это не порождает проблем, так как вызывающий код сможет вывести представление и сгенерировать подходящий код:

```
($) :: forall r a (b :: TYPE r). (a -> b) -> a -> b
f $ x = f x
```

Однако, было замечено, что для оператора аппликации можно получить другую реализацию, не использующую levity-полиморфное связывание[48]:

```
($) :: forall ra rb (a :: TYPE ra) (b :: TYPE rb). (a -> b) -> a -> b
($) f = f
```

Упражнение 26 *Функция* `error` *полиморфна по представлению результата:*

```
error :: forall (r :: RuntimeRep) (a :: TYPE r). String -> a
```

Почему это не нарушает ограничение на связыватели?

Таким образом, в Haskell полиморфизм по представлениям несколько вырожден и помогает лишь в небольшом количестве случаев, однако немаловажных. Если позволить мономорфизацию по `RuntimeRep` параметрам, получится система аналогичная гибридной реализации параметрического полиморфизма 2.2.3, только с большим контролем со стороны программиста над мономорфизацией.

[45] Выключить упрощения: `ghci> :set -fprint-explicit-foralls -fprint-explicit-runtime-reps`
[46] https://gitlab.haskell.org/ghc/ghc/-/issues/14917
[47] https://mail.haskell.org/pipermail/haskell-cafe/2023-January/135770.html
[48] https://gitlab.haskell.org/ghc/ghc/-/merge_requests/10131

3 Специальный (ad-hoc) полиморфизм

Как-то Joe Fasel в разговоре с Philip Wadler высказал идею того, что перегрузка функций (overloading) должна находить своё отражение в типах. Wadler понял его неправильно Hudak et al. [2007]. Но то, что он понял, — оказалось классами типов Wadler and Blott [1989].

Christopher Strachey ввёл классификацию полиморфизма на две категории Strachey [2000]. Параметрический — один и тот же код работает с данными различных типов. **Специальный (ad-hoc) полиморфизм** — код выбирается в зависимости от типа. Например, один и тот же символ умножения по-разному действует на целые числа и на числа с плавающей точкой.

Перегрузка в языках обозначает возможность назвать несколько функций с различными наборами входных параметров одинаково. В месте вызова компилятор статически определяет по типам аргументов, какую из них действительно следует вызвать.

```
1 std::string toString(int x) { ... }
2 std::string toString(std::string fmt, double d) { ... }
```

Классы типов обязуют сначала задекларировать именованную сущность (собственно, класс типов), включающую в себя пачку деклараций функций, которые могут быть перегружены для различных типов.

```
1 class Show a where
2   show :: a -> String

3 instance Show Int where
4   show :: Int -> String
5   show = ...
```

Необходимо заметить, что декларация класса типов содержит формальный типовой параметр, по вхождениям которого в тип функции, собственно, выбирается перегрузка. Таких параметров может быть много[49], они могут иметь стрелочные кайнды. Например, в случае класса типов `Applicative`, выбор реализации операции `pure` будет происходить по типовому конструктору результата, то есть даже не по полноценному типу.

```
1 class Functor f => Applicative (f :: Type -> Type) where
2   pure :: a -> f a
3   ...

4 instance Applicative Maybe where
5   pure :: a -> Maybe a
6   ...
```

Классы типов позволяют ограничить набор возможных типовых аргументов теми, для которых реализован инстанс нужного класса типов. Обычная перегрузка сама по себе такого

[49] Расширение MultiParamTypeClasses.

механизма не задаёт, но языки могут сочетать её с полиморфизмом и дополнительными ограничениями: например, concepts и `requires` в C++ позволяют потребовать корректность определённых выражений[50].

```
showPrefixed :: Show a => a -> String -> String
```

Если сравнивать классы типов с переопределением (overriding) в ООП языках, то разрешение вызова виртуальной функции происходит с использованием таблицы, хранящейся объекте первого параметра (получателя вызова, receiver). Классы типов же опираются исключительно на тип, поэтому, например, возможно определение констант в классах типов[51]:

```
class Bounded a where
  minBound :: a
  maxBound :: a
```

В то же время, классы типов не являются типами, а, скорее, предикатами на типах. Тип удовлетворяет такому предикату, или свойству, если для него есть соответствующий инстанс. Поэтому, в частности, привычный способ в ООП создать гетерогенную коллекцию элементов, имеющих общий интерфейс, напрямую не сработает с классами типов. Например, такой тип не будет корректным: `[Show]`. Мы вернёмся к этой проблеме в 3.4.2.

В этой главе мы сначала разберём, как классы типов устроены внутри: это словари функций, которые компилятор передаёт неявно и конструирует автоматически. Затем посмотрим, как та же идея выбора по типу переносится с кода на сами типы (семейства) и на ограничения (кайнд `Constraint`). В завершение рассмотрим, какие возможности языка и полезные техники выражают через классы типов.

3.1 Классы типов в языке

Несмотря на поразительное могущество, идея реализации классов типов крайне проста. Она была уже во всей полноте представлена в первой работе Wadler and Blott [1989]. В дальнейших работах уточнялся механизм вывода типов в виде сведения к классической системе типов в стиле Hindley-Milner Hall et al. [1996]. Остальные работы, в основном, предлагают огромное разнообразие различных расширений и приложений Jones et al. [1997].

3.1.1 Словари

Рассмотрим идею реализации классов типов на примере полиморфной сортировки. Сортировка для списка элементов конкретного типа пишется тривиально:

[50] `https://eel.is/c++draft/expr.prim.req`

[51] Современные ООП языки, тем не менее, стремятся поддержать статические члены в интерфейсах, что делает их ближе к классам типов и позволяет делать похожие вещи. Например, статические требования в протоколах Swift (`https://docs.swift.org/swift-book/documentation/the-swift-programming-language/protocols/`) или статические абстрактные члены интерфейсов в C# (`https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/tutorials/static-virtual-interface-members`).

```
sort :: [Int] -> [Int]
sort = \case [] -> []; x:xs -> insert x (sort xs)
  where
    insert x xs = let (l, r) = List.partition (<) x) xs in l ++ x : r
```

В реализации единственная информация о типе, которой мы пользуемся — порядок на его обитателях. Таким образом, при переходе к полиморфной сортировке, нам нужно принять словарь с предикатами, задающими нужный порядок для данного типа.

```
data OrdDict a = OrdDict { less :: a -> a -> Bool }

sort :: OrdDict a -> [a] -> [a]
sort d@OrdDict{ less } = \case [] -> []; x:xs -> insert x (sort d xs)
  where
    insert x xs = let (l, r) = List.partition (`less` x) xs in l ++ x : r
```

Теперь, чтобы воспользоваться сортировкой на списке чисел, нужно сконструировать нужный рекорд и вызвать с ним функцию на списке конкретных типов:

```
intOrd :: OrdDict Int
intOrd = OrdDict { less = (<) }

ghci> sort intOrd [3, 2, 1]
[1,2,3]
```

Возможна ситуация, когда инстанс для одного типа зависит от инстанса для другого. Например, порядок на списках можно получить автоматически, зная порядок на элементах. В случае словарей мы это моделируем функцией между словарями:

```
listOrd :: OrdDict a -> OrdDict [a]
listOrd d = OrdDict { less = ... less d ... }
```

Теперь мы можем сортировать список списков, конструируя нужный словарь:

```
ghci> sort (listOrd intOrd) [[3, 2], [2, 1], [0]]
[[0],[2,1],[3,2]]
```

Упражнение 27 *Допишите `listOrd` так, чтобы списки сравнивались лексикографически.*

Сравнение явной передачи словарей и классов типов можно увидеть в следующей таблице:

1. Определение словаря функций

```
data MyOrd a = MyOrd
  { less :: a -> a -> Bool }
```

2. Экземпляр словаря для конкретного типа
   - Именованное значение

```
intMyOrd :: MyOrd Int
intMyOrd = MyOrd { less = (<) }
```

3. Явный параметр функции

```
sort :: MyOrd a -> [a] -> [a]
```

4. Передаётся пользователем

```
test = sort intMyOrd [3, 2, 1]
```

1. Определение класса типов

```
class MyOrd a where
  less :: a -> a -> Bool
```

2. Объявление типа представителем класса типов
   - Не имеет имени

```
instance MyOrd Int where
  less = (<)
```

3. Неявный параметр функции

```
sort :: MyOrd a => [a] -> [a]
```

4. Передаётся компилятором

```
test = sort [3, 2, 1]
```

Таким образом, словарь — это **свидетель (witness)** или доказательство того, что тип удовлетворяет ограничению.

Упражнение 28 *Какой словарь будет соответствовать higher-kinded классу типов* `Functor`*?*

3.1.2 Неявные аргументы

Можно думать так, что слева от `=>` передаются неявные аргументы функций, выводимые компилятором из контекста. То есть, например, не стоит удивляться вхождениям `=>` в типе аргумента, это просто функция с неявным аргументом. Так, следующий код не скомпилируется, потому что в месте использования переменной `x` нет значения типа `Show b`:

```
f :: (Show b => b) -> b
f x = x -- ошибка
```

Можно это значение принять в функции `f`, тогда оно автоматически пропагируется в `x`:

```
f :: Show b => (Show b => b) -> b
f x = x
```

Haskell даёт возможность делать неявными и обычные аргументы функции[52]. Фактически, это реализация динамического связывания в статическом языке Lewis et al. [2000] (см. далее 5.3.1). Неявные аргументы берутся из скоупа по имени и подставляются автоматически:

```
sortBy :: (a -> a -> Bool) -> [a] -> [a]

sort :: (?cmp :: a -> a -> Bool) => [a] -> [a]
sort = sortBy ?cmp
```

[52] Расширение ImplicitParams.

Haskell также предоставляет возможность сохранять словари в структуры данных:

```haskell
data ShowDict a where
  ShowDict :: Show a => ShowDict a

f :: ShowDict b -> (Show b => b) -> b
f d x = case d of ShowDict -> x -- в области видимости доступен инстанс Show b
```

Упражнение 29 *Аргумент `x` имеет тип `Show b => b`, то есть ждёт словарь. В какой момент он его получает? Требует ли такая передача словаря ленивости языка, или достаточно передать функцию от словаря? Что вычисляется до вызова `f` в энергичном языке?*

3.1.3 Вывод инстансов

Чтобы вызвать ограниченно-полиморфную функцию, GHC производит вывод инстансов или, иначе говоря, автоматически конструирует свидетелей. Вывод инстансов тесно интегрирован с общей системой вывода типов Haskell Peyton Jones [2019].

В действительности вывод инстансов — это ни что иное, как *задача населения типа*. Действительно, после трансляции в Core (промежуточное представление в GHC), классы типов представляют собой словари функций. У нас в контексте имеются конкретные словари и функции, позволяющие из одних словарей получать другие. Требуется найти терм, конструирующий словарь нужного типа.

Пусть внутри функции `f :: Show a => ..` вызывается функция `g :: Show [a] => ..`, а инстанс для списков доступен из импортов[53]. Нужный словарь строится применением функции из инстанса к словарю из контекста (рис. 12).



Рис. 12: Население типа словаря: компилятор находит терм `d2 d1` типа `ShowDict [a]` и подставляет его в вызов `g` (вызов со словарём показан схематично).

[53] Инстансы можно импортировать пустым импортом: `import Module ()`.

Вывод инстансов происходит рекурсивно. Чтобы вывести `ShowDict [a]`, выводится сначала посылка `ShowDict a`. То есть получается рекурсия по структуре типа. Иначе говоря, вывод инстансов можно эксплуатировать как вычислительный примитив уровня типов. Так, например, мы можем опускать информацию из типов в термы (аналогично GHC.TypeLits)[54]:

```
type data Nat = Zero | Suc Nat

class KnownNat (n :: Nat) where
  natVal :: Int

instance KnownNat Zero where
  natVal = 0

instance KnownNat n => KnownNat (Suc n) where
  natVal = 1 + natVal @n

ghci> natVal @(Suc (Suc Zero))
-- выведется natVal {knownSuc (knownSuc knownZero)}
```

В общем случае процесс населения типа, как можно предположить по вычислительной аналогии, неразрешим. Поэтому GHC накладывает большое количество ограничений на вид инстансов, которые гарантируют тотальность вывода. Подробно эти ограничения описаны в Sulzmann et al. [2007a]. Также GHC предоставляет различные расширения, ослабляющие эти ограничения и перекладывающие часть ответственности на плечи программиста[55]. Если отказаться от гарантии завершимости вывода[56], можно легко написать разворот списка типов на этапе компиляции, как и любую другую функцию. Пусть класс `ShowT (tys :: [Type])` с методом `showTypes :: String` умеет печатать список типов (его реализацию оставим упражнением, см. 3.4.6):

```
class Reverse (acc :: [Type]) (tys :: [Type]) where
  showReverse :: String

instance ShowT acc => Reverse acc '[] where
  showReverse = showTypes @acc

instance Reverse (ty : acc) tys => Reverse acc (ty : tys) where
  showReverse = showReverse @(ty : acc) @tys

ghci> showReverse @'[] @'[Char, Int, Double]
"Double Int Char"
```

[54] Тип метода `natVal` не упоминает `n`, поэтому вызвать его можно только с явной типовой аппликацией. Такие сигнатуры разрешает расширение AllowAmbiguousTypes.

[55] `https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/instances.html`

[56] Расширение UndecidableInstances.

При компиляции вывод инстансов определяет порядок типов в аккумуляторе и строит терм, передающий нужные словари в `showReverse` и `showTypes`. Методы этих словарей — обычный код, который при исполнении строит строку; оптимизатор может дополнительно упростить его, если располагает нужной информацией.

Можно заметить, что процесс вывода классов типов очень похож на вычисление логических программ, например, на Prolog, только без backtracking'а (перебора различных вариантов решений в поисках подходящего). Как, впрочем, и вывод типов в Haskell Peyton Jones [2019] в целом: собранные по программе эквивалентности можно рассматривать как логическую программу, решение этой системы типовых уравнений — как исполнение этой программы.

Между классами типов и выводом типов существует интересная синергия (рис. 13)[57]. Исходя из термов, выводятся типы. Затем, исходя из типов, выводятся инстансы классов типов. То есть мы пишем какой-то интересный интеллектуальный код, а параллельно с нами компилятор выписывает неинтересный код.

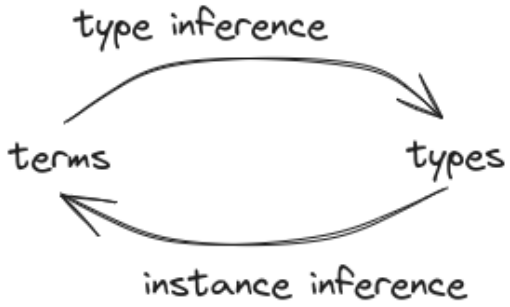


Рис. 13: Классы типов + вывод инстансов = кодогенерация.

Вывод инстансов опирается только на вид "головы" декларации — справа от `=>`, а ограничения слева применяются постфактум. Это можно использовать, чтобы писать более общие инстансы. Так, например, работает **constraint trick**[58], позволяющий разрешать ad-hoc-полиморфные функции в параметрически-полиморфном контексте.

3.1.4 Построение типа по значению

После того как мы научились опускать значения из типов, закономерно научиться обратному — поднимать значения в типы. Воспользуемся техникой, описанной в Kiselyov and Shan [2004]. Существует соответствующая библиотека Data.Reflection (см. далее 3.4.7).

В действительности мы, конечно, не можем честно получить синтаксически тип нужного размера, просто потому, что типы существуют строго до стадии исполнения. Однако, как мы знаем, словари классов типов имеют воплощение в рантайме (случай полиморфной

[57](youtube) Hackett: a metaprogrammable Haskell.
[58]`https://chrisdone.com/posts/haskell-constraint-trick/`

рекурсии — как раз пример, когда этого нельзя полностью избежать). Поэтому воспользуемся continuation-passing style, который будет подробно рассмотрен далее в главе 6: вместо того чтобы вернуть результат, примем продолжение, умеющее работать с любым типом с `KnownNat` (пользуемся типовыми абстракциями и аппликациями, см. 2.1)[59]:

```
reify :: Int -> (forall n. KnownNat n => a) -> a
reify n k
  | n <= 0 = k @Zero
  | otherwise = reify (n - 1) \ @n' -> k @(Suc n')
```

Продолжение, передаваемое в рекурсивный вызов, захватывает словарь для типа `n` и конструирует словарь для `Suc n`.

Наконец, можем написать следующую удивительную тождественную функцию, поднимающую сначала значение в тип, а потом опускающее тип обратно в термы:

```
wonderId :: Int -> Int
wonderId n = reify n (\ @t -> natVal @t)
```

Упражнение 30 *Что вернёт `wonderId (-3)`? Является ли `wonderId` тождественной функцией?*

3.1.5 Имплиситы и когерентность

Классы типов можно не делать специальным языковым механизмом, но вместо этого предоставлять на языковом уровне неявные параметры и население, достаточные для реализации классов типов.

Так, в Scala существует механизм имплиситов (implicits) Křikava et al. [2019][60]. Параметры функций могут быть помечены ключевым словом `implicit`, тогда Scala попытается их вывести самостоятельно с помощью доступных в области видимости `implicit` деклараций. Объявления переменных, функций и конструкторов объектов также могут быть помечены `implicit`, тогда они будут использоваться при населении. Теперь мы можем смоделировать словарь функций, например, с помощью интерфейсов (которые в Scala называются `trait`) и ООП синглтонов, чтобы получить классы типов Oliveira et al. [2010]:

```
// Пачка функций.
trait Show[T] {
    def show(x: T): String
}

// Обёртка для удобства вызова.
def show[T](x: T)(implicit ev: Show[T]): String = ev.show(x)
```

[59] В не самых свежих версиях GHC потребуется воспользоваться техникой `Proxy` из 2.1.7.

[60] Дизайн неявных параметров в Scala3 изменился (youtube) Scala Implicits Revisited, Martin Odersky.

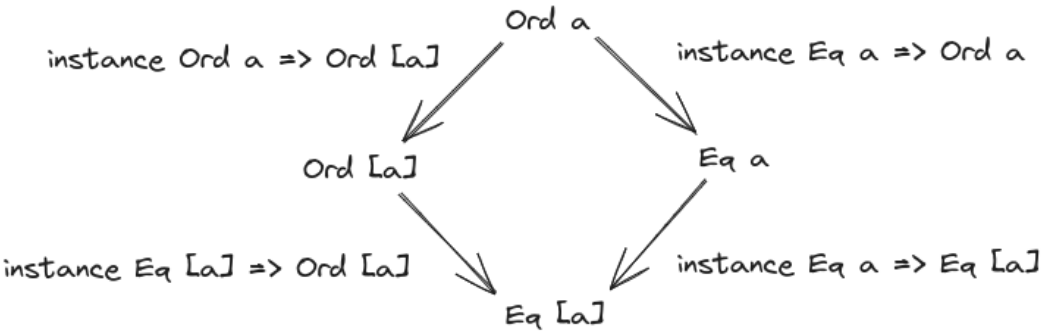


Рис. 14: Когерентность инстансов — диаграмма коммутирует.

```
// Объект-синглтон, значение для пачки функций.
implicit object intShow extends Show[Int] {
    def show(x: Int): String = x.toString
}

def showAll[T](xs: List[T])(implicit ev: Show[T]): String =
    xs.map(show(_)).mkString(", ")
```

Как мы говорили ранее (2.1), работа с типовыми параметрами похожа на работу с обычными. Также, вывод типов можно рассматривать как процесс восстановления пропущенных типовых аппликаций. Заметим, что примерно этим же занимается и механизм вывода имплиситов. Таким образом, при попытке сделать функции с имплиситами функциями первого класса, будут возникать сложности схожие со сложностями first-class полиморфизма (2.1.2).

В языках с зависимыми типами неявные параметры[61] особенно нужны, потому что, например, типы — это ровно такие же параметры функции, как и все остальные. Поэтому вывод типов — это фактически вывод неявных аргументов функций. Более того, зависимые функции, вместе с аргументами часто принимают доказательства каких-то свойств этих аргументов, которые тоже хочется по возможности выводить из контекста автоматически. Такой механизм вывода можно переиспользовать для эмулирования классов типов[62] Devriese and Piessens [2011]. В обратную сторону тоже работает — можно механизмы зависимой типизации эмулировать классами типов McBride [2002].

Как мы увидим далее, неявные параметры сами по себе тоже нужны как статическая аппроксимация динамических свободных переменных для реализации системы эффектов (см. далее 8). Однако, иметь классы типов отдельной языковой возможностью всё же полезно, несмотря на то, что они, вроде бы, представляют собой те же неявные параметры (пусть и с рекурсивным механизмом населения). Так, можно поддержать важное свойство — **когерентность инстансов (coherence)**: словари для одного и того же ограничения, полученные

[61] https://agda.readthedocs.io/en/v2.7.0.1/language/implicit-arguments.html
[62] https://agda.readthedocs.io/en/v2.7.0.1/language/instance-arguments.html

разными способами, неотличимы (рис. 14). В обычной модели классов типов декларации инстансов глобальны, а выбор реализации однозначен; расширения, допускающие перекрывающиеся или некогерентные инстансы, могут нарушить это свойство[63]. Соответственно, не имеет значения происхождение того или иного инстанса. Иначе говоря, об этом можно не думать, это снимает существенное количество когнитивной нагрузки и упрощает рефакторинг[64]. В то время как остальные подходы требуют трепетного отношения к контексту вызова, потому что из него может прийти неожиданная реализация.

Упражнение 31 *Множество* `Set a` *хранит элементы упорядоченными и при вставке и поиске пользуется порядком* `Ord a`*. Что произойдёт, если множество построить с одним порядком на элементах, а искать в нём с другим? Почему обычная когерентная модель классов типов в Haskell исключает такую ситуацию, а выбор имплисита из локального контекста допускает её?*

3.1.6 Правила (rules) и специализация

GHC позволяет прямо в коде, с помощью специально прагмы, указывать оптимизирующие правила переписывания для компилятора[65] Jones et al. [2001]. Например:

```
{-# RULES
  "map/map"    forall f g xs.  map f (map g xs) = map (f . g) xs
  "map/append" forall f xs ys. map f (xs ++ ys) = map f xs ++ map f ys
 #-}
```

Первый закон представляет собой не что иное, как закон функторов. В идеале, мы формулируем законы на этапе дизайна Maguire [b], проверяем их выполнение с помощью property-based testing[66], а потом используем их для оптимизаций.

Можно переписать полиморфную версию функции на специализированную, если типы подходят. Для этого нужно реализовать специализированную версию (совпадение семантики — полностью ответственность программиста) и задать соответствующее правило переписывания:

```
genericLookup :: Ord a => Table a b   -> a   -> b
intLookup     ::          Table Int b -> Int -> b

{-# RULES "genericLookup/Int" genericLookup = intLookup #-}
```

Основной эффект такой оптимизации — гарантированное превращение динамических вызовов функций классов типов в статические (потому что тип известен, следовательно, — и соответствующий ему словарь).

[63] https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/instances.html#overlapping-instances

[64] Edward Kmett - Type Classes vs. the World.

[65] https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/rewrite_rules.html

[66] (youtube) John Hughes - Keynote: How to specify it!

3.1.7 Отступление: дефункционализация

Дефункционализация (defunctionalization) — техника избавления от функций высших порядков в программе[67] Xia. Впервые предложена в Reynolds [1972, 1998].

Идея заключается в том, чтобы заменить каждое создание лямбда-функции вызовом конструктора некоторого алгебраического типа данных. А каждый call-cite функции заменить на вызов специальной first-order функции `apply`, интерпретирующей данный алгебраический тип.

Рассмотрим пример из функции высших порядков `map` и двух колсайтов, создающих лямбда-функции:

```
1 map :: (Int -> Int) -> [Int] -> [Int]
2 map f = \case [] -> []; x:xs -> f x : map f xs

3 example1 xs = map (\x -> x + 1) xs
4 example2 y xs = map (\x -> x * y) xs
```

Для каждого лямбда-литерала заводим по конструктору, хранящему замыкание. Аппликацию заменяем на вызов `apply`. Таким образом получили функцию `map` первого порядка.

```
1 data Fun = F1 | F2 Int
2 apply :: Fun -> Int -> Int
3 apply df x = case df of F1 -> x + 1; F2 y -> x * y

4 map :: Fun -> [Int] -> [Int]
5 map df = \case [] -> []; x:xs -> apply df x : map df xs

6 example1 xs = map F1 xs
7 example2 y xs = map (F2 y) xs
```

Упражнение 32 *Добавим третий колсайт, в котором функция приходит снаружи:*

```
1 example3 f xs = map (\x -> f x + 1) xs
```

Как его дефункционализировать? Что для этого нужно знать об остальной программе?

3.1.8 Эмуляция полиморфизма высших порядков

Далеко не во всех языках есть полиморфизм высших порядков (2.1), но иногда он бывает полезен. Самое распространённое его применение — эмуляция классов типов стрелочных кайндов вроде `Monad`.

Заметим, что типовый конструктор кайнда `Type -> Type` — это функция на типах, принимающая один тип, и возвращающая другой. Применим дефункционализацию, чтобы избежать необходимости параметризовать один типовый конструктор другим Xia, Yallop and White [2014].

[67] https://en.wikipedia.org/wiki/Defunctionalization

Поставим в соответствие типовому конструктору `List` тип-"символ" `ListSym` (для примера используем Kotlin):

```
class ListSym
```

Заведём тип, соответствующий аппликации символа к типу, хранящий оригинальное значение со стёртым типом:

```
class Apply<Sym, T>(val value: Any)
```

Установим изоморфизм между изначальным типом, полученным типовой аппликацией конструктора, и новой аппликацией символа:[68]

```
fun <T> List<T>.to(): Apply<ListSym, T> = Apply(this)
fun <T> Apply<ListSym, T>.from(): List<T> = this.value as List<T>
```

Теперь мы можем объявить интерфейс монад и задать реализацию для списка с помощью объекта-синглтона:

```
interface Monad<M> {
    fun <T> pure(x: T): Apply<M, T>
    infix fun <T, R> Apply<M, T>.bind(k: (T) -> Apply<M, R>): Apply<M, R>
}

object ListMonad : Monad<ListSym> {
    override fun <T> pure(x: T): Apply<ListSym, T> = listOf(x).to()
    override fun <T, R> Apply<ListSym, T>.
            bind(k: (T) -> Apply<ListSym, R>): Apply<ListSym, R> =
        this.from().flatMap { k(it).from() }.to()
}
```

И наконец мы можем писать функции над произвольными монадами:[69]

```
fun <M> Monad<M>.go(x: Apply<M, Int>): Apply<M, Int> =
    x bind { it -> pure(it + 1) } bind { it -> pure(it + 2) }

fun test(xs: List<Int>): List<Int> = ListMonad.go(xs.to()).from()
```

Не лишним будет отметить, что результирующий код выглядит несколько чудовищно. Скорее всего, использование этой техники не окупает себя и нужно выбирать другой стиль программирования.

[68] Слева от точки в декларации указывается дополнительный аргумент функции с синтаксисом передачи совпадающим с вызовом метода на объекте. Из тела функции на него можно ссылаться с помощью `this`.

[69] Аргументы слева от точки умеют самостоятельно запрыгивать в последующие вызовы. Собственно говоря, они являются неявными параметрами.

3.2 Семейства

Идея ad-hoc полиморфизма в том, чтобы в зависимости от типа получать различный код. Семейства начинались как продолжение этой идеи в плоскость данных и типов. Так, ассоциированные синонимы типов — различные типы для различных индексов (типовых параметров) Chakravarty et al. [2005a]. Ассоциированные `data` — различные представления для различных индексов Chakravarty et al. [2005b]. В конце концов эти идеи были обобщены до открытых семейств Schrijvers et al. [2008], потом были введены закрытые Eisenberg et al. [2014].

Можно считать, что семейства[70][71] — это типовые конструкторы, задающие множество типов. Конкретный тип из множества можно выбрать, передав типовой параметр, называемый **индексом**. Сравните с обычными полиморфными конструкторами типа, которые ведут себя одинаково вне зависимости от типовых параметров.

Большое количество интересных примеров использования можно найти, например, в Kiselyov et al. [2010].

3.2.1 Data families

Data families позволяют выбирать декларацию алгебраического типа в зависимости от типового индекса. Например, для более эффективной реализации структур данных. Это напоминает специализацию шаблонов в C++.

```
1 data family XList elem
2 data instance XList () = UnitList Int -- достаточно хранить длину
3 data instance XList Bool = BoolList ByteArray -- по биту на элемент
```

Единственный способ работать с data family — разместить реализацию в классе типов, чтобы она сопровождала значение с того момента, когда типовой индекс конкретный.

```
1 class XListOp elem where
2   xelem :: elem -> XList elem -> Bool

3 instance XListOp () where
4   xelem () (UnitList size) = size > 0

5 instance XListOp Bool where
6   xelem key (BoolList bs) = ...

7 xelemAll :: XListOp elem => XList elem -> [elem] -> Bool
8 xelemAll xs = all (`xelem` xs)
```

К сожалению, в отличие от специализации шаблонов, нельзя задать определение по-умолчанию вроде `data instance XList a = AList [a]`.

[70] https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_families.html
[71] https://serokell.io/blog/type-families-haskell

3.2.2 Synonym families

Synonym families (семейства типов) фактически являются функциями на типах. Они бывают открытыми, закрытыми и ассоциированными. Открытые семейства открыты в том смысле, что инстансы можно писать отдельно от декларации, подобно классам типов. Закрытые, наоборот, полностью описываются в одном месте, кейсы упорядочены сверху вниз, что несколько ослабляет ограничения на паттерны.

```
type family Plus (n :: Nat) (m :: Nat) :: Nat where
  Plus Zero m = m
  Plus (Suc n) m = Suc (Plus n m)
```

Чтобы посмотреть, во что вычисляется нечто на уровне типов, можно воспользоваться следующей командой в ghci:

```
ghci> :k! Plus (Suc Zero) (Suc (Suc Zero))
Plus (Suc Zero) (Suc (Suc Zero)) :: Nat
= Suc (Suc (Suc Zero))
```

Упражнение 33 *Реализуйте закрытое семейство* `Length :: [Type] -> Nat`*, вычисляющее длину списка типов.*

Ассоциированные семейства работают аналогично, только объявляются в рамках некоторого класса типов, являясь некоторой функциональной альтернативой функциональным зависимостям между параметрами класса[72] Jones [2000] (которые выглядят скорее реляционно). Иначе говоря, позволяют поставить в соответствие типу, для которого написан инстанс, другой тип. Например, коллекции — тип её элементов:

```
class Container c where
  type Elem c
  elements :: c -> [Elem c]

instance Container [a] where
  type Elem [a] = a
  elements = id

instance Container ByteString where
  type Elem ByteString = Word8
  elements = ByteString.unpack
```

[72] Расширение FunctionalDependencies.

В современных языках часто встречаются ассоциированные семейства под видом ассоциированных типов[73]. Так, Swift сильно полагается на ассоциированные типы: типовых параметров у протоколов (интерфейсов) в нём нет[74]. В то же время Scala пытается отслеживать, в присутствии экзистенциальных типов (интерфейсов), из какого именно значения пришёл тот или иной ассоциированный тип с помощью path-dependent types Amin et al. [2014].

3.2.3 Инъективные семейства

Семейства типов отличаются от типовых конструкторов примерно так же, как функции от конструкторов данных. Конструкторы пассивны и не редуцируются (`Maybe Int`), в то время как функции вычисляются в какой-то результат (e.g. `F Int ∼ Bool`). В частности, как и функции, семейства не обязательно инъективны.

Для типовых конструкторов, зная, что сконструированные ими типы эквивалентны, можно вывести, что типовые аргументы эквивалентны тоже. Например:

```
Maybe a ∼ Maybe b ⇒ a ∼ b
```

Очевидно, что для семейств типов это свойство по умолчанию не выполняется:

```
type family NonInjective a where
  NonInjective Int = Double
  NonInjective Char = Double
```

Haskell предоставляет явный синтаксис для объявления инъективных семейств типов, напоминающий функциональные зависимости в классах типов[75] Stolarek et al. [2015]. Конечно, компилятор проверит, что реализация инъективна. Синтаксис требует связать результат именем через равенство и указать, аналогично функциональным зависимостям, что результат определяет какие-то из типовых индексов семейства:

```
type family InjectiveB a b = r | r -> b
  ...
```

3.2.4 Семейства первого класса

Помимо инъективности, семейства типов также не обязательно обладают свойством **generativity**, критически важным для вывода типов, — один и тот же результат не обязательно получен из одного и того же семейства:

```
f a ∼ g a ⇒ f ∼ g
```

[73] Ассоциированные типы являются фактически экзистенциальными типами, их связывает с ассоциированными семействами логический процесс сколемизации.

[74] (youtube) 2017 LLVM Developers' Meeting: "Implementing Swift Generics"

[75] Расширение TypeFamilyDependencies.

Вместе инъективность и generativity — **matchability**. Когда Haskell работает с типом стрелочного кайнда, он подразумевает, что этот тип matchable. Соответственно, семейства не могут передаваться в качестве параметров, а все их вхождения должны быть полностью применёнными ко всем аргументам (fully saturated). Либо нужно явно указать, что семейство возвращает конструктор (в этом конкретном месте семантика зависит от переноса аргументов направо от `::`):

```
type family ToCtor (s :: Symbol) :: Type -> Type where
  ToCtor "maybe" = Maybe
  ToCtor "identity" = Identity
```

Одним из способов обойти это ограничение является дефункционализация семейств Xia, Eisenberg and Stolarek [2014]. Как мы и обсуждали ранее 3.1.7, вместо функции первого класса заводится некоторый символ, обозначающий её, и функция интерпретации, умеющая сделать действие, соответствующее этому символу. В данном случае функцией интерпретации будет открытое семейство `Eval` [Maguire, a, глава 10].

Символ функции, вычисляющей результат кайнда `a`, будем представлять типовым конструктором, результат которого имеет кайнд `Exp a`. Реализацию функции зададим инстансом семейства `Eval` для этого символа:

```
type Exp a = a -> Type
type family Eval (e :: Exp a) :: a

data Snd :: (a, b) -> Exp b
type instance Eval (Snd '(a, b)) = b
```

Декларация `data` обязана в итоге возвращать кайнд `Type`, поэтому объявить символ с кайндом `(a, b) -> b` нельзя. Вместо этого результирующий кайнд выражения записан фантомным параметром при `Exp`, который никогда не применяется: полный кайнд символа — `Snd :: (a, b) -> b -> Type`. По этому параметру семейство `Eval` и узнаёт, какого кайнда результат вернуть. Таким образом, кайнд `Exp` — это чисто трюк, позволяющий приписать кайнды таким программам на уровне типов.

Символы — обычные типовые конструкторы, поэтому их, в отличие от семейств, можно передавать в другие функции не до конца применёнными. Так, можно написать функцию высшего порядка на типах:

```
data Map :: (a -> Exp b) -> [a] -> Exp [b]
type instance Eval (Map f '[]) = '[]
type instance Eval (Map f (x : xs)) = Eval (f x) : Eval (Map f xs)

ghci> :k! Eval (Map Snd '[ '(Int, Bool), '(Char, Double) ])
= [Bool, Double]
```

Сравните с дефункционализацией на уровне термов: конструкторам `F1` и `F2` соответствуют декларации `Snd` и `Map`, а функции `apply` — семейство `Eval`. Поскольку семейство открытое, новые функции можно добавлять, не трогая уже написанные.

Упражнение 34 *Реализуйте в этом стиле свёртку списка типов:*

```
data Foldr :: (a -> b -> Exp b) -> b -> [a] -> Exp b
```

В Haskell ведутся работы[76] по устранению saturation ограничения Kiss et al. [2019]. Для этого нужно различать matchable и unmatchable типовые функции. Это предлагается делать дополнительным индексированием стрелочных кайндов: $\to\equiv\to^M$ и $\twoheadrightarrow\equiv\to^U$. И, конечно, эти индексы могут быть полиморфными.

```
data Matchability = Matchable | Unmatchable

hMap
  :: forall (m :: Matchability) (c :: Type -> Constraint)
   . forall (f :: Type ->^m Type) (as :: [Type])
   . All c as => (forall a. c a => a -> f a) -> HList as -> HList (Map f as)
```

3.3 Кайнд `Constraint`

Давно появлялись предложения добавить в GHC поддержку синонимов для констреинтов, семейств констреинтов и т.д Orchard and Schrijvers [2010]. В итоге был предложен[77] и реализован[78] некоторый механизм унификации типов и констреинтов. Таким образом, всё, что работало для типов, стало работать и для констреинтов.

В GHC был добавлен специальный кайнд `Constraint`[79]:

- Класс типов конструирует констреинт: `Monad :: (Type -> Type) -> Constraint`;
- Эквивалентность является констреинтом: `(a ~ b) :: Constraint`;
- Пустой кортеж констреинтов является констреинтом: `() :: Constraint`;
- Кортеж констреинтов является констреинтом: `(Eq a, a ~ b) :: Constraint`.

Теперь, например, мы можем реифицировать словарь как объект языка:[80]

```
data Dict (c :: Constraint) where
  Dict :: c => Dict c
```

Вспомним гетерогенный список `HList`, рассмотренный ранее (2.1.6). Эта структура данных является first-class аналогом variadic generics в C++ или Swift[81] (собственно, смысл вариадиков — не работать `HList`-подобными структурами напрямую). Например, мы можем написать `map` для такой структуры, если все типы удовлетворяют определённому ограничению. Для этого сначала реализуем семейство, генерирующее кортеж констреинтов для каждого типа из списка:

[76] GHC proposal: Unsaturated Type Families.
[77] https://gitlab.haskell.org/ghc/ghc/-/wikis/kind-fact
[78] http://blog.omega-prime.co.uk/2011/09/10/constraint-kinds-for-ghc/
[79] Расширение ConstraintKinds.
[80] https://hackage.haskell.org/package/constraints-0.14.2/docs/src/Data.Constraint.html#Dict
[81] https://github.com/swiftlang/swift-evolution/blob/main/proposals/0398-variadic-types.md

```haskell
type family All (c :: k -> Constraint) (tys :: [k]) :: Constraint where
  All c '[] = ()
  All c (ty : tys) = (c ty, All c tys)

-- All Show [Int, Double] ~ (Show Int, (Show Double, ()))
```

Теперь можем реализовать `map`:

```haskell
hmap :: forall c res tys . All c tys
     => (forall ty . c ty => ty -> res) -> HList tys -> [res]
hmap f = \case
  HNil -> []
  HCons x xs -> f x : hmap @c f xs

ghci> hmap @Show show (HCons (1 :: Int) $ HCons 'a' HNil)
["1","'a'"]
```

Упражнение 35 *Реализуйте сравнение гетерогенных списков:*

```haskell
heq :: All Eq tys => HList tys -> HList tys -> Bool
```

Откуда в рекурсивном вызове берётся словарь для хвоста списка?

Больше такого рода упражнений в гетерогенных конструкциях можно найти в de Vries and Löh [2014].

Констреинты также могут быть параметрически-полиморфными[82] Bottu et al. [2017]:

```haskell
data Rose f x = Rose x (f (Rose f x))

instance (Eq a, forall b. Eq b => Eq (f b)) => Eq (Rose f a) where
  Rose x1 rs1 == Rose x2 rs2 = x1 == x2 && rs1 == rs2
```

Типовые переменные, обозначающие констреинты, компилятор не выводит. Обычную типовую переменную он восстанавливает по типам аргументов, а переменная-констреинт встречается только слева от `=>`, и подсказать её значение нечему: компилятор ищет свидетельство для уже известного констреинта, но не подбирает констреинт под имеющиеся свидетельства. Поэтому, если в типе функции есть `forall (c :: Constraint) . c => ...`, значение `c` приходится указывать вручную типовой аппликацией. Именно так мы вызывали `hmap`: без `@Show` вызов не типизируется, хотя первым аргументом и передана функция `show`.

Словари существуют во время исполнения, а значит, у констреинтов, как и у типов, есть рантайм-представление (2.3)[83]:

[82] https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/quantified_constraints.html
[83] https://hackage.haskell.org/package/base-4.21.0.0/docs/GHC-Exts.html#t:CONSTRAINT

```haskell
CONSTRAINT :: RuntimeRep -> Type
type Constraint = CONSTRAINT LiftedRep
```

Обычный словарь представлен указателем на структуру в куче, отсюда `LiftedRep`. Параметр позволяет свидетельству иметь и другое представление, как `TYPE` позволяет это обычным значениям. Задуманные применения — неявные параметры неупакованных типов, передаваемые без упаковки, и строгие классы, словари которых не бывают отложенными вычислениями[84].

3.4 Использование ad-hoc полиморфизма

Часто языки, имеющие что-то напоминающее классы типов, стремятся выразить через них как можно больше других языковых возможностей и полезных техник. Оказывается, это на удивление мощный механизм. Рассмотрим в этом параграфе некоторые избранные примеры.

3.4.1 Сериализация

Классическим примером использования классов типов является сериализация. Проблема в том, что десериализация производится, когда самого объекта ещё нет (ущербный Java подход заполнения объекта дефолтными значениями с последующей мутацией мы не рассматриваем). Поэтому нет возможности написать ООП интерфейс Serializable.

Стандартная библиотека сериализации в Kotlin[85] предоставляет сущность `KSerializer`, которая является интерфейсом для отдельного объекта-сериализатора нашего типа (для эффективности тут используется CPS в виде потоков событий encoder и decoder, вернёмся к этому подходу далее 4.4.1):

```kotlin
interface KSerializer<T> {
    fun serialize(encoder: Encoder, value: T)
    fun deserialize(decoder: Decoder): T
}
```

Очевидно, чтобы сконструировать сериализатор полиморфного типа, нужны сериализаторы типов-параметров. Что уже отчётливо напоминает классы типов.

```kotlin
class PairSerializer<K, V>(
    keySerializer: KSerializer<K>,
    valueSerializer: KSerializer<V>,
): KSerializer<Pair<K, V>> { ... }
```

[84] GHC proposal: Generalising Type and Constraint. На момент GHC 9.10 эти возможности ещё не реализованы.
[85] https://github.com/Kotlin/kotlinx.serialization

Однако в Kotlin нет классов типов, а значит создавать сериализаторы придётся вручную. С этим, однако несколько помогает пачка технологий: `inline fun`, `reified` дженерики, рефлексия и компиляторный плагин библиотеки, который, однако не безопасен с точки зрения типов.

3.4.2 Экзистенциальные типы

Квантор существования в типах является механизмом инкапсуляции [Pierce, 2002, глава 24]. Так, мы можем в типе τ заменить вхождения подтипа σ на переменную α и получить $\exists\alpha.\,\tau'$, где $[\alpha \mapsto \sigma]\,\tau' \equiv \tau$. Таким образом, мы скрыли от пользователя часть типа.

Например, можно с помощью экзистенциальных типов привести разные типы к одной форме и сложить их значения в один список. В Haskell экзистенциальный тип выражают через `data` декларацию, в которой полиморфный конструктор данных и мономорфный тип[86]. Так, тривиальный экзистенциальный тип $\exists\alpha.\,\alpha$ можно получить в Haskell в виде типа `Any`, заданного следующим образом:

```
1  data Any where
2    Any :: forall a . a -> Any  -- логически эквивалентно (exists a . a) -> Any

3  list :: [Any]
4  list = [Any 42, Any "Hello", Any (Just Nothing)]
```

В месте деконструирования `Any`, будет доступно значение некоторого неизвестного типа. Очевидно, с таким значением ничего сделать нельзя. Однако, помимо значения, можно положить в конструктор свидетельство о том, что этот неизвестный тип удовлетворяет некоторому классу типов[87].

```
1  data Has (c :: Type -> Constraint) where
2    Has :: c a => a -> Has c
```

Значение типа `Has` свидетельствует о том, что существует некоторый населённый тип `a`, который принадлежит определённому классу типов. Например, рассмотрим `Show`:

```
1  showAll :: [Has Show] -> String
2  showAll = List.intercalate "," . map \(Has x) -> show x
```

В общем случае, чтобы элиминировать такой тип данных нужны типы высших рангов [Pierce, 2002, глава 24.3]:

```
1  foldHas :: Has c -> (forall a . c a => a -> b) -> b
2  foldHas (Has x) k = k x
```

[86] Подобно идиоме type erasure в C++.

[87] Подобно Rust fat pointers (пара из указателя на данные и указателя на таблицу виртуальных функций) и ООП объектам.

Подробнее можно посмотреть в [Maguire, a, глава 7] и [Pierce, 2002, глава 24].

Упражнение 36 *Почему нельзя написать функцию* `unHas :: Has c -> a`*, достающую значение из обёртки? Чем от неё отличается* `foldHas`*?*

3.4.3 Разрешение имён

Процесс разрешения имён (name resolution) в языках программирования определяет, с какой программной сущностью связать то или иное употребление имени. Разрешение имён рассматривает импорты, иерархию пространств имён и скоупов, типы выражений... Как правило, это сложный процесс, неотделимый от вывода типов.

Однако, в GHC стадия разрешения имён довольно простая и отрабатывает до вывода типов. На её тривиальную суть намекает её название — Renamer — она просто переписывает имена в программе на fully-qualified имена, опираясь на импорты.

С одной стороны, простота — это хорошо. С другой — строгое отделение от вывода типов накладывает неприятное ограничение: типы не могут участвовать в разрешении имён. Наиболее остро эта проблема стоит с метками полей в рекордах. Приходится называть все поля в модуле по-разному, чтобы избежать клешей.

Чтобы заставить разрешение имён зависеть от типов, Haskell снова прибегает к классам типов. А именно, определяется класс типов `IsLabel`, который зависит от символа и ожидаемого типа:

```
class IsLabel (s :: Symbol) a where
  fromLabel :: a
```

Для вызова `fromLabel` есть синтаксический сахар[88]:

```
#name ≡ fromLabel @"name"
```

Теперь разрешение имени `name` будет учитывать тип[89]:

```
data Pet = Pet { name :: String }
instance (r ~ String) => IsLabel "name" (Pet -> r) where
  fromLabel Pet{ name } = name

data Person = Person { name :: String, pets :: [Pet] }
instance (r ~ String) => IsLabel "name" (Person -> r) where
  fromLabel Person{ name } = name

ghci> #name pet
"Rex"
```

[88] Расширение OverloadedLabels.

[89] Чтобы избавиться от ошибки переопределения, нужно включить NoFieldSelectors.

Обратите внимание на вид инстансов. Если написать инстанс сразу для типа `Pet -> String`, то вызов `#name pet` не типизируется, пока тип результата не известен из контекста: вывод инстансов смотрит только на голову, а она не совпадает с `Pet -> r`. Поэтому голова оставлена общей, а равенство `r ~ String` вынесено в ограничение и применяется постфактум. Это и есть constraint trick из 3.1.3.

3.4.4 Несинтаксические типовые эквивалентности, System FC

Современный Haskell является синтаксически богатым языком, который, однако, несмотря на многообразие конструкций, транслируется в маленький типизированный внутренний язык. Это язык *System* F_C Sulzmann et al. [2007b], расширяющий *System F* (2.1) несинтаксическими эквивалентностями типов. Оказывается, этого достаточно, чтобы поддержать такие возможности Haskell как обобщённые алгебраические типы, ассоциированные семейства типов, функциональные зависимости и т.д.

А именно, вводится встроенный констреинт $\sim$, свидетельствующий об эквивалентности двух типов[90]. Например, тип функции `id` может быть записан таким странным образом:

```
f :: forall a b . a ~ b => a -> b
f = id
```

На самом деле это функция от четырёх параметров: двух типовых параметров, коерции и аргумента. Коерция — это значение размера 0, автоматически выводимое компилятором, которое является свидетельством того, что два соответствующих типа эквивалентны.

Например, GADT из 2.1.5 рассахаривается следующим образом:

```
data Expr ty where
  Const :: Int -> Expr Int
  IsZero :: Expr Int -> Expr Bool
  If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty
-- транслируется в
data Expr ty where
  Const :: forall ty . ty ~ Int => Int -> Expr ty
  IsZero :: forall ty . ty ~ Bool => Expr Int -> Expr ty
  If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty
```

И после паттерн-матчинга по конструкторам, констреинт эквивалентности попадёт в ветку и позволит системе вывода типов сделать необходимые переписывания.

Про вывод типов при наличии локальных предположений можно почитать в классической статье *OutsideIn*(X) Vytiniotis et al. [2011].

Очевидно, что Haskell может населить констреинт эквивалентности следуя рефлексивности, симметричности и транзитивности. Также, компилятор может генерировать новые аксиомы (пользователь напрямую свои аксиомы записать не может). Например, по семейству

[90] https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/equality_constraints.html

типов компилятор генерирует аксиомы равенства аппликированного конструктора семейства результирующим типам:

```
type family Plus (n :: Nat) (m :: Nat) :: Nat where
  Plus Zero m = m
  Plus (Suc n) m = Suc (Plus n m)
-- раскроется в
axiom Plus Zero m ~ m
axiom Plus (Suc n) m ~ Suc (Plus n m)
```

Упражнение 37 *Рассахарьте в равенства типов вектор* `Vec` *из 2.1.6. Какие равенства окажутся в области видимости в каждой из веток функции* `vzip`*?*

3.4.5 Коерции и роли

Haskell имеет поддержку **generative type abstractions** в виде `newtype` деклараций. Эта техника позволяет задавать доменно-специфичные типы, которые во время исполнения не отличимы от оборачиваемых типов, но позволяют различать их. Так, мы можем ввести обёртки для чисел, которые в предметной области представляют собой идентификаторы различных сущностей. Теперь система типов не даст их перепутать.

```
newtype ModuleId = ModuleId Int64
newtype CourseId = CourseId Int64
```

Существует крайне недооценённая практика программирования[91], когда у нас в программе есть чёткая граница, на которой происходит парсинг данных из внешнего мира. После неё сырые неструктурированные данные обогащаются структурой и принимают смысл внутри предметной области. Либо же мы отвергаем эти данные как некорректные. В оставшейся же части программы мы уже пользуемся типизированными данными, свойства которых уже установлены и гарантированы. Например, мы можем быть уверены, что число `ModuleId` строго больше нуля.

Однако, если у нас есть коллекция обёрнутых данных, а мы хотим с ней поработать как с коллекцией сырых, то нам придётся трансформировать коллекцию, несмотря на то, что эта трансформация ничего не делает. В общем случае оптимизатор Haskell не справится её элиминировать, потому что работает с типизированным промежуточным представлением и не сможет избавиться от обхода, меняющего тип[92].

```
newtype Csv = Csv { unCsv :: String }

concatC :: [Csv] -> Csv
concatC = Csv . concat . map unCsv
```

[91] `https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/`

[92] Для списков в стандартной библиотеке есть специальное правило переписывания (3.1.6), убирающее такой `map`. Для пользовательской структуры данных обход останется.

Поэтому в Haskell есть механизм безопасных коерций между типами, у которых одинаковое представление во время исполнения. Это реализовано с помощью магического класса типов `Coercible`. Его имплементирует компилятор автоматически (см. рис. 15).

```haskell
class Coercible from to where -- упрощённо
  coerce :: from -> to
```

The most important rules that GHC uses to solve Coercible constraints are as follows (the full rules are given in Figure 5):

(1) The *unwrapping rule*:

- For every **newtype** NT = MkNT t, we have Coercible t NT if and only if the constructor MkNT is in scope.

(2) The *lifting rule*:

- For every type constructor TC r p n, where
  - r stands for TC's parameters at a representational role,
  - p for those at a phantom role and
  - n for those at a nominal role,

  if Coercible r1 r2, then Coercible (TC r1 p1 n) (TC r2 p2 n).

(3) Coercible is an equivalence relation:

- The *reflexivity rule*: Coercible a a.
- The *symmetry rule*: If Coercible a b then Coercible b a.
- The *transitivity rule*: If Coercible a b and Coercible b c then Coercible a c.

Рис. 15: Принципы построения инстансов `Coercible` Breitner et al. [2014].

Теперь, можем избавиться от лишней трансформации списка:

```haskell
concatC :: [Csv] -> Csv
concatC = coerce (concat :: [String] -> String)
```

Безопасность коерций обеспечивает **система ролей**. Каждый типовой параметр имеет специальное свойство — роль.

Роль phantom имеют фантомные типовые параметры. Их можно свободно коерсить (нет пререквизитов для инстансов `Coercible`):

```haskell
data Phantom h = Phantom
data NestedPhantom b = MkNP [Phantom b] | SomethingElse

instance Coercible (Phantom a) (Phantom b)
instance Coercible (NestedPhantom a) (NestedPhantom b)
```

Типовой параметр имеет роль representational, если типовой конструктор можно коэрсить только при условии, что можно коэрсить аргументы:

```
data Maybe a = Nothing | Just a
instance Coercible a b => Coercible (Maybe a) (Maybe b)
```

Роль nominal имеет типовой параметр, если типовой конструктор можно коэрсить только при условии, что аргументы эквивалентны. Это требуется, если типовой аргумент индексирует семейство или констреинт.

```
type family F a
data Applied a = Applied (F a)
instance (a ~ b) => Coercible (Applied a) (Applied b)

data ShowDict a where
  ShowDict :: Show a => a -> ShowDict a
instance (a ~ b) => Coercible (ShowDict a) (ShowDict b)
```

Иногда компилятор выводит неправильную роль типовому параметру. Например, если инварианты структуры зависят на конкретную имплементацию какого-то класса типов для типового аргумента, что совершенно не видно в декларации самого типа. В таком случае, роли можно указать явно:

```
type role Map nominal representational
data Map k v = ...
```

Упражнение 38 *Почему у первого параметра* `Map k v` *роль nominal? Приведите пример обёртки над ключом, для которой приведение* `Map k v` *к* `Map k' v` *сломало бы словарь.*

Подробнее можно прочитать в Breitner et al. [2014] и [Maguire, a, глава 8].

3.4.6 Type reflection

Рефлексия — это языковой механизм получения информации о типах во время исполнения (на уровне термов). Звучит знакомо, и действительно, Haskell реализует этот механизм через классы типов Peyton Jones et al. [2016].

Библиотека предоставляет магический класс типов `Typeable`, который компилятор реализует автоматически для каждого конкретного типа. Чтобы получить информацию о типе, в области видимости должен быть инстанс `Typeable` для этого типа. Структура типа представлена типом-суммы `TypeRep`, который предоставляет возможность дополнительного типового контроля с помощью обобщённых алгебраических типов данных и типовых тегов.

```
class Typeable a where
  typeRep# :: TypeRep a
```

Например, следующим образом можно получить имя конструктора типа:

```
typeName :: forall a. Typeable a => String
typeName = tyConName $ typeRepTyCon $ typeRep $ Proxy @a

ghci> typeName @Int
```

Упражнение 39 *Реализуйте класс* `ShowT (tys :: [Type])` *из 3.1.3, который позволяет распечатать список типов.*

С помощью структуры представления типа и экзистенциальных типов в Haskell можно эмулировать динамическую типизацию. А именно: любой тип может быть преобразован в `Dynamic`, а потом безопасно преобразован обратно.

```
data Dynamic where
  Dynamic :: Typeable a => a -> Dynamic

fromDynamic :: Typeable a => Dynamic -> Maybe a
```

Это может быть полезно, например, для определения гетерогенного хранилища ключ-значение:

```
type Store = Map Key Dynamic
data Ref ty = Ref Key
get :: Typeable ty => Store -> Ref ty -> Maybe ty
```

3.4.7 Data reflection

Как мы обсуждали ранее, в обычной когерентной модели классов типов выбор реализации для данного ограничения однозначен, а полученные разными путями словари неотличимы. И единственный способ объявить инстанс в Haskell — декларацией на верхнем уровне, то есть он не может зависеть ни от каких локальных данных. Однако в Haskell есть библиотека Data.Reflection[93], которая позволяет создавать локальные инстансы для свежих, чёрной магией сгенерированных[94], типов.

Она пользуется идеей “поднятия значений в типы”, обсуждённой нами ранее (см. 3.1.4), но в несколько более общем виде. Вместо заведения классов типов вида `Known_`, вводится один класс типов, индексированный типом термов `terms`, которые спускаются из типов:

```
class Reifies ty terms | ty -> terms where
  reflect :: Proxy ty -> terms
```

[93] https://www.tweag.io/blog/2017-12-21-reflection-tutorial/
[94] https://www.schoolofhaskell.com/user/thoughtpolice/using-reflection

Также, с помощью следующей функции, библиотека позволяет сгенерировать свежий тип и инстанс `Reifies`, который по этому свежему типу возвращает данное значение типа `a` (переданное первым аргументом). Поскольку он передаётся в функцию высшего ранга, свежий тип не может утечь из скоупа 2.1.2:

```
reify :: a -> (forall fresh . Reifies fresh a => Proxy fresh -> res) -> res
```

Чтобы воспользоваться нестандартным инстансом класса типов для некоторого типа `a`, нужно объявить новый тип (например, с помощью `newtype`), содержащий данный, и написать для него нужный инстанс (см., например, Down). Мы не хотим объявлять по новой декларации для каждого случая, поэтому заведём обёртку, похожую на Data.Tagged, которая позволяет добавлять фантомный типовой тег к типу значения. Варьируя тег, можно получить сколь угодно много типов, оборачивающих данный.

```
newtype Wrapped tag a = Wrapped { unwrap :: a }
```

Объявим тип обёртки `Wrapped tag a` представителем нужного класса типов. Код для реализации будем с помощью `reflect` получать по типу тега в виде честного словаря.

```
data ReifiedOrd a = ReifiedOrd { compare :: a -> a -> Ordering }

instance Reifies tag (ReifiedOrd a) => Ord (Wrapped tag a) where
  compare = coerce $ compare $ reflect $ Proxy @tag
```

Наконец, можем вызвать функцию сортировки, подменив локально порядок на обратный:

```
sort :: Ord a => [a] -> [a]

sortReverse :: forall a . Ord a => [a] -> [a]
sortReverse xs =
  let dict = ReifiedOrd { compare = flip compare } in
  reify dict \(Proxy :: Proxy fresh) ->
    coerce $ sort @(Wrapped fresh a) $ coerce xs
```

3.4.8 Открытые структуры

В динамических языках можно создавать объекты на ходу, последовательно дописывая в них содержимое, и не вводя предварительно декларацию. В Haskell тоже так можно, используя пары для произведений и `Either` для сумм. Например, можно добавить новое поле, создав новую пару: `(oldObj, newField)`.

Однако, такая реализация не оптимальна как с точки зрения эффективности (о более эффективных реализациях можно почитать в [Maguire, a, глава 11]), так и с точки зрения удобства использования. А именно — порядок полей имеет значение и на типах в Haskell нет отношения подтипизации (например, нельзя передать значение с меньшим количеством полей или вариантов). Но можно заметить, что констреинты лишены этих недостатков. Поэтому можно организовывать тип структуры данных, например, таким образом:

```
(Int, Double) заменяем на (Member Int d, Member Double d) => Prod d
```

3.4.9 Исключения и открытая иерархия

Важный аспект работы с ошибками заключается в том, что многие из них обрабатываются единообразно. Таким образом, ошибки должны образовывать иерархию наподобие той, которая в ООП языках получается с помощью наследования, чтобы иметь возможность реагировать сразу на группу ошибок одним кодом. Так, возникает задача моделирования подобной иерархии в Haskell.

Более того, статически типизированные ошибки это активная область исследований, будем говорить об этом в рамках систем эффектов (см. далее 8). Классические исключения же динамически типизированные. Особенно хорошо этот вариант подходит для ошибок программиста, которые по-хорошему не должны обрабатываться в программе кроме как закрытием ресурсов[95][96].

Поддержка исключений присутствует в системе исполнения Haskell как простого и привычного способа обработки исключительных ситуаций: ошибок программиста, исполнения неполного паттерн-матчинга, асинхронных системных сигналов Marlow et al. [2001]. . . Исключения динамически типизированные и образуют иерархию Marlow [2006]. Если породить исключение может и чистый код, так как $\bot$, по семантике Haskell, населяет любой тип, то поймать исключение можно только[97] в `IO` Jones [2001], используя специальные примитивы языка.

Чтобы сделать тип исключением, нужно объявить инстанс `Exception` для него:

```
1  class (Typeable a, Show a) => Exception a where
2    toException :: a -> SomeException
3    toException = SomeException

4    fromException :: SomeException -> Maybe a
5    fromException (SomeException e) = cast e
```

Где `SomeException` — это экзистенциальная обёртка наподобие `Dynamic` (см. 3.4.6), в которую заворачивается конкретный тип исключения. Ловя `SomeException`, можно поймать любое исключение (`cast` всегда сработает).

```
1  data SomeException where
2    SomeException :: Exception a => a -> SomeException

3  instance Exception SomeException
```

Система исполнения Haskell предоставляет интринсики для кидания и ловли исключений, обернём их для поддержки любого `Exception` типа:

```
1  throw :: Exception e => e -> a
```

[95] Joe Duffy — The Error Model: ошибки программиста отделяются от ожидаемых ошибок и приводят к аварийному завершению.

[96] `https://wiki.haskell.org/Error_vs._Exception`

[97] (stackoverflow) Why can Haskell exceptions only be caught inside the IO monad?

```
throw e = primThrow (toException e)

catch :: Exception e => IO a -> (e -> IO a) -> IO a
catch io handler = io `primCatch` \e -> case fromException e of
  Nothing -> throw e
  Just e' -> handler e'
```

В простейшем случае свой тип исключения можно реализовать в две строчки. Чтобы его поймать, нужно либо ловить сам этот тип, либо SomeException, потому что для них обоих fromException на объекте вида SomeException MyError вернёт Just[98].

```
data MyError = MyError deriving Show
instance Exception MyError

ghci> throw MyError `catch` \(e :: MyError) -> putStrLn "caught"
caught
```

Добавим исключение DivisionByZero и ещё один более общий тип исключений между ним и SomeException — SomeArithException (таким образом, будет три способа поймать DivisionByZero). Для этого сделаем SomeArithException экзистенциальной обёрткой, а каждое арифметическое исключение будем автоматически оборачивать в неё. В fromException на каждом уровне вложенности будем пытаться получить оборачивающий конструктор рекурсивным вызовом.

```
data SomeArithException where
  SomeArithException :: Exception a => a -> SomeArithException
instance Show SomeArithException where
  show (SomeArithException e) = show e

-- родитель --- SomeException (реализация по умолчанию)
instance Exception SomeArithException

data DivisionByZero = DivisionByZero deriving Show
instance Exception DivisionByZero where
  toException = toException . SomeArithException
  fromException e = do
    SomeArithException e' <- fromException e
    cast e'
```

Так, во время бросания DivisionByZero будет конструироваться объект вида:

```
SomeException (SomeArithException DivisionByZero)
```

[98] `instance Exception SomeException where fromException = Just`

Реализация `fromException` для конкретного типа умеет убедиться в наличии соответствующего конструктора в результирующем объекте исключения.

Упражнение 40 *Добавьте в иерархию исключение* `Overflow`*, тоже арифметическое. Какими обработчиками оно ловится, а какими нет?*

3.4.10 Легковесные частичные стек-трейсы

Забавной эксплуатацией классов типов в Haskell являются легковесные частичные стек-трейсы[99]. Вообще для сбора трейсов нужна поддержка рантайма, что в случае Haskell усложняется ещё и тем, что модель вычислений, редукция графов, реальных трейсов не содержит и их приходится эмулировать. Мы же получим трейсы без поддержки рантайма.

В стандартной библиотеке определён констреинт `GHC.Stack.HasCallStack`, позволяющий получить информацию о месте вызова функции. Эту информацию фактически размещает компилятор в процессе вывода инстансов. Если в месте вызова доступна информация с уровня выше, компилятор распространяет её дальше. Таким образом, доступна информация только на определённую глубину стека вызовов.

```
myHead :: HasCallStack => [a] -> a
myHead []     = error "empty"
myHead (x:xs) = x

bad :: Int
bad = myHead []

ghci> bad
*** Exception: empty
CallStack (from HasCallStack):
  error, called at Bad.hs:8:15 in main:Bad
  myHead, called at Bad.hs:12:7 in main:Bad
  -- no information about bad call site here
```

`HasCallStack` — это просто имплисит (см. 3.1.2), про который знает компилятор:

```
type HasCallStack = (?callStack :: CallStack)
```

3.4.11 Кастомизируемые ошибки типизации

При программировании сложных с точки зрения типов библиотек, желательно предоставлять пользователям более информативные ошибки типизации, чем ошибки по умолчанию. Для этого в GHC есть механизм в GHC.TypeLits, позволяющий сконструировать специальный тип, информация из которого попадёт в сообщение об ошибке. Например, этот тип можно

[99] https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/callstack.html

вернуть из synonym family при некорректном наборе аргументов. Или же можно воспользоваться constraint trick (см. 3.1.3) и разместить такой тип в качестве посылки в инстансе. Если инстанс подошел и компилятор начал обрабатывать ограничения слева, значит, что-то пошло не так [Maguire, a, глава 12].

```
instance (TypeError
  ( Text "Attempting to show a function of type "
    :<>: Text "'" :<>: ShowType (a -> b) :<>: Text "'"
    :$$: Text "Did you forget to apply an argument?"
  )) => Show (a -> b) where
  show = undefined -- реализация не важна, до исполнения дело не дойдёт
```

3.4.12 Вычисление до нормальной формы

Haskell ленив: значение вычисляется только тогда, когда оно понадобилось, и только настолько, насколько понадобилось. Встроенная функция seq вычисляет свой первый аргумент до **слабой головной нормальной формы (WHNF)**, то есть до первого конструктора, и не трогает его поля. Иногда этого мало и значение нужно вычислить целиком, до **нормальной формы**:

- чтобы измерить время самого вычисления, а не создания отложенного вычисления;
- чтобы работа была сделана в нужном потоке, а не в том, который первым обратится к результату;
- чтобы исключение возникло там, где стоит его обработчик;
- чтобы не держать в памяти цепочку отложенных вычислений.

Параметрически-полиморфная функция обойти значение целиком не может: способ обхода зависит от типа. Значит, это работа для класса типов[100] [Marlow, 2011, глава 2]:

```
class NFData a where
  rnf :: a -> () -- reduce to normal form

instance NFData Int where
  rnf x = x `seq` ()

instance NFData a => NFData [a] where
  rnf [] = ()
  rnf (x : xs) = rnf x `seq` rnf xs

deepseq :: NFData a => a -> b -> b
deepseq x y = rnf x `seq` y
```

[100] https://hackage.haskell.org/package/deepseq

```haskell
force :: NFData a => a -> a
force x = x `deepseq` x
```

Разницу между `seq` и `deepseq` легко увидеть на списке, внутри которого спрятано невычислимое значение:

```
ghci> xs = [1, undefined, 3] :: [Int]
ghci> xs `seq` "ok"
"ok"
ghci> length xs
3
ghci> xs `deepseq` "ok"
*** Exception: Prelude.undefined
```

Вернёмся к исключениям. Чистый код порождает исключение не там, где он написан, а там, где его результат вычисляется. Поэтому обработчик ловит исключение, только если вычисление произошло внутри него. Функция `evaluate` вычисляет значение лишь до первого конструктора, и исключение, спрятанное глубже, уйдёт мимо обработчика:

```haskell
safe, safe' :: [Int] -> IO (Either SomeException [Int])
safe  xs = try (evaluate xs)          -- не поймает
safe' xs = try (evaluate (force xs)) -- поймает
```

Цена: `rnf` обходит значение целиком, даже если оно уже вычислено, и каждый следующий вызов обходит его заново. Для функций же инстанс вычисляет только до WHNF: заглянуть в замыкание нельзя. Строгие поля помогают контролировать отложенные вычисления при построении структуры[101]. Однако строгость поля требует лишь WHNF, а вложенные значения могут оставаться ленивыми. Например, в `data Box = Box ![Int]` поле списка строгое, но значение `Box [1, undefined]` можно вычислить до WHNF: достаточно первого конструктора списка, а его элементы ещё не понадобились. Чтобы гарантировать полную нормализацию структуры, нужно контролировать строгость всех вложенных данных или явно выполнять `rnf`.

Можно заметить, что инстансы `NFData` полностью определяются формой типа: нужно обойти все поля всех конструкторов. Писать такой код руками не приходится, его порождает компилятор (см. 9).

Упражнение 41 *Реализуйте инстанс* `NFData` *для дерева:*

```haskell
data Tree a = Leaf | Node (Tree a) a (Tree a)
```

Упражнение 42 *Почему* `length xs `seq` ()` *не заменяет* `rnf xs`*? Что именно вычислит каждое из выражений?*

[101] Расширение StrictData делает строгими по умолчанию поля типов данных, объявленных в данном модуле.

4 Типы данных

Рекурсивные структуры данных можно описывать через способы их конструирования, потребления или порождения. Например, список можно представить цепочкой конструкторов, процедурой свёртки с заданными обработчиками или состоянием и функцией, выдающей очередной элемент. Выбор представления влияет на интерфейс: кто управляет обходом, какие операции удобно выразить и как организовать вычисление без промежуточных структур.

Чтобы разобраться в этих представлениях, начнём с вариантности и алгебры типов: научимся переносить функции через типовые конструкторы и находить изоморфизмы между типами. Затем отделим форму рекурсивной структуры от самой рекурсии и получим универсальные свёртки и развёртки. Они позволят связать привычные типы данных с кодировками Чёрча, паттернами Visitor и Iterator, а также с потоками, в которых данные могут порождаться по запросу.

На примере оптимизаций fusion мы увидим, как смена представления помогает устранить промежуточные структуры, сохранив композиционный стиль программирования. Сопоставляя эти подходы, будем обращать внимание на способы наблюдения данных, возможность работать с бесконечными структурами и роль ленивости. Эти связи пригодятся при проектировании API и при изучении интерпретаторов в следующей главе.

Разделы 4.1, 4.2 в основном следуют [Maguire, a, глава 1].

4.1 Вариантность

Рассмотрим вариантность с точки зрения программирования [Maguire, a, глава 3]. Категорный взгляд на эту тему можно найти в статье, написанной в жанре пьесы Hinze et al. [2012].

4.1.1 Функторы

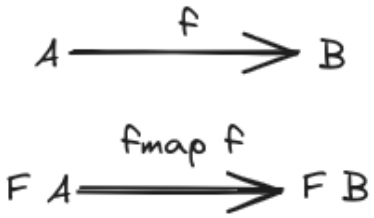
Пусть у нас есть функция `h :: a -> b` и типовой конструктор `F`. Можно ли с помощью `h` преобразовать `F a` в `F b`? Для списка ответ знаком: достаточно применить `map h`. Но для других типовых конструкторов направление преобразования может оказаться обратным. **Вариантность** описывает, как преобразование параметра связано с преобразованием всего типа.

Ковариантный функтор — пара из некоторого типового конструктора `F` и операции на функциях `fmap :: (a -> b) -> (F a -> F b)`. Эта операция сохраняет направление стрелки и должна уважать тождественную функцию и композицию:

```
1 class Functor f where
2   fmap :: (a -> b) -> (f a -> f b)

3 -- Законы:
4 fmap id == id
```

```haskell
fmap (g . h) == fmap g . fmap h
```



Рассмотрим теперь предикат: `type Predicate a = a -> Bool`. Имея предикат на `b` и функцию `a -> b`, мы можем проверять значения типа `a`: сначала преобразуем аргумент, затем применим предикат. Так, из проверки числа получается проверка длины строки:

```haskell
type Predicate a = a -> Bool

contramapP :: (a -> b) -> Predicate b -> Predicate a
contramapP h predicate = predicate . h

longEnough :: Predicate Int
longEnough n = n >= 8

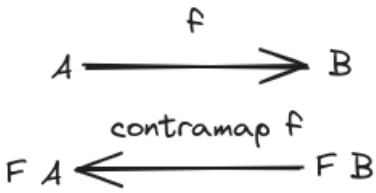
longText :: Predicate String
longText = contramapP length longEnough
-- longText "hello" == False
```

Функция `length :: String -> Int` превращает предикат на числах в предикат на строках: направление преобразования предикатов противоположно направлению `length`.

Контравариантный функтор — пара из типового конструктора и операции на функциях, разворачивающей стрелку. При переносе композиции порядок операций также меняется:

```haskell
class Contravariant f where
  contramap :: (a -> b) -> (f b -> f a)

-- Законы:
contramap id == id
contramap (g . h) == contramap h . contramap g
```



Типовой конструктор можно объявить ковариантным или контравариантным функтором (или никаким из них) относительно некоторого типового параметра в зависимости от вида декларации соответствующих конструкторов данных. А именно, от знака позиций, в которых входит этот типовой параметр в тип.

4.1.2 Знаки позиций в типе

Разовьём интуитивное понимание знаков позиций. Тип `A` входит в положительной позиции в `B` если его значение можно извлечь из `B`. И наоборот, тип `A` входит в отрицательной позиции, если его значение нужно, наоборот, предоставить. Рассмотрим знаки позиций типов в базовых типовых конструкторах:

Тип	знак позиции `A`	знак позиции `B`
`Either A B`	$+$	$+$
`(A, B)`	$+$	$+$
`A -> B`	$-$	$+$

Действительно, из суммы и произведения можно извлечь компоненты с помощью паттерн-матчинга, а из стрелки можно получить правый тип аппликируя её к аргументу. В то же время значение типа слева от стрелки нужно предоставить.

На плюс и минус действуют интуитивные алгебраические законы при рассмотрении более сложных типов. Рассмотрим на примере `f :: ((A, B) -> C) -> (D, E)`.

- Плюс на плюс даёт плюс. Действительно, нужно лишь применить две элиминации вместо одной, чтобы получить заветный тип. В нашем примере, чтобы получить `D`, нужно сначала аппликировать функцию, а потом разобрать пару.
- Плюс на минус (и наоборот) даёт минус. Действительно, `C` нам нужно предоставить: `f (\ab -> provideC)`.
- Минус на минус даёт плюс. Пару `(A, B)` нам предоставляют: `f (\ab -> ...)`.

Упражнение 43 *Убедитесь что плюс на минус даёт минус.*

Возвращаясь к функторам, если типовой параметр входит в декларацию только в положительных позициях, типовой конструктор можно естественным образом объявить ковариантным функтором относительно этого параметра. Если в только в отрицательных — контравариантным функтором[102]. Такой параметр называют **ковариантным** и **контравариантным** соответственно. Иначе говоря, ковариантный функтор позволяет постобработать значение вычисления, а контравариантный — предобработать аргумент вычисления.

Если параметр встречается в позициях обоих знаков, его называют **инвариантным**: в общем случае одного преобразования `a -> b` уже недостаточно. Например, чтобы превратить функцию `a -> a` в функцию `b -> b`, нужны преобразования в обе стороны:

[102] Если параметр вообще не встречается в полях, он фантомный. В этом случае можно определить оба отображения, игнорируя переданную функцию.

```haskell
adapt :: (a -> b) -> (b -> a) -> (a -> a) -> (b -> b)
adapt forward backward f = forward . f . backward
```

Преобразование `backward` обслуживает отрицательное вхождение параметра, а `forward` — положительное.

Упражнение 44 *Объявите* `instance Contravariant F` *для типа:*

```haskell
data F a = L (a -> Bool) | R Int
```

4.1.3 Ещё функторы

Одновременное ковариантное отображение по двум параметрам описывает класс **бифункторов**:

```haskell
class Bifunctor f where
  bimap :: (a -> c) -> (b -> d) -> f a b -> f c d
```

Например, у `Either` оба параметра положительны: `bimap` применяет первую функцию к содержимому `Left`, а вторую — к содержимому `Right`.

Отображение, контравариантное по первому параметру и ковариантное по второму, описывает класс **профункторов**:

```haskell
class Profunctor p where
  dimap :: (c -> a) -> (b -> d) -> p a b -> p c d
```

Функциональная стрелка — основной пример профунктора: сначала пред-обрабатываем вход, затем вызываем функцию и пост-обрабатываем выход.

```haskell
instance Profunctor (->) where
  dimap pre post f = post . f . pre
```

По той же схеме можно устроить тип SQL-запроса `Sql input output`: `dimap` добавляет преобразования входных данных и результата. Пусть заданы преобразования и исходный запрос:

```haskell
serialize   :: Age -> Text
deserialize :: Text -> [User]
query       :: Sql Text Text
```

Тогда получаем запрос с новым интерфейсом:

```haskell
dimap serialize deserialize query :: Sql Age [User]
```

4.1.4 ООП и подтипизация

Также понятие вариантности встречается в объектно-ориентированных языках и вообще в теории подтипизации: здесь речь идёт о переносе отношения подтипизации на параметризованные типы.

Действительно, **отношение подтипизации** `B <: A` говорит о том, что значение типа `B` безопасно использовать в позиции, где ожидается значение типа `A`. Такое использование можно представить как специальное преобразование `upcast :: B -> A`. При ковариантности из `B <: A` (`upcast :: B -> A`.) следует `F B <: F A` (наличие `upcast' :: F B -> F A`.), при контравариантности — `F A <: F B`.

Упражнение 45 *Убедитесь в вашем любимом языке с подтипизацией и поддержкой вариантности, что минус на минус даёт плюс.*

4.2 Изоморфизм

э

Один и тот же смысл можно представить разными типами. При проектировании функции или модели данных полезно понимать, какие типы несут одинаковое "количество информации". Для этого введём понятие изоморфизма, а затем воспользуемся алгеброй типов, чтобы навигироваться в пространстве решений.

В этом разделе рассматриваем полностью определённые значения и тотальные функции, то есть функции, определённые на всех входах[103].

Два типа `A` и `B` называются **изоморфными** (обозначают $A \cong B$) тогда и только тогда, когда существует такая пара функций `to :: A -> B` и `from :: B -> A`, что[104]

```
1 to . from = id
2 from . to = id
```

Иначе говоря, между обитателями таких типов можно установить взаимно-однозначное соответствие. Функции перехода позволяют сменить представление данных, а затем восстановить исходное значение без потерь. В этом смысле два изоморфных типа заключают в себе одинаковое "количество информации". Например, типы `Bool` и `Maybe ()` в этом смысле совершенно взаимозаменимы. Покажем это, предъявив пару взаимообратных функций:

[103] Это упрощённая модель, а не полная семантика Haskell: мы не учитываем незавершающиеся вычисления и частично определённые значения. Например, у типа `Maybe ()` есть значение `Just undefined`. Для приведённых ниже функций преобразование туда и обратно теряет даже его внешний конструктор: `to (from (Just undefined))` даёт неопределённое вычисление целиком. Поэтому законы изоморфизма здесь утверждаются в выбранной модели.

[104] Под равенством термов можно понимать разное, например, $\alpha\beta\eta$-эквивалентность. Мы будем пользоваться **экстенсиональным равенством** для функций — две функции равны, когда равны их результаты на всех входах. `https://ncatlab.org/nlab/show/function+extensionality`

```haskell
to :: Bool -> Maybe ()
to b = if b then Just () else Nothing

from :: Maybe () -> Bool
from m = case m of Nothing -> False; Just () -> True
```

Здесь `False` соответствует `Nothing`, а `True` — `Just ()`. Перебрав эти случаи, проверяем, что обе композиции возвращают исходное значение.

Несмотря на смысловую взаимозаменимость, для кодирования информации о том, передал ли пользователь программе определённый флаг, мы, скорее всего, воспользуемся типом `Bool` ввиду нефункциональных соображений о читабельности кода. Аналогично можно рассматривать соображения эффективности.

С категорным взглядом на происходящее можно ознакомиться в Hinze and James [2010]. Мы же придерживаемся теоретико-множественной интерпретации типов.

4.2.1 Кардинальность: суммы, произведения, экспоненты

Типы можно воспринимать как синтаксис для записи множеств, а населяющие их термы — как синтаксические записи элементов этих множеств. Так терм `(True, False)` — запись элемента множества пар, записываемого в синтаксисе типов как `(Bool, Bool)` (вместо математического $\mathbb{B} \times \mathbb{B}$). Или же терм `\x -> x + 1` является записью функции прибавляющей единицу из множества функций над целыми числами, записываемого как `Integer -> Integer` (вместо математического $\mathbb{Z} \to \mathbb{Z}$).

В теоретико-множественной модели изоморфизм — это биекция, поэтому он существует тогда и только тогда, когда множества равномощны. Между двумя конечными множествами из n элементов существует $n!$ биекций[105]. Научимся определять количество таких элементов. С помощью $|\cdot|$ будем записывать **кардинальность** типа — количество элементов в соответствующем множестве.

Тип и его декларация	кардинальность
`data Void`	0
`data Unit = Unit`[106]	1
`data Bool = False | True`	2

Идея алгебраических типов данных в том, что сложные типы можно строить из простых с помощью операции $+$ (“или”) и операции $\times$ (“и”)[107]:

Тип	кардинальность
`data Either a b = Left a | Right b`	$\lvert a\rvert + \lvert b\rvert$
`data Pair a b = Pair a b`	$\lvert a\rvert \times \lvert b\rvert$

[105] Для конечных типов, значения которых можно перечислить, функции перехода можно задать таблицей соответствий. Для бесконечных типов равномощность сама по себе ещё не даёт алгоритма преобразования: математическая биекция не обязательно реализуема программой (вычислима).

[106] `Unit` записывается в Haskell с помощью специального синтаксиса `()`, означающем как бы пустой кортеж.

[107] `https://stanford-cs242.github.io/f18/lectures/02-2-algebraic-data-types.html`

Посчитаем количество значений различных типов (вы можете убедиться в справедливости заключения, перечислив все значения вручную):

- `|Either Unit (Either Bool Bool)| = |Unit| + (|Bool| + |Bool|) = 5`.
- `|Pair (Either Bool Unit) (Pair Unit Void)| = 0` — тип `Void` не населён, как и кортеж, его включающий.
- Если `data Example = FirstAlternative Bool | AnotherOne Unit Bool Bool`, то `|Example| = |Bool| + |Unit| * |Bool| * |Bool| = 2 + 1 * 2 * 2 = 6`.

Рассмотрим теперь функции. У типа `Bool -> Bool` есть четыре различных тотальных функции:

Функция	результат на `False`	результат на `True`
`const False`	`False`	`False`
`id`	`False`	`True`
`not`	`True`	`False`
`const True`	`True`	`True`

Для каждого из двух аргументов независимо выбирается один из двух результатов, поэтому получаем $2^2 = 4$ функции. Мы считаем функции по их поведению, а не записи программ: например, `id` и `\x -> x` здесь один и тот же элемент множества. Аналогично для каждого элемента конечного множества A есть $|B|$ вариантов результата. Поэтому функциональную стрелку называют экспоненциальным типом:

$$|A \to B| = |B|^{|A|}.$$

В теоретико-множественной интерпретации эта формула описывает множество всех тотальных функций; для бесконечных множеств среди них могут быть невычислимые функции.

Вернёмся к проектированию типов. Пусть пользователь может задать два логических флага, причём они должны появляться только вместе. Сравним два представления:

```
type Together = Maybe (Bool, Bool)
type Independently = (Maybe Bool, Maybe Bool)
```

У первого типа $1 + 2 \cdot 2 = 5$ значений: оба флага отсутствуют либо оба заданы одной из четырёх пар. У второго типа $(1+2)\cdot(1+2) = 9$ значений: каждый флаг может отсутствовать независимо от другого. Четыре дополнительных состояния имеют вид `(Just b, Nothing)` или `(Nothing, Just b)`, где `b` — `False` или `True`. Эти типы неизоморфны; для сформулированного требования подходит первый, поскольку в нём нельзя представить состояние с одним флагом. Если же независимое отсутствие флагов допустимо, нужен второй.

Подсчёт значений помогает проверить, соответствует ли тип предметной области. При проектировании полезно руководствоваться следующими соображениями:

- В типе должно быть не меньше элементов, чем в предметной области, чтобы все необходимые объекты были представимы.

- В типе должно быть как можно меньше элементов, которых нет в предметной области, чтобы пространство ошибок было минимальным.
- Далее среди изоморфных типов выбирается оптимальный исходя из нефункциональных требований.

Прежде чем работать с некоторым объектом предметной области, информацию о нём, в соответствии со вторым правилом, следует привести в максимально структурное представление, дающее наибольшее количество гарантий[108].

4.2.2 Алгебраическое представление типа

Как мы увидели выше, чтобы показать наличие изоморфизма между двумя типами можно либо предъявить пару взаимообратных функций, либо показать, что кардинальности этих двух типов совпадают. В этом разделе мы научимся сопоставлять типу некоторую алгебраическую запись, отражающую его структуру и кардинальность. Так, мы сможем синтаксическими преобразованиями формул получать эквивалентные записи, из которых будем восстанавливать типы, заведомо изоморфные данному[109].

В основу алгебраического представления положим вычисление кардинальности типов. Имена конструкторов при этом несущественны: мы рассматриваем типы с точностью до изоморфизма, сохраняя информацию об их структуре.

Тип	алгебраическая формула
`data Void`	0
`data Unit = Unit`	1
`data Bool = False | True`	$1+1$ (обозначим как 2)
`data Maybe a = Nothing | Just a`	$1+a$
`data Either a b = Left a | Right b`	$a+b$
`data Pair a b = Pair a b`	$a \times b$
`a -> b`	b^a

Упражнение 46 *Запишите в алгебраическом виде следующий тип:*

```
1 data T a b = Undefined | Defined a (a -> b)
```

В качестве отношения эквивалентности будем использовать изоморфизм соответствующих типов. Для сумм, произведений и экспонент получаются знакомые алгебраические законы (рис. 16). Разберём дистрибутивность:

$$a \times (b + c) \cong (a \times b) + (a \times c).$$

Слева у нас пара, второй компонент которой выбирается из двух вариантов. Справа выбор варианта сделан снаружи, а общий компонент хранится внутри выбранной пары. Функции перехода повторяют эту структуру:

[108] https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/
[109] https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types

```haskell
distribute :: (a, Either b c) -> Either (a, b) (a, c)
distribute (x, Left y)  = Left (x, y)
distribute (x, Right z) = Right (x, z)

factor :: Either (a, b) (a, c) -> (a, Either b c)
factor (Left (x, y))  = (x, Left y)
factor (Right (x, z)) = (x, Right z)
```

Проверим обе композиции. Для левых вариантов получаем:

```haskell
factor (distribute (x, Left y))
  = factor (Left (x, y))
  = (x, Left y)

distribute (factor (Left (x, y)))
  = distribute (x, Left y)
  = Left (x, y)
```

Для правых вариантов рассуждение аналогично. Мы восстановили исходные значения в обоих направлениях, поэтому предъявили изоморфизм, а не только равенство кардинальностей.

Закону $(c^b)^a \cong c^{a \times b}$ соответствуют знакомые операции каррирования и раскаррирования:

```haskell
-- (c^b)^a ≅ c^(a×b)
to :: (a -> b -> c) -> (a, b) -> c
to = uncurry
from :: ((a, b) -> c) -> a -> b -> c
from = curry
```

Упражнение 47 *Покажите, что* $(a + b) + c \cong a + (b + c)$.

Упражнение 48 *Покажите, что* $c^{a+b} \cong c^a \times c^b$.

Интересным наблюдением может быть то, что функции можно использовать как структуры данных, в соответствие с изоморфизмом $c^{a+b} \cong c^a \times c^b$. Действительно, в таком случае аргумент функции выступает индексом (его кардинальность должна совпадать с размером коллекции).

```haskell
-- a × a ≅ a^2
get :: (a, a) -> (Bool -> a)
get (x, y) idx = if idx then x else y
tabulate :: (Bool -> a) -> (a, a)
tabulate f = (f True, f False)
```

sums

$$X + 0 \cong X \quad X + Y \cong Y + X$$
$$(X + Y) + Z \cong X + (Y + Z)$$

products

$$X \times 1 \cong X \quad 0 \times X \cong 0 \quad X \times Y \cong Y \times X$$
$$(X \times Y) \times Z \cong X \times (Y \times Z)$$

product over sum

$$(X + Y) \times Z \cong (X \times Z) + (Y \times Z)$$

exponentials

$$1^X \cong 1 \quad X^0 \cong 1 \quad X^1 \cong X$$
$$Z^{X+Y} \cong Z^X \times Z^Y \quad (Z^Y)^X \cong Z^{X \times Y}$$

exponential over product

$$(Y \times Z)^X \cong Y^X \times Z^X$$

Рис. 16: Законы школьной алгебры ностальгии ради Hinze and James [2010].

Каноническим представлением алгебраического типа (canonical representation) называют сумму произведений типов:

$$\sum_i \prod_j t_{ij}$$

Сумма соответствует выбору конструктора данных, а каждое произведение — набору его полей. Именно так устроены обычные `data` декларации в Haskell[110]. У рекурсивного типа такое описание задаёт один слой структуры, в полях которого могут находиться подструктуры того же типа. В следующем разделе мы отделим эту форму слоя от рекурсии, не пытаясь развернуть всю структуру в конечный полином.

4.3 Рекурсивные типы

Рекурсивные типы, подобно рекурсивным термам, могут упоминать себя в своих определениях. Например, тип натурального числа встречается в типе поля конструктора следующего числа:

```
1 fac n = if n <= 1 then 1 else n * fac (n - 1)
2 data Nat = Zero | Suc Nat
```

Мы в основном посмотрим на рекурсивные типы с практической точки зрения. Наша цель — отделить форму одного слоя структуры от рекурсивных ссылок на её подструктуры. Тогда

[110] Алгебраическая аналогия распространяется и на производные: для полиномиальных типов производная описывает контексты с одним пропущенным элементом. Такие контексты используются при построении зиперов — структур для навигации и локального изменения данных Huet [1997], McBride [2001], Abbott et al. [2003].

обработку слоя можно будет задавать отдельно, а саму схему рекурсии переиспользовать для списков, деревьев и других типов. Однако, их формальное теоретико-типовое описание, теоретико-категорная и теоретико-множественная интерпретации представляют отдельный интерес [Pierce, 2002, часть 4].

4.3.1 Просто список

Рассмотрим конечные списки. Такой список содержит либо ноль элементов, либо один, либо два... Алгебраически это запишется следующим образом:

$$L = 1 + a + a^2 + a^3 + \dots$$

Каждое слагаемое описывает списки одной длины: 1 — пустой список, a — одноэлементные, a^2 — двухэлементные и так далее. Все списки конечны, но для перечисления всех возможных длин потребовалась бесконечная запись. Поработаем с ним как с формальным рядом. Вынесем a за скобки:

$$L = 1 + a \times (1 + a + a^2 + \dots)$$

Выражение в скобках снова описывает список, поэтому получаем рекурсивное уравнение:

$$L = 1 + a \times L$$

Легко видеть, что это на самом деле знакомое нам определение списка из Haskell:

```
1 data List a = Nil | Cons a (List a)
```

Мы получили конечную запись определения, но рекурсия в ней осталась. Теперь выделим нерекурсивную форму одного слоя, заменив в ней хвост на параметр. Сам тип списка должен быть изоморфен этой форме, в которую снова подставлен список[111]:

$$L \cong 1 + a \times L$$

Расширим язык типов абстракцией (полиморфизмом) и решим полученное рекурсивное уравнение в стиле λ-исчисления, с помощью некоторого комбинатора неподвижной точки:

$$L = FIX\ \lambda r.\, 1 + a \times r$$

Закодируем это на Haskell. Сначала зафиксируем форму списка, а рекурсию замкнём в отдельном типе `FixList a`:

[111] Само уравнение ещё не выбирает конечные списки. В модели множеств конечные списки образуют наименьшую неподвижную точку соответствующего конструктора типов. Ленивый Haskell допускает также бесконечные и частично определённые значения; здесь вывод по длинам относится к конечным, полностью определённым спискам.

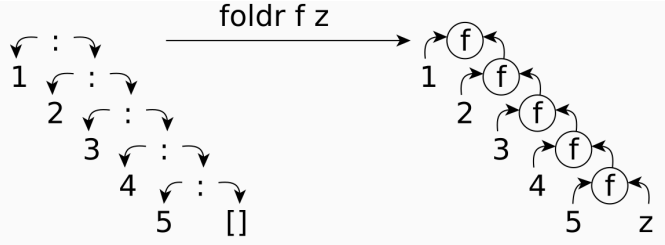
```haskell
type Shape a r = Either () (a, r) -- λar. 1 + a × r
data FixList a = In (Shape a (FixList a))
-- FixList a ≅ Shape a (FixList a) ≅ Shape a (Shape a (FixList a)) ≅ ...

example :: FixList Int -- [1, 2, 3]
example = In (Right (1,
            In (Right (2,
            In (Right (3, In (Left ()))))))))
```

У параметров `Shape a r` разные роли: `a` — тип элемента, а `r` — место для хвоста. Сам `Shape` не рекурсивен; в `FixList` на место `r` подставлен снова `FixList a`. Таким образом, мы разделили определение списка на две части: одна отвечает за форму слоя, другая — за рекурсию[112]. Форму можно переиспользовать в определении свёртки: теперь на место хвоста подставляется результат его обработки.

```haskell
foldFixList :: (Either () (a, r) -> r) -> FixList a -> r
foldFixList phi (In shape) = case shape of
  Left () -> phi (Left ())
  Right (x, xs) -> phi (Right (x, foldFixList phi xs))
-- сравните с классическим определением
foldr :: (a -> r -> r) -> r -> [a] -> r
foldr f ini list = case list of
  [] -> ini
  x:xs -> f x (foldr f ini xs)
```

Действительно, свёртка в общем смысле позволяет заменить каждый конструктор данных в дереве на некоторую функцию. В результате получается вычисление, имеющее доступ ко всему содержимому структуры данных, и возвращающее результат агрегации:



То есть количество и типы аргументов свёртки зависят от “формы” сворачиваемого типа. В случае списка требуется значение вместо `Nil` и бинарная функция вместо `Cons`. Мы же объединяем оба случая в одну функцию с помощью типа-формы. Сравните:

```haskell
foldr       (\e r -> e + r) 0
foldFixList (\case Left () -> 0; Right (e, r) -> e + r)
```

[112] Техника абстрагирования по “рекурсивной ссылке на себя” называется **открытой рекурсией**. Так, мы предоставляем пользователю больше контроля. Например, на уровне термов комбинатор рекурсии может подсчитывать количество рекурсивных вызовов и оборвать вычисление при превышении какого-то лимита.

4.3.2 Неподвижная точка функтора

Абстрагируем `FixList` по типу-форме:

```
import Data.Kind (Type)

type Fix :: (Type -> Type) -> Type
newtype Fix f = In { out :: f (Fix f) }

data ListF a r = Nil | Cons a r
type List a = Fix (ListF a)
```

Упражнение 49 *Какие типы будут у* `In` *и* `out`*?*

Можно показать, что `[a]` ≅ `List a`:

```
to :: [a] -> List a
to = \case
  [] -> In Nil
  x:xs -> In $ Cons x (to xs)

from :: List a -> [a]
from (In shape) = case shape of
  Nil -> []
  Cons x xs -> x : from xs
```

Сравним два применения одной и той же формы:

`ListF a (List a)`	один слой с хвостом-списком;
`ListF a Int`	один слой с числом вместо хвоста.

Например, числом может быть результат свёртки хвоста. Чтобы получить такой слой, сделаем тип формы функтором по последнему параметру. Его `fmap` преобразует только рекурсивную позицию `r`, сохраняя элемент типа `a`:

```
instance Functor (ListF a) where
  fmap :: (rec -> other) -> ListF a rec -> ListF a other
  fmap f = \case
    Nil -> Nil
    Cons x xs -> Cons x (f xs)
```

Упражнение 50 *Выразите следующее дерево как неподвижную точку функтора. Объявите инстанс функтора для типа-формы.*

```
data Tree a = Leaf a | Node a (Tree a) (Tree a)
```

4.3.3 Схемы рекурсии

Подобно тому, как структурное императивное программирование, в сравнении с беспорядочным использованием `goto`, помогает рассуждать о программах, так **схемы рекурсии** позволяют алгебраически описывать свойства рекурсивных функций, в отличие от "неструктурной" рекурсии[113] Meijer et al. [1991], Meijer and Hutton [1995].

Практическая идея проста: схема фиксирует способ рекурсивного обхода, а пользователь задаёт обработку одного слоя. Это позволяет переиспользовать обход и рассуждать об общих законах преобразования программ Maguire [b].

Универсальная свёртка называется **катаморфизмом**. Пусть `f` — функтор формы, `r` — тип результата, а `phi :: f r -> r` обрабатывает один слой с результатами вместо поддеревьев. Выведем устройство катаморфизма по типам:

$$\texttt{Fix f} \xrightarrow{\texttt{out}} \texttt{f (Fix f)} \xrightarrow{\texttt{fmap (cata phi)}} \texttt{f r} \xrightarrow{\texttt{phi}} \texttt{r}$$

Функция `out` открывает один слой. Затем `fmap (cata phi)` помещает результаты рекурсивной свёртки на места поддеревьев, а `phi` получает результат для всего слоя. Композиция этих трёх операций и есть искомая свёртка:

```
cata :: Functor f => (f r -> r) -> Fix f -> r
cata phi = phi . fmap (cata phi) . out
```

Иначе говоря, `cata` заменяет все конструкторы `In` на функцию `phi`, которая содержит информацию о том, как реагировать на разные варианты конструкторов в дереве:

```
example           = In  (Cons 1 (In  (Cons 2 (In  (Cons 3  (In  Nil))))))
cata phi example ⇝ phi (Cons 1 (phi (Cons 2 (phi (Cons 3 (phi Nil))))))
```

Например, сумма списка будет выглядеть следующим образом (рис. 17):

```
sum :: List Int -> Int
sum = cata sumAlg

sumAlg :: ListF Int Int -> Int
sumAlg = \case
  Nil -> 0
  Cons x result -> x + result
```

Для списка `[1, 2, 3]` хвост `[2, 3]` сворачивается в `5`. Поэтому алгебра внешнего слоя получает `Cons 1 5 :: ListF Int Int`: элемент `1` сохранился, а на месте хвоста находится его сумма. Последний шаг — `sumAlg (Cons 1 5)`, то есть `1 + 5 = 6`.

Функцию `f a -> a` называют **f-алгеброй**. Действительно, если в качестве функтора `f` взять сигнатуру алгебры, а в качестве `a` носитель, то f-алгебра будет задавать некоторую интерпретацию сигнатуры:

[113] https://reasonablypolymorphic.com/blog/recursion-schemes/index.html

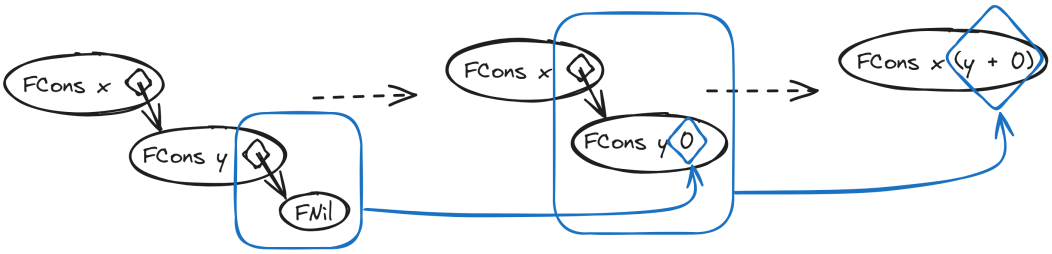


Рис. 17: В новом слое результаты свёртки занимают позиции рекурсивных подструктур. Исходная структура при этом не изменяется.

```haskell
data MonoidSig carrier = Mempty | Mappend carrier carrier

interpretSig :: MonoidSig Int -> Int
interpretSig = \case Mempty -> 0; Mappend l r -> l + r
```

Упражнение 51 *С помощью какой алгебры можно скопировать структуру данных?*

Упражнение 52 *С помощью какой алгебры можно распечатать список в строчку?*

В противоположность универсальной свёртке, можно построить **анаморфизм** — универсальную развёртку (аналогично `unfold` для списка). Здесь **f-коалгебра** `psi :: s -> f s` показывает, как из значения-зерна получить один слой структуры данных с новыми зёрнами в рекурсивных позициях. Снова проследим типы:

$$\texttt{s} \xrightarrow{\texttt{psi}} \texttt{f s} \xrightarrow{\texttt{fmap (ana psi)}} \texttt{f (Fix f)} \xrightarrow{\texttt{In}} \texttt{Fix f}$$

Коалгебра создаёт слой, рекурсивные вызовы разворачивают новые зёрна в поддеревья, а `In` упаковывает слой в рекурсивный тип:

```haskell
ana :: Functor f => (s -> f s) -> s -> Fix f
ana psi = In . fmap (ana psi) . psi
-- сравните с классическим определением развёртки списка
unfoldr :: (s -> Maybe (a, s)) -> s -> [a]
```

Например, зададим обратный отсчёт от положительного числа:

```haskell
countDown :: Integer -> ListF Integer Integer
countDown n = if n > 0 then Cons n (n - 1) else Nil
```

В результате `countDown 3` получаем `Cons 3 2`: здесь 3 — элемент, а 2 — зерно для хвоста. Коалгебра описала только один слой; рекурсию обеспечивает `ana`, применяя её к новым зёрнам:

```
countDown 3 ⇝ Cons 3 2
countDown 2 ⇝ Cons 2 1
countDown 1 ⇝ Cons 1 0
countDown 0 ⇝ Nil
```

Таким образом, `ana countDown 3 :: List Integer` представляет список `[3, 2, 1]`.

Упражнение 53 *Реализуйте анаморфизм, строящий список от* 0 *до заданного* n.

Также вводят **гиломорфизм (hylomorphism)**, который объединяет построение структуры и её свёртку. Гиломорфизм задаётся как композиция анаморфизма и катаморфизма. Промежуточная структура описывает дерево рекурсивных вызовов: коалгебра задаёт подзадачи, а алгебра собирает их результаты.

```
hylo :: Functor f => (a -> f a) -> (f b -> b) -> a -> b
hylo psi phi = cata phi . ana psi
```

Например, для вычисления факториала неотрицательного числа используем уже знакомый обратный отсчёт и алгебру произведения:

```
productAlg :: ListF Integer Integer -> Integer
productAlg = \case
  Nil -> 1
  Cons n acc -> n * acc

fac :: Integer -> Integer
fac = hylo countDown productAlg
-- fac 0 == 1; fac 5 == 120
```

При `n = 3` развёртка описывает список `[3, 2, 1]`, а свёртка заменяет пустой хвост на `1` и каждый шаг `Cons` на умножение:

```
  fac 3
⇝ cata productAlg (ana countDown 3)
⇝ productAlg (Cons 3 (productAlg (Cons 2
    (productAlg (Cons 1 (productAlg Nil))))))
⇝ 3 * (2 * (1 * 1))
⇝ 6
```

Можно ввести ещё много различных рекурсивных схем[114] и описать их свойства. Однако, мы пока остановимся.

Катаморфизм связан со структурной индукцией: результат для составного значения определяется через результаты для его подструктур. Анаморфизм связан с коиндуктивным построением: очередной наблюдаемый слой задаётся вместе с состояниями для продолжения [Pierce, 2002, глава 21].

[114] `https://wiki.haskell.org/Zygohistomorphic_prepromorphisms`

4.4 Всё через свёртки

Вместо хранения рекурсивной структуры можно описать, как свернуть её с любой выбранной алгеброй. Потребитель передаёт обработчики конструкторов, а представление структуры связывает их вызовы в нужном порядке. Так с помощью катаморфизма получаем изоморфизм между данными и их представлением через свёртку[115]:

$$\texttt{Fix f} \cong \texttt{forall r . (f r -> r) -> r}.$$

Квантор `forall r` существенен: функцию типа `(f Int -> Int) -> Int` можно применять только к алгебрам с результатом `Int`, а полиморфное представление допускает любой тип результата. В частности, выбрав `r = Fix f` и алгебру `In`, можно восстановить структуру:

```
to :: Functor f => Fix f -> (forall a . (f a -> a) -> a)
to x alg = cata alg x

from :: (forall a . (f a -> a) -> a) -> Fix f
from g = g In
```

Проверим обратимость преобразований. После кодирования и восстановления получаем `cata In x`: каждый слой собирается исходным конструктором, поэтому результат снова равен `x`. В другую сторону `to (from g) alg` даёт `cata alg (g In)`. Параметричность `g` позволяет заменить конструирование с помощью `In` и последующую свёртку непосредственным применением `g alg`. Получившаяся функция ведёт себя так же, как исходная `g`.

Например, следующие два списка эквивалентны (все конструкторы `In` заменяем на данную алгебру):

```
data ListF elem rec = Nil | Cons elem rec

xs1 :: Fix (ListF Int)
xs1 = In (Cons 1 (In (Cons 2 (In (Cons 3 (In Nil))))))

xs2 :: (ListF Int a -> a) -> a
xs2 = \alg -> alg (Cons 1 (alg (Cons 2 (alg (Cons 3 (alg Nil))))))

ghci> xs2 @Int \case Nil -> 0; Cons x acc -> x + acc
6
```

Теперь разберём, что значит передать алгебру списка. Чтобы обработать `ListF elem r`, достаточно задать значение для `Nil` и функцию для `Cons`:

$$\texttt{ListF elem r -> r} \cong \texttt{(r, elem -> r -> r)}.$$

[115] Здесь рассматриваем индуктивные структуры с обычными функторами формы, построенными из сумм и произведений, и параметрические функции. Изоморфизм понимается экстенсионально; частичные значения и операции Haskell вроде `seq` требуют дополнительных оговорок.

Вместо этой пары можно передать её компоненты отдельными аргументами. Выберем порядок, принятый у `foldr`: сначала шаг для `Cons`, затем значение для `Nil`. Получается **список Чёрча**[116]:

```
xs3 :: forall r . (Int -> r -> r) -> r -> r
xs3 = \f ini -> f 1 (f 2 (f 3 ini))
```

Одно и то же значение допускает разные интерпретации:

```
ghci> xs3 (+) 0
6
ghci> xs3 (:) []
[1,2,3]
```

Этот переход можно провести для любой рассматриваемой алгебраической формы. Запишем её как сумму произведений (см. 4.2.2):

$$f\ a \cong \sum_i \prod_j (t_{ij}\ a)$$

Тогда алгебра может быть записана следующим образом:

$$f\ a \to a \cong a^{\sum_i \prod_j (t_{ij}\ a)} \cong \prod_i a^{\prod_j (t_{ij}\ a)}$$

Каждый компонент получившегося произведения — обработчик одного конструктора. Каррирование превращает поля конструктора и набор обработчиков в отдельные аргументы функций. Внешний квантор по типу результата при этом сохраняется.

Упражнение 54 *Какая знакомая вам стандартная функция работы со списками по* `data` *списку возвращает список Чёрча?*

Попробуем интуитивно понять, что это всё значит. Заметим, что список Чёрча принимает функции, соответствующие веткам паттерн-матчинга или аргументам сворачивающей функции. Вместо конструкторов же сразу вызываются соответствующие функции. То есть, вместо того, чтобы создать структуру данных и доставить её к месту деконструирования (паттерн-матчингу), мы *доставляем место деконструирования к месту конструирования*. Например, `xs3 (+) 0` сразу связывает сложения, а `xs3 (:) []` связывает вызовы конструкторов списка. Само функциональное представление может хранить данные в замыканиях; при совмещении построения с потреблением промежуточное представление иногда удаётся устранить полностью.

Мы уже работали с обратным процессом, дефункционализацией, когда функции первого класса превращались в вызовы конструкторов, а их тела — в ветки паттерн-матчинга 3.1.7.

[116] Типизированное обобщение этого представления для индуктивных типов называют кодированием Бёма–Берардуччи: `https://okmij.org/ftp/tagless-final/course/Boehm-Berarducci.html`.

Можно заметить, что тут мы имеем дело с обратным процессом, **рефункционализацией**, когда вместо вызовов конструкторов сразу вызывается соответствующий интерпретирующий код Danvy and Millikin [2009].

В таком преобразовании выбор ветки паттерн-матчинга заменяется вызовом соответствующего обработчика, который может быть замыканием или методом интерфейса. Результат вычисления сохраняется, но стоимость вызовов и размещение данных зависят от реализации и оптимизаций.

Стоит также отметить, что изоморфизм `Fix f ≃ forall d . (f d -> d) -> d` является обобщением изоморфизма `a ≃ forall r . (a -> r) -> r` (при `f = Const a`), который лежит в основе CPS-трансформации, рассматриваемой нами далее в главе 6.

4.4.1 Deforestation & list fusion

В функциональном программировании мы строим новые функции путём композирования имеющихся, более простых, функций. Этот подход позволяет переиспользовать реализованную функциональность, снижая сложность кода и вероятность ошибок. Однако, он может приводить к излишним накладным расходам на аллокацию и деконструирование промежуточных структур данных.

Например, сравните следующие две реализации. Первая переиспользует готовые функции, но без fusion создаёт промежуточный список по мере потребления:

```
allComposed p xs = and (map p xs)
-- или fused версия
allFused p = \case
  [] -> True
  x:xs -> p x && allFused p xs
```

Очевидно, это задача оптимизирующего компилятора превращать хороший код с абстракциями в быстрый код. Оптимизация, избавляющая программы от (промежуточных) структур данных (деревьев) называется **дефорестацией (deforestation)**. В результате две функции, как говорят, “сплавляются” — fuse.

Термин и алгоритм дефорестации предложил Philip Wadler Wadler [1988]. Раскрытие определений и переписывание выражений позволяют получить ситуацию `case K args of ...`, в которой построение и разбор конструктора устраняются вместе (рис. 18). Бесконтрольное раскрытие рекурсивных определений может не завершаться и увеличивать код; для алгоритма Уодлера завершение доказывается на ограниченном классе программ в форме `treeless`[117].

Сфокусируемся на списках и рассмотрим более практичные решения[118]. Первым очевидным решением было бы для каждой пары функций, работающих со списками, добавить специальное правило переписывания (см. 3.1.6), дефорестирующее с помощью алгебраических

[117] Дефорестация связана с более общими методами преобразования программ, в частности суперкомпиляцией Романенко.

[118] `https://markkarpov.com/tutorial/ghc-optimization-and-fusion.html`

append (*append xs ys*) *zs*

transforms to

```
h0 xs ys zs
where
h0 xs ys zs  =  case xs of
                   Nil        :  h1 ys zs
                   Cons x xs  :  Cons x (h0 xs ys zs)
h1 ys zs     =  case ys of
                   Nil        :  zs
                   Cons y ys  :  Cons y (h1 ys zs)
```

---

flip (*flip zt*)

transforms to

```
h0 zt
where
h0 zt  =  case zt of
             Leaf z          :  Leaf z
             Branch xt yt    :  Branch (h0 xt) (h0 yt)
```

Рис. 18: Примеры работы дефорестирующего алгоритма Wadler [1988].

свойств списочных трансформаций. Для n функций имеется до n^2 упорядоченных пар; отдельные правила для сочетаний плохо масштабируются и требуют сопровождения.

```
{-# RULES
"map/map" forall f g xs. map f (map g xs) = map (f . g) xs
  #-}
```

Техника foldr/build list fusion Gill et al. [1993] определяет общий способ конструирования и деконструирования списка, позволяющий применять одно правило ко многим сочетаниям функций. Деконструировать список будем с помощью `foldr`. Конструировать будем с помощью функции `build`:

```
build :: (forall b . (a -> b -> b) -> b -> b) -> [a]
build g = g (:) []
```

Например, список `[1, 2, 3]` и функция `map` теперь запишутся следующим образом[119]:

```
list123 :: [Int]
list123 = build \s z -> s 1 (s 2 (s 3 z))

map :: (a -> b) -> [a] -> [b]
map f xs = build \s z -> foldr (\x acc -> s (f x) acc) z xs
```

На вход `build` передаётся список Чёрча. Квантор `forall b` гарантирует, что производитель параметричен по типу результата: он связывает переданные шаг и базу, поэтому вместо построения списка с последующей свёрткой можно сразу передать нужные обработчики[120] Gill et al. [1993]:

```
foldr f ini (build g) ≡ g f ini
```

Вернёмся к примеру `and (map p xs)`. Раскроем `and` через `foldr`, а `map` — через `build`, затем применим правило:

```
  and (map p xs)
= foldr (&&) True
    (build (\c z -> foldr (\x r -> c (p x) r) z xs))
= (\c z -> foldr (\x r -> c (p x) r) z xs) (&&) True
= foldr (\x r -> p x && r) True xs
```

Последняя строка при раскрытии `foldr` даёт `allFused` из начала раздела. В обходе больше нет конструкторов промежуточного списка булевых значений: результат `p x` сразу передаётся в `(&&)`. Ленивость второго аргумента `(&&)` сохраняет остановку при первом `False`.

Дефорестацию также можно производить вручную (см. 19). Пусть каждое сообщение со-

[119] В стандартной библиотеке Haskell функции работы со списками написаны нормально, но рядом написаны правила `RULES` (см. 3.1.6), которые подменяют их реализацию на fold/build.

[120] Это следствие параметричности, а не только раскрытия определений. Для Haskell правило требует корректного производителя: использование `seq` для исследования строгости переданных обработчиков может нарушить закон.

```
std::variant<Msg1, Msg2>
    deserialize(bytes bs) {
    if (...) {
        return Msg1(...);
    } else {
        return Msg2(...);
    }
}
```

```
template<class Impl>
auto deserialize(bytes bs) {
    if (...) {
        return Impl::processMsg1(...);
    } else {
        return Impl::processMsg2(...);
    }
}
```

Рис. 19: Ручная дефорестация в C++.

держит одно целочисленное поле, которое парсер передаёт в `processMsg1` или `processMsg2`. Тогда вместо создания `variant` можно сразу вычислить результат:

```
struct SignedValue {
    static int processMsg1(int value) { return value; }
    static int processMsg2(int value) { return -value; }
};
// bs -- входные байты; оба обработчика возвращают int
int result = deserialize<SignedValue>(bs);
```

Парсер выбирает обработчик по виду сообщения; параметр шаблона задаёт эти обработчики при компиляции.

4.4.2 Visitor pattern

Рассмотрим некоторое дерево и его свёртку:

```
data Tree a = Leaf | Node a [Tree a]
foldTree :: Tree a -> r -> (a -> [r] -> r) -> r
foldTree tree z f = case tree of
  Leaf -> z
  Node x ts -> f x (map (\t -> foldTree t z f) ts)
```

Перепишем и переименуем:

```
data Visitor a r = Visitor { onLeaf :: r, onNode :: a -> [r] -> r }
visitTree :: Tree a -> Visitor a r -> r
visitTree t (Visitor z f) = foldTree t z f
```

Теперь представим само дерево через свёртку: интерфейс содержит функцию `visit`, а конструкторы задаются её реализациями:

```haskell
data Tree a = Tree { visit :: forall r . Visitor a r -> r }

leaf :: Tree a
leaf = Tree { visit = \Visitor{onLeaf} -> onLeaf }

node :: a -> [Tree a] -> Tree a
node x ts = Tree
  { visit = \v@Visitor{onNode} -> onNode x (map (\t -> visit t v) ts) }
```

Определим два visitor'а для одного дерева:

```haskell
sumVisitor :: Visitor Int Int
sumVisitor = Visitor 0 (\x rs -> x + sum rs)

heightVisitor :: Visitor a Int
heightVisitor = Visitor 0 (\_ rs -> 1 + maximum (0 : rs))

exampleTree :: Tree Int
exampleTree = node 1 [leaf, node 2 [], node 3 [leaf]]
-- visit exampleTree sumVisitor    == 6
-- visit exampleTree heightVisitor == 2
```

В `onNode` приходят результаты для детей: суммы в первом случае и высоты во втором. Так одна структура допускает разные интерпретации без изменения её представления.

Для наглядности, покажем этот код ещё и на Kotlin:

```kotlin
interface Visitor<a, r> {
    fun onLeaf(): r
    fun onNode(x: a, subtrees: List<r>): r
}

interface Tree<a> {
    fun <r> visit(visitor: Visitor<a, r>): r
}
class Leaf<a> : Tree<a> {
    override fun <r> visit(visitor: Visitor<a, r>): r = visitor.onLeaf()
}
class Node<a>(val value: a, val subtrees: List<Tree<a>>) : Tree<a> {
    override fun <r> visit(visitor: Visitor<a, r>): r =
        visitor.onNode(value, subtrees.map { it -> it.visit(visitor) })
}
```

Применение visitor'а выглядит так:

```kotlin
val tree: Tree<Int> = Node(1, listOf(Leaf<Int>(), Node(2, listOf())))
val total = tree.visit(object : Visitor<Int, Int> {
    override fun onLeaf(): Int = 0
    override fun onNode(x: Int, subtrees: List<Int>): Int =
        x + subtrees.sum()
}) // total == 3
```

Здесь показан вариант Visitor, реализующий свёртку. Другие варианты передают visitor'у сам узел, позволяя ему управлять обходом; наш `onNode` получает уже результаты обработки поддеревьев. В частности, в Kotlin вызов `subtrees.map` вычисляет все результаты для детей до вызова `onNode`, поэтому этот интерфейс не позволяет visitor'у пропустить их обход. Само дерево при этом может по-прежнему храниться в объектах `Leaf` и `Node`; представление через Visitor ещё не гарантирует устранение этих объектов.

To be continued...

Список литературы

Christopher Strachey. Fundamental concepts in programming languages. *Higher-order and symbolic computation*, 13:11–49, 2000. URL https://facweb.cdm.depaul.edu/smitsch/courses/csc447fa23/assets/articles/strachey-fundamental-concepts-in-programming-languages.pdf.

Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys (CSUR)*, 17(4):471–523, 1985. URL https://doi.org/10.1145/6041.6042.

Benjamin C Pierce. *Types and programming languages*. MIT press, 2002.

John Launchbury and Simon L Peyton Jones. State in haskell. *Lisp and symbolic computation*, 8(4): 293–341, 1995. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/state-lasc.pdf.

Sandy Maguire. *Thinking with Types, Type-Level Programming in Haskell*. a. URL https://leanpub.com/thinking-with-types/?

Simon Peyton Jones. Type inference as constraint solving: how ghc's type inference engine actually works. Keynote talk at Zurihac 2019, 2019. URL https://simon.peytonjones.org/type-inference/.

Benjamin C Pierce and David N Turner. Local type inference. *Acm transactions on programming languages and systems (toplas)*, 22(1):1–44, 2000. URL https://doi.org/10.1145/345099.345100.

David Raymond Christiansen. Bidirectional typing rules: A tutorial. 2013. URL https://davidchristiansen.dk/tutorials/bidirectional.pdf.

Jana Dunfield and Neelakantan R Krishnaswami. Sound and complete bidirectional typechecking for higher-rank polymorphism with existentials and indexed types. *Proceedings of the ACM on Programming Languages*, 3(POPL):1–28, 2019. URL https://doi.org/10.1145/3290322.

Alejandro Serrano, Jurriaan Hage, Simon Peyton Jones, and Dimitrios Vytiniotis. A quick look at impredicativity. *Proceedings of the ACM on Programming Languages*, 4 (ICFP):1–29, 2020. URL https://www.microsoft.com/en-us/research/publication/a-quick-look-at-impredicativity/.

Frank Emrich, Sam Lindley, Jan Stolarek, James Cheney, and Jonathan Coates. Freezeml: Complete and easy type inference for first-class polymorphism. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 423–437, 2020. URL https://link.springer.com/article/10.1208/s12249-010-9382-3.

Brent A Yorgey, Stephanie Weirich, Julien Cretin, Simon Peyton Jones, Dimitrios Vytiniotis, and José Pedro Magalhães. Giving haskell a promotion. In *Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation*, pages 53–66, 2012. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/p53-yorgey.pdf.

Stephanie Weirich, Justin Hsu, and Richard A Eisenberg. System fc with explicit kind equality. *ACM SIGPLAN Notices*, 48(9):275–286, 2013. URL https://doi.org/10.1145/2544174.2500599.

Vitaly Bragilevsky. *Haskell in Depth*. Manning. URL https://www.manning.com/books/haskell-in-depth.

Bruno Blanchet. Escape analysis for object-oriented languages: application to java. *Acm Sigplan Notices*, 34(10):20–34, 1999. URL https://doi.org/10.1145/320385.320387.

Richard A Eisenberg and Simon Peyton Jones. Levity polymorphism. *ACM SIGPLAN Notices*, 52 (6):525–539, 2017. URL https://doi.org/10.1145/3140587.3062357.

Paul Hudak, John Hughes, Simon Peyton Jones, and Philip Wadler. A history of haskell: being lazy with class. In *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, pages 12–1, 2007. URL https://www.researchgate.net/profile/Simon-Peyton-Jones/publication/221501761_A_history_of_Haskell_Being_lazy_with_class/links/0c960517e31f50f743000000/A-history-of-Haskell-Being-lazy-with-class.pdf.

Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 60–76, 1989. URL https://doi.org/10.1145/75277.75283.

Cordelia V Hall, Kevin Hammond, Simon L Peyton Jones, and Philip L Wadler. Type classes in haskell. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 18(2):109–138, 1996. URL https://doi.org/10.1145/227699.227700.

Simon Peyton Jones, Mark Jones, and Erik Meijer. Type classes: an exploration of the design space. In *In Haskell Workshop*, 1997. URL https://courses.cs.washington.edu/courses/cse590p/06sp/multi.pdf.

Jeffrey R Lewis, John Launchbury, Erik Meijer, and Mark B Shields. Implicit parameters: Dynamic scoping with static types. In *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 108–118, 2000. URL https://doi.org/10.1145/325694.325708.

Martin Sulzmann, Gregory J Duck, Simon Peyton-Jones, and Peter J Stuckey. Understanding functional dependencies via constraint handling rules. *Journal of functional programming*, 17(1): 83–129, 2007a. URL https://doi.org/10.1017/S0956796806006137.

Oleg Kiselyov and Chung-chieh Shan. Functional pearl: implicit configurations–or, type classes reflect the values of types. In *Proceedings of the 2004 ACM SIGPLAN workshop on Haskell*, pages 33–44, 2004. URL https://d1wqtxts1xzle7.cloudfront.net/43582096/Functional_pearl_implicit_configurations20160310-32037-1bu6179-libre.pdf.

Filip Křikava, Heather Miller, and Jan Vitek. Scala implicits are everywhere: A large-scale study of the use of scala implicits in the wild. *Proceedings of the ACM on Programming Languages*, 3 (OOPSLA):1–28, 2019. URL https://doi.org/10.1145/3360589.

Bruno CdS Oliveira, Adriaan Moors, and Martin Odersky. Type classes as objects and implicits. *ACM Sigplan Notices*, 45(10):341–360, 2010. URL https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=d30d65ca9ce7891352024a5c71ebe0ae8c41f7ac.

Dominique Devriese and Frank Piessens. On the bright side of type classes: instance arguments in agda. *ACM SIGPLAN Notices*, 46(9):143–155, 2011. URL https://archive.alvb.in/msc/thesis/reading/typeclasses-agda_Devriese.pdf.

Conor McBride. Faking it simulating dependent types in haskell. *Journal of functional programming*, 12(4-5):375–392, 2002. URL https://doi.org/10.1017/S0956796802004355.

Simon Peyton Jones, Andrew Tolmach, and Tony Hoare. Playing by the rules: rewriting as a practical optimisation technique in ghc. In *Haskell workshop*, volume 1, pages 203–233, 2001. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2001/09/rules.pdf.

Sandy Maguire. *Algebra-Driven Design, Elegant Solutions from Simple Building Blocks*. b. URL https://leanpub.com/algebra-driven-design/.

Li-yao Xia. *Defunctionalization*. URL https://poisson.chat/aquarium/defunctionalization.pdf.

John C Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM annual conference-Volume 2*, pages 717–740, 1972. URL https://doi.org/10.1145/800194.805852.

John C Reynolds. Definitional interpreters revisited. *Higher-Order and Symbolic Computation*, 11: 355–361, 1998. URL https://doi.org/10.1023/A:1010075320153.

Jeremy Yallop and Leo White. Lightweight higher-kinded polymorphism. In *International Symposium on Functional and Logic Programming*, pages 119–135. Springer, 2014. URL https://www.cl.cam.ac.uk/~jdy22/papers/lightweight-higher-kinded-polymorphism.pdf.

Manuel MT Chakravarty, Gabriele Keller, and Simon Peyton Jones. Associated type synonyms. *ACM SIGPLAN Notices*, 40(9):241–253, 2005a. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2005/01/at-syns.pdf.

Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. Associated types with class. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–13, 2005b. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2005/01/assoc.pdf.

Tom Schrijvers, Simon Peyton Jones, Manuel Chakravarty, and Martin Sulzmann. Type checking with open type functions. In *Proceedings of the 13th ACM SIGPLAN international conference on Functional programming*, pages 51–62, 2008. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2008/01/icfp2008.pdf.

Richard A Eisenberg, Dimitrios Vytiniotis, Simon Peyton Jones, and Stephanie Weirich. Closed type families with overlapping equations. *ACM SIGPLAN Notices*, 49(1):671–683, 2014. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/popl137-eisenberg.pdf.

Oleg Kiselyov, Simon Peyton Jones, and Chung-chieh Shan. Fun with type functions. *Reflections on the Work of CAR Hoare*, pages 301–331, 2010. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/typefun.pdf?from=https://research.microsoft.com/~simonpj/papers/assoc-types/fun-with-type-funs/typefun.pdf&type=exact.

Mark P Jones. Type classes with functional dependencies. In *European Symposium on Programming*, pages 230–244. Springer, 2000. URL https://doi.org/10.1007/3-540-46425-5_15.

Nada Amin, Tiark Rompf, and Martin Odersky. Foundations of path-dependent types. *Acm Sigplan Notices*, 49(10):233–249, 2014. URL http://lampwww.epfl.ch/~amin/dot/fpdt_post.pdf.

Jan Stolarek, Simon Peyton Jones, and Richard A Eisenberg. Injective type families for haskell. *ACM SIGPLAN Notices*, 50(12):118–128, 2015. URL https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1070&context=compsci_pubs.

Richard A Eisenberg and Jan Stolarek. Promoting functions to type families in haskell. *ACM SIGPLAN Notices*, 49(12):95–106, 2014. URL https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1000&context=compsci_pubs.

Csongor Kiss, Tony Field, Susan Eisenbach, and Simon Peyton Jones. Higher-order type-level programming in haskell. *Proceedings of the ACM on Programming Languages*, 3(ICFP):1–26, 2019. URL https://dl.acm.org/doi/pdf/10.1145/3341706.

Dominic Orchard and Tom Schrijvers. Haskell type constraints unleashed. In *International Symposium on Functional and Logic Programming*, pages 56–71. Springer, 2010. URL https://kar.kent.ac.uk/57498/1/constraint-families.pdf.

Edsko de Vries and Andres Löh. True sums of products. In *Proceedings of the 10th ACM SIGPLAN workshop on Generic programming*, pages 83–94, 2014. URL https://doi.org/10.1145/2633628.2633634.

Gert-Jan Bottu, Georgios Karachalias, Tom Schrijvers, Bruno C d S Oliveira, and Philip Wadler. Quantified class constraints. *ACM SIGPLAN Notices*, 52(10):148–161, 2017. URL https://www.pure.ed.ac.uk/ws/portalfiles/portal/42495988/quantcc.pdf.

Martin Sulzmann, Manuel MT Chakravarty, Simon Peyton Jones, and Kevin Donnelly. System f with type equality coercions. In *Proceedings of the 2007 ACM SIGPLAN international workshop on Types in languages design and implementation*, pages 53–66, 2007b. URL https://doi.org/10.1145/1190315.1190324.

Dimitrios Vytiniotis, Simon Peyton Jones, Tom Schrijvers, and Martin Sulzmann. Outsidein (x) modular type inference with local assumptions. *Journal of functional programming*, 21(4-5): 333–412, 2011. URL https://doi.org/10.1017/S0956796811000098.

Joachim Breitner, Richard A Eisenberg, Simon Peyton Jones, and Stephanie Weirich. Safe zero-cost coercions for haskell. In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*, pages 189–202, 2014. URL https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1013&context=compsci_pubs.

Simon Peyton Jones, Stephanie Weirich, Richard A Eisenberg, and Dimitrios Vytiniotis. A reflection on types. In *A List of Successes That Can Change the World: Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday*, pages 292–317. Springer, 2016. URL https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1002&context=compsci_pubs.

Simon Marlow, Simon Peyton Jones, Andrew Moran, and John Reppy. Asynchronous exceptions in haskell. In *Proceedings of the ACM SIGPLAN 2001 conference on Programming language design and implementation*, pages 274–285, 2001. URL https://classes.cs.uchicago.edu/archive/2007/spring/32102-1/papers/p274-marlow.pdf.

Simon Marlow. An extensible dynamically-typed hierarchy of exceptions. In *Proceedings of the 2006 ACM SIGPLAN Workshop on Haskell*, pages 96–106, 2006. URL https://archive.alvb.in/msc/03_infoafp/papers/2012-12-18_HoorCollege_MarlowExtensibleExceptions_dk.pdf.

Simon Peyton Jones. Tackling the awkward squad: monadic input/output, concurrency, exceptions, and foreign-language calls in haskell. *NATO SCIENCE SERIES SUB SERIES III COMPUTER AND SYSTEMS SCIENCES*, 180:47–96, 2001. URL https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=2e6c9d76f9cb690dc18019fc894ba9572a8c2812.

Simon Marlow. Parallel and concurrent programming in haskell. In *Central European Functional Programming School*, pages 339–401. Springer, 2011.

URL https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=bd127ea1952996864c1542e1453973a78973ad5c.

Ralf Hinze, Jennifer Hackett, and Daniel WH James. Functional pearl: F for functor. 2012. URL www.cs.ox.ac.uk/people/daniel.james/functor/functor.pdf.

Ralf Hinze and Daniel WH James. Reason isomorphically! In *Proceedings of the 6th ACM SIGPLAN workshop on Generic programming*, pages 85–96, 2010. URL http://www.cs.ox.ac.uk/people/ralf.hinze/publications/WGP10.pdf.

Gérard Huet. The zipper. *Journal of functional programming*, 7(5):549–554, 1997. URL https://doi.org/10.1017/S0956796897002864.

Conor McBride. The derivative of a regular type is its type of one-hole contexts. *Unpublished manuscript*, pages 74–88, 2001. URL https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=7de4f6fddb11254d1fd5f8adfd67b6e0c9439eaa.

Michael Abbott, Thorsten Altenkirch, Neil Ghani, and Conor McBride. Derivatives of containers. In *International Conference on Typed Lambda Calculi and Applications*, pages 16–30. Springer, 2003. URL http://www.strictlypositive.org/derivcont.pdf.

Erik Meijer, Maarten Fokkinga, and Ross Paterson. Functional programming with bananas, lenses, envelopes and barbed wire. In *Conference on functional programming languages and computer architecture*, pages 124–144. Springer, 1991. URL https://ris.utwente.nl/ws/portalfiles/portal/6142049/meijer91functional.pdf.

Erik Meijer and Graham Hutton. Bananas in space: Extending fold and unfold to exponential types. In *Proceedings of the seventh international conference on Functional programming languages and computer architecture*, pages 324–333, 1995. URL https://dl.acm.org/doi/pdf/10.1145/224164.224225.

Olivier Danvy and Kevin Millikin. Refunctionalization at work. *Science of Computer Programming*, 74(8):534–549, 2009. ISSN 0167-6423. doi: https://doi.org/10.1016/j.scico.2007.10.007. URL https://www.sciencedirect.com/science/article/pii/S0167642309000227. Special Issue on Mathematics of Program Construction (MPC 2006).

Philip Wadler. Deforestation: Transforming programs to eliminate trees. In *European Symposium on Programming*, pages 344–358. Springer, 1988. URL https://link.springer.com/content/pdf/10.1007/3-540-19027-9_23.pdf.

СА Романенко. ВЫЯВЛЕНИЕ И ДОКАЗАТЕЛЬСТВО СВОЙСТВ ФУНКЦИОНАЛЬНЫХ ПРОГРАММ МЕТОДАМИ СУПЕРКОМПИЛЯЦИИ. URL https://www.keldysh.ru/council/1/klyuchnikov-diss.pdf.

Andrew Gill, John Launchbury, and Simon L Peyton Jones. A short cut to deforestation. In *Proceedings of the conference on Functional programming languages and computer architecture*, pages 223–232, 1993. URL https://dl.acm.org/doi/pdf/10.1145/165180.165214.